

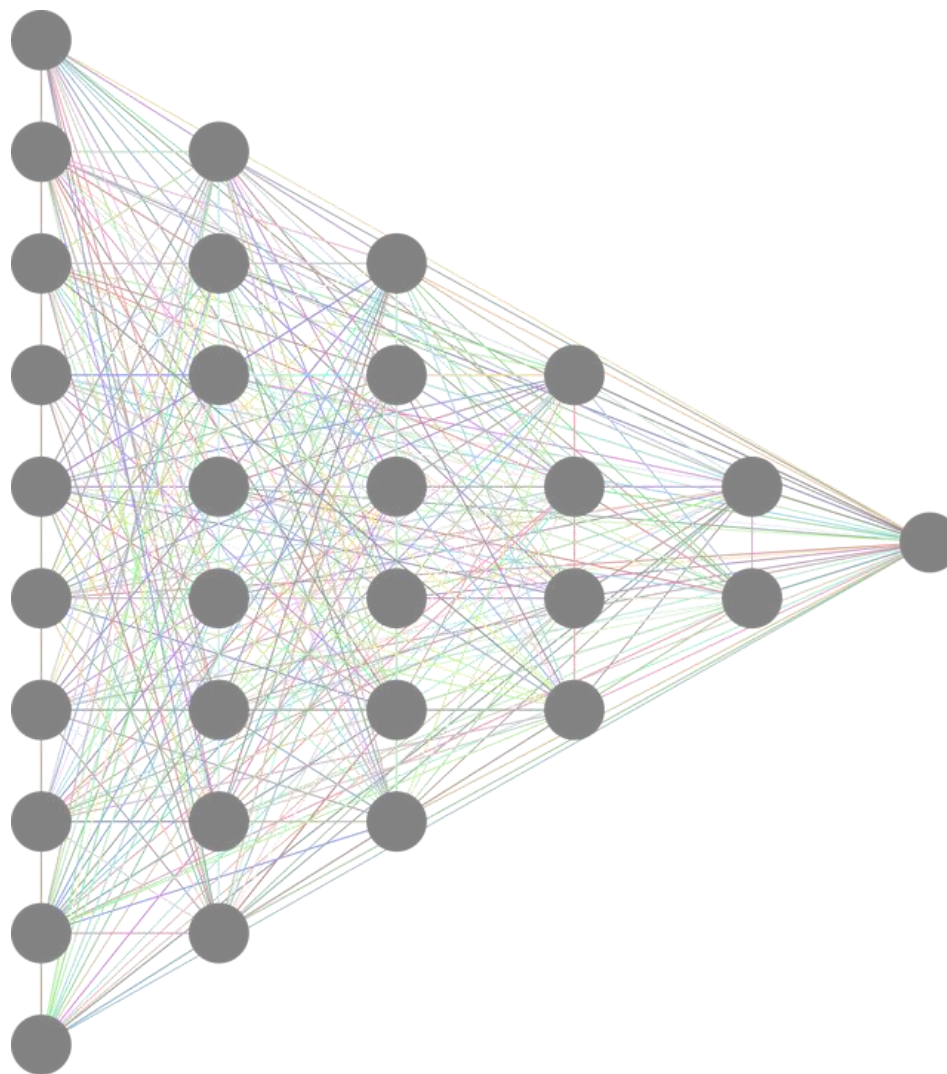
# **Computer Vision**

**2026. 2. 23**

**3C AI Campus**  
**박 준 오**

# III. Transfer Learning

---



## II. 이미지 전처리(OpenCV)

| 데이터셋 유형 | 전처리 기법 1    | 전처리 기법 2          | 전처리 기법 3            |
|---------|-------------|-------------------|---------------------|
| 도로 표지판  | 특정 색상 채널 강조 | 기하학적 변환(회전 등)     | 밝기 변화(Augmentation) |
| 실내 물체   | Canny 엣지 검출 | 대비 조정             | 조명 조건 변화            |
| 과일      | HSV 변환      | Gaussian 블러 후 샤프닝 | 색온도 조절              |
| 애완동물    | 대비 조정       | 배경 제거(세그멘테이션)     | 히스토그램 평활화           |
| 꽃       | 특정 채널 강조    | 색온도 조절            | 다양한 조명 조건 적용        |
| 자동차     | 그레이스케일 변환   | 노이즈 추가 및 제거       | 배경 블러(배경 흐림)        |
| 음식      | 블러 후 윤곽 강조  | 색 대비 조정           | 색감 선명화              |
| 문서      | 이진화 처리      | 기울기 보정(기하학적 교정)   | 텍스트 대비 강화           |
| 인물      | 히스토그램 평활화   | 얼굴 랜드마크 정규화       | 밝기 균등화              |
| 가전 제품   | 노이즈 추가 및 제거 | 배경 블러             | 색상 대비 조절            |

## II. Deep Learning **LeNet**

---

**LeNet-5**  
**(1998)**

## II. Deep Learning **LeNet**

### 등장 배경:

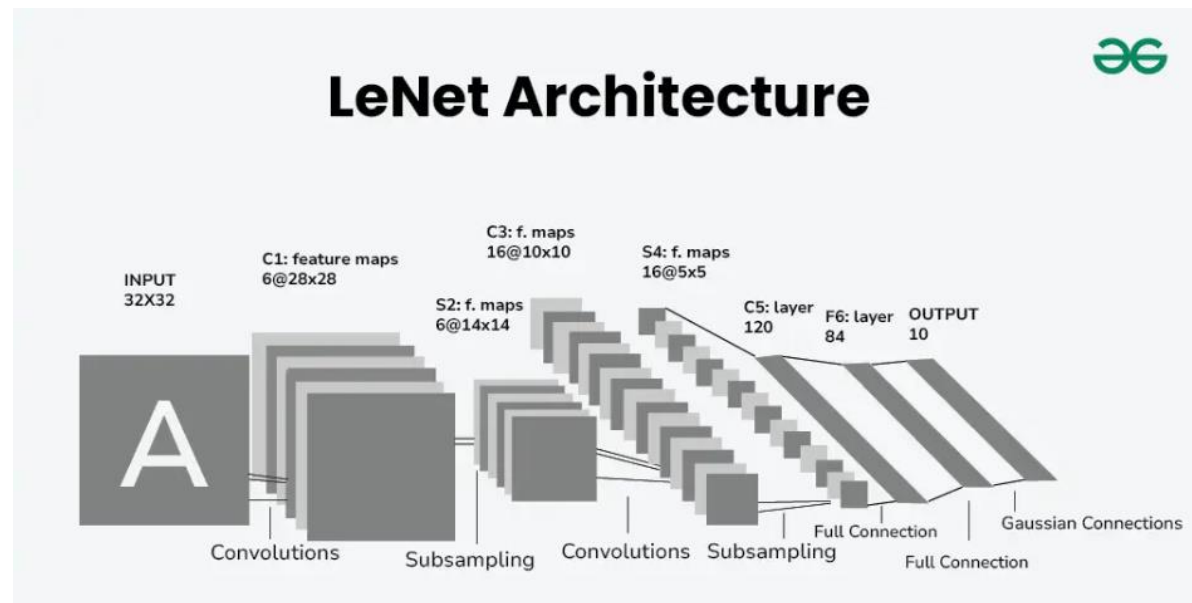
손글씨 숫자 인식 (MNIST) 문제 해결을 위해 개발됨. 당시로서는 혁신적인 구조지만, 컴퓨팅 파워 부족과 대량 데이터 부족으로 제한적 사용.

### 기술적 특이점:

초기 **CNN 구조의 대표: Conv → Pool → FC 구조 확립**

Activation 함수로 **tanh 사용**

이미지에서 특징을 추출하고 분류하는 계층 구조 개념 정립



## II. Deep Learning **AlexNet**

---

**AlexNet**

**(2012, ImageNet 대회 우승)**

## II. Deep Learning **AlexNet**

### 등장 배경:

GPU의 발전, 대용량 데이터셋(ImageNet)의 등장

기존 전통적인 머신러닝 기법(SVM + hand-crafted features)을 압도

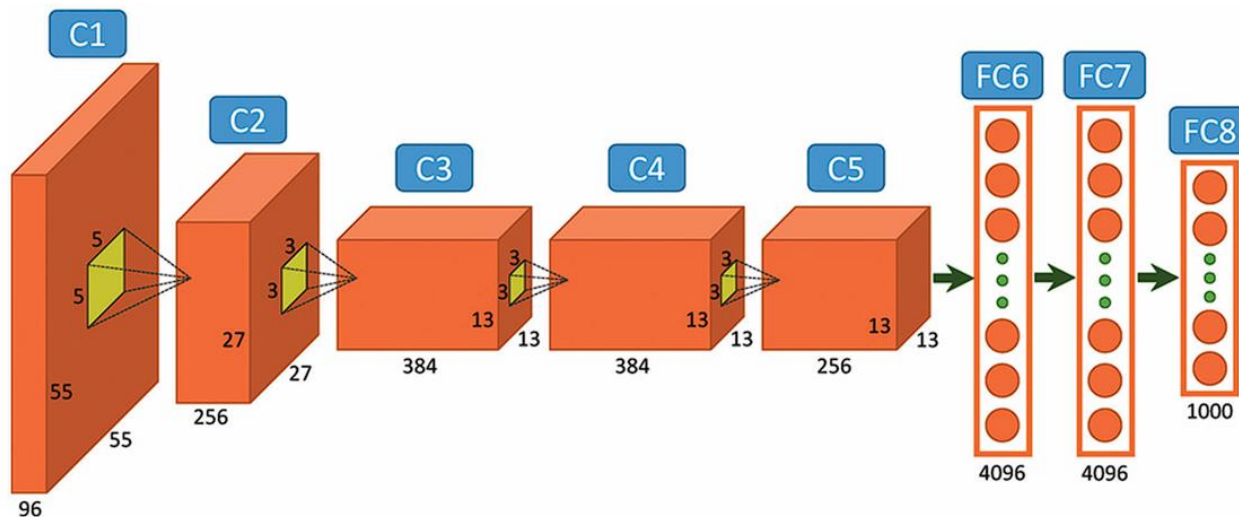
### 기술적 특이점:

ReLU 도입으로 학습 속도 대폭 향상

GPU 병렬 연산 (당시 GTX 580 두 개 사용)

Dropout 도입 → 과적합 방지

CNN이 비전 분야를 지배하는 시작점



## II. Deep Learning **VGGNet**

---

**VGGNet**  
**(2014)**

## II. Deep Learning **VGGNet**

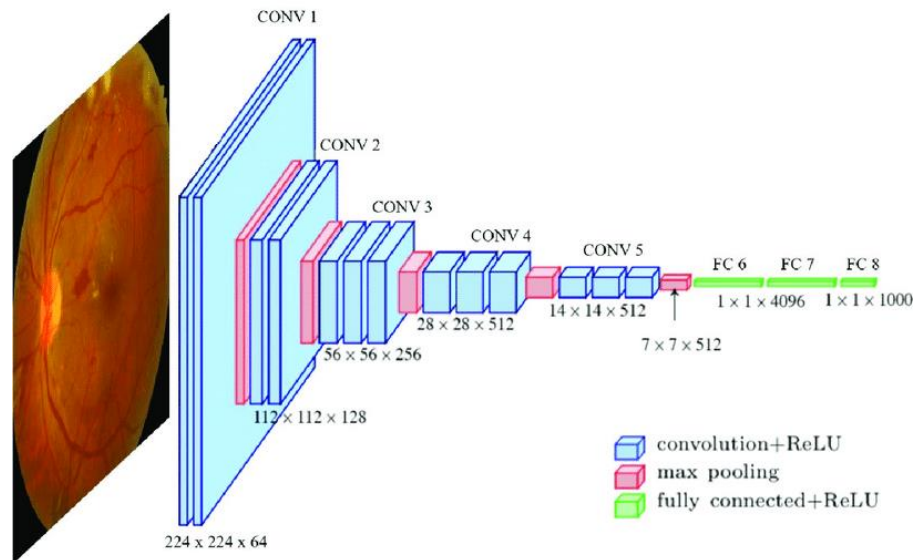
### 등장 배경:

AlexNet 이후, 단순하지만 **깊은 구조가 성능 향상에 중요하다**는 가설을 실험

### 기술적 특이점:

3x3 필터만 사용, **깊이 증가(16~19 layers)**

단순하고 반복적인 구조 → 하드웨어 최적화 용이  
파라미터 수는 많지만, 구조가 정형화되어 모듈화에 용이



## II. Deep Learning **Inception**

---

**Inception**  
**(GoogLeNet, 2014)**

## II. Deep Learning Inception

### 등장 배경:

VGG는 깊이 증가로 인한 연산량 증가 문제가 있었음

다양한 필터 크기를 병렬로 사용하는 아이디어로 정보 손실 방지 + 계산 효율화

### 기술적 특이점:

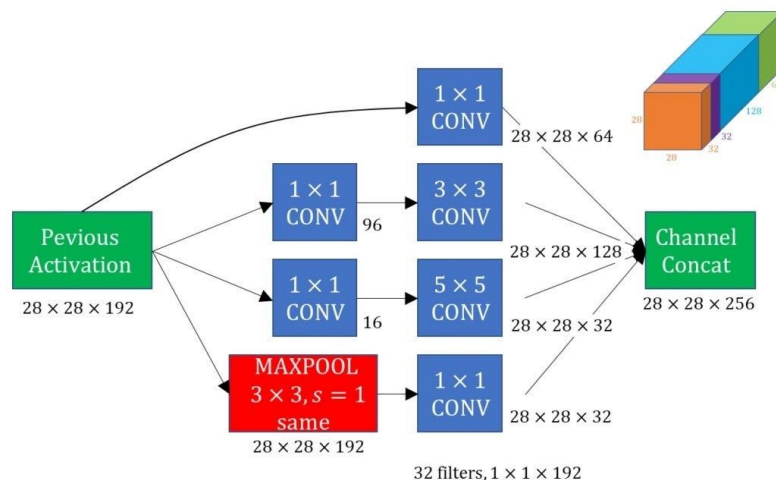
Inception 모듈: 1x1, 3x3, 5x5 컨볼루션을 병렬로 수행

연산량 줄이기 위해 1x1 conv로 차원 축소

Network-in-Network 개념 적용

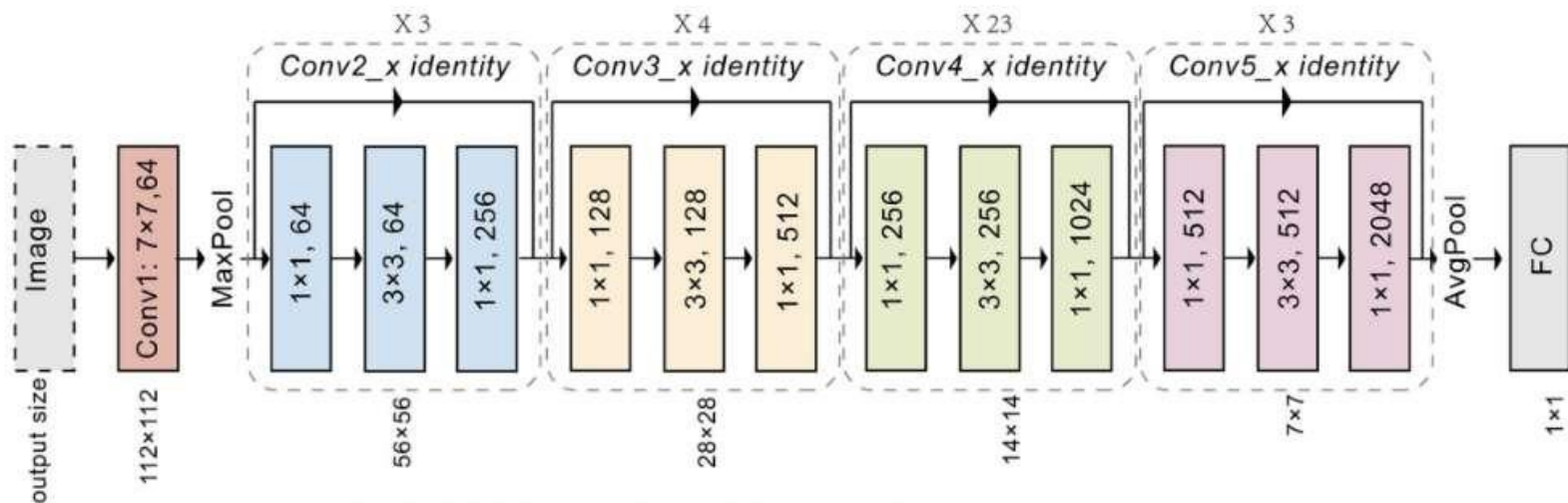
깊고 넓은 네트워크 구조 설계 가능

### An example of an Inception module



## II. Deep Learning **ResNet**

# ResNet (Residual Network)



## II. Deep Learning ResNet

### 2. ResNet 학습 (Residual Network)

#### 2.1 ResNet의 등장 배경

딥러닝의 깊이가 증가할수록 기울기 소실(Vanishing Gradient) 문제 발생  
깊은 네트워크가 오히려 성능 저하 (degradation) 현상 발생  
“이전보다 더 깊은 네트워크가 왜 더 못하냐?”는 문제의식

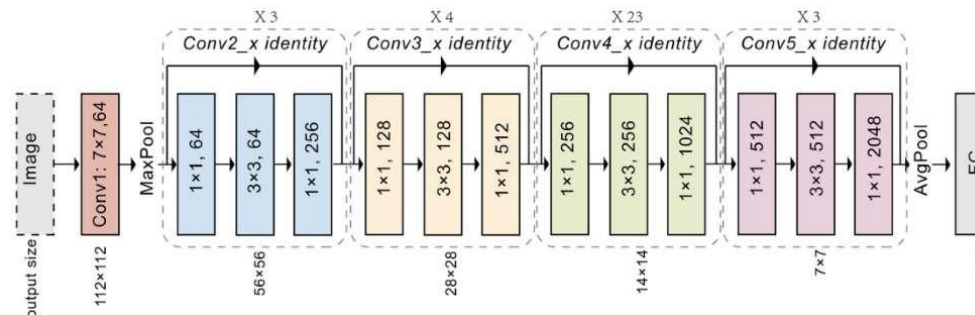
#### 기술적 특이점:

Residual Learning 도입:  $y=F(x)+x$

학습할 함수를 직접 예측하는 게 아니라 잔차(residual)를 학습

매우 깊은 구조(152 layers 이상)도 안정적으로 학습 가능

이후 모든 모델의 표준 설계 요소가 됨



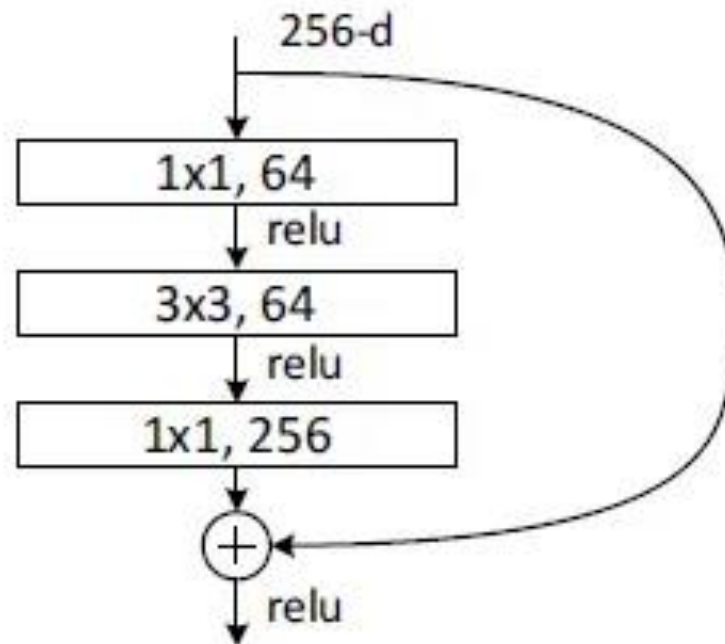
## II. Deep Learning ResNet

---

### 2.2 ResNet 내부 구조 분석

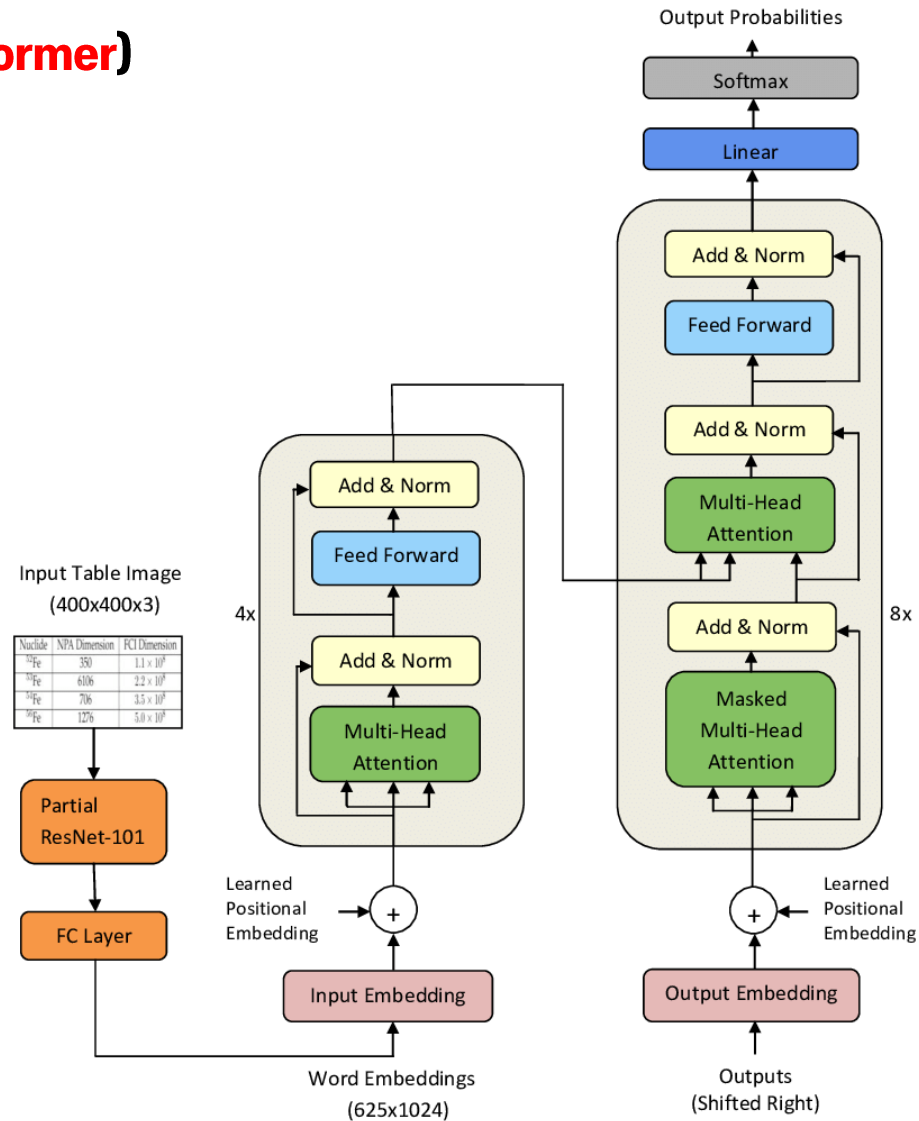
#### Residual Block의 동작 방식

$F(x) + x$  형태의 스킵 연결 (Skip Connection)



# II. Deep Learning ResNet

## 2.3 ResNet 응용(Transformer)



# II. Deep Learning (ResNet)

[GET STARTED](#)[GUIDES](#)[API](#)[EXAMPLES](#)[KERAS TUNER](#)[KERAS RS](#)[KERAS HUB](#)

## Keras 3 API documentation

[Models API](#)[Layers API](#)[Callbacks API](#)[Ops API](#)[Optimizers](#)[Metrics](#)[Losses](#)[Data loading](#)[Built-in small datasets](#)

## ResNet and ResNetV2

### ResNet50 function

[\[source\]](#)

```
keras.applications.ResNet50(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
    classifier_activation="softmax",  
    name="resnet50",  
)
```

Instantiates the ResNet50 architecture.

### ResNet and ResNetV2

- [ResNet50 function](#)
- [ResNet101 function](#)
- [ResNet152 function](#)
- [ResNet50V2 function](#)
- [ResNet101V2 function](#)
- [ResNet152V2 function](#)

# **III.Transfer Learning**

---

## **III.Transfer Learning**

# III. Transfer Learning

## 전이 학습(Transfer Learning)

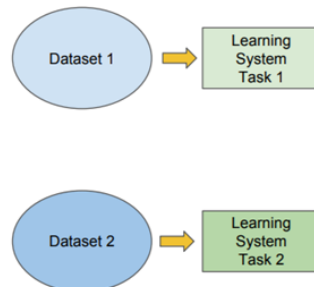
'한 분야의 문제를 해결하기 위해서 얻은 지식과 정보를 다른 문제를 푸는데 사용하는 방식'

딥러닝의 '이미지 분류' 문제를 해결하는데 사용했던 신경망을 다른 데이터셋 혹은 다른 문제(task)에 적용시켜 푸는 것을 의미.

ImageNet으로 학습시킨 CNN구조는 다른 딥러닝 프레임워크에서 API로 사용할 수 있다

### Traditional ML

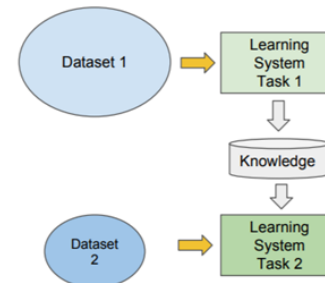
- Isolated, single task learning:
  - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

### Transfer Learning

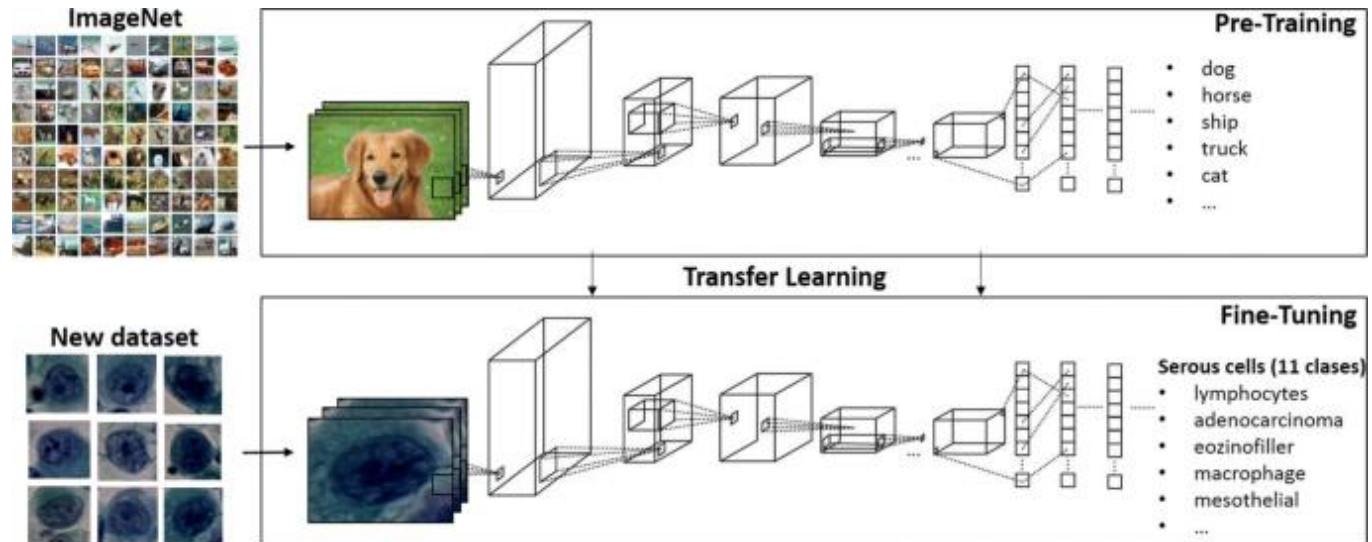
- Learning of a new tasks relies on the previous learned tasks:
  - Learning process can be faster, more accurate and/or need less training data



# III. Transfer Learning

신경망 학습을 위한 대량의 데이터셋, 가장 대표적으로 **ImageNet**.

ImageNet은 2010년~2017년까지 매해 열린 대회 ImageNet Large-Scale Visual Recognition (ILSVRC)을 위한 데이터셋으로 자동차나 고양이를 포함한 **1000개의 클래스, 총 1400만개의 이미지로 구성**

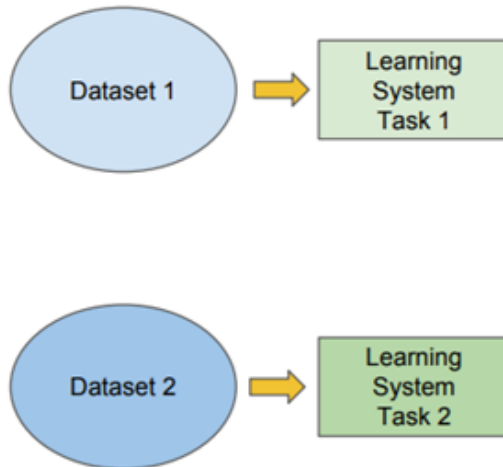


# III. Transfer Learning

---

## Traditional ML

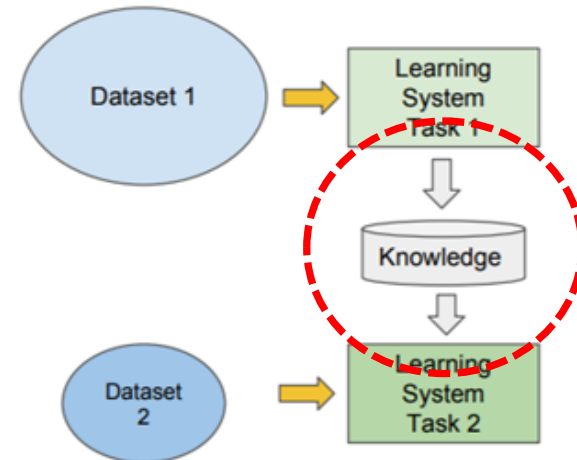
- Isolated, single task learning:
  - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

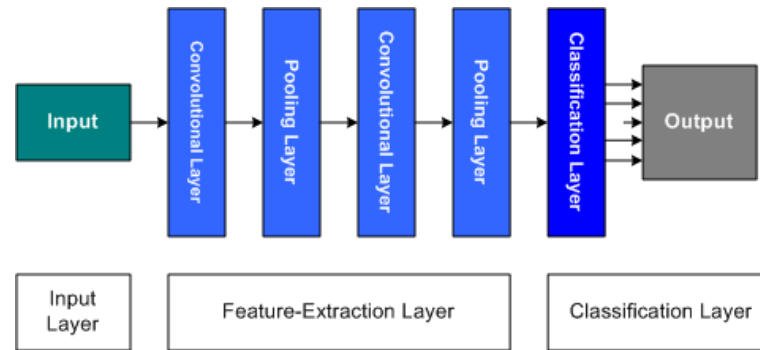
## Transfer Learning

- Learning of a new task relies on the previous learned tasks:
  - Learning process can be faster, more accurate and/or need less training data



# III. Transfer Learning(**pretrained model**)

---

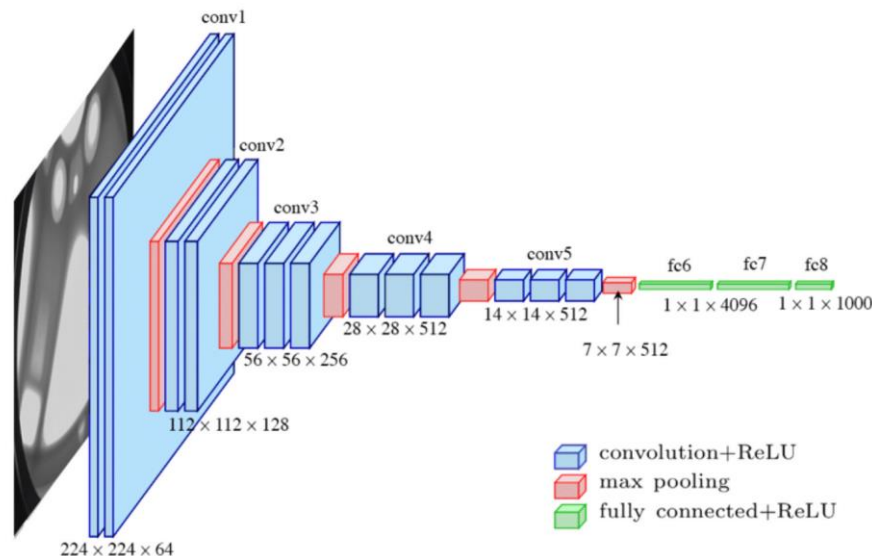
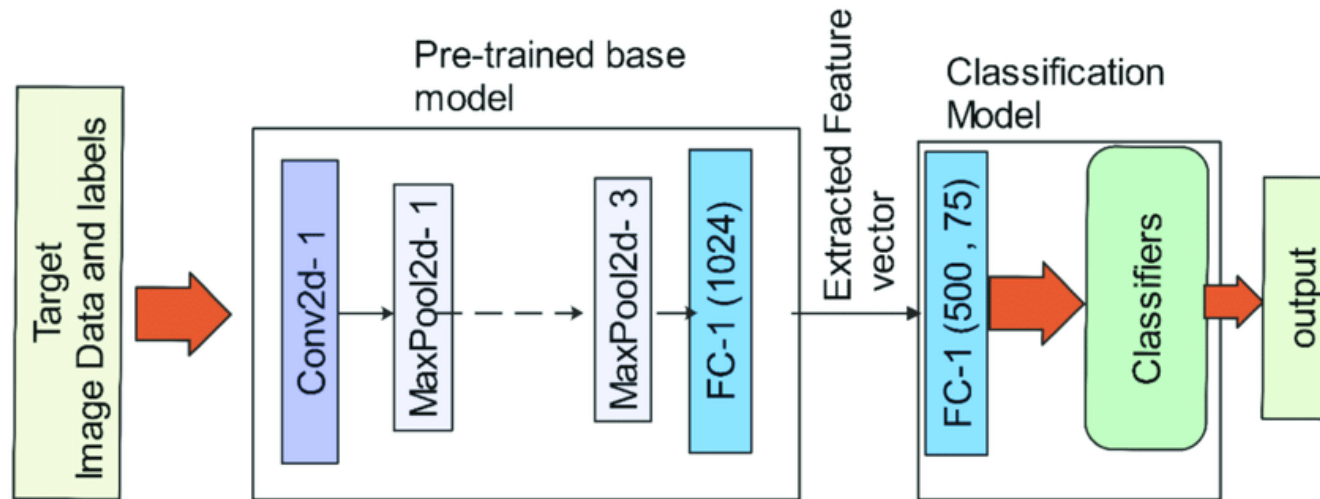


**Pretrained Model** = feature-extractor + classifier

**Feature extractor** : 입력 데이터에서 중요한 특징, 표현을 추출하는 역할.  
이미지 분류의 경우,  
이미지의 텍스처, 모양, 색상 등 시각적 특징을 추출

**Classifier** : feature extractor로부터 추출된 특징을 기반으로 입력 데이터를 분류하는 역할.  
이미지 분류의 경우,  
classifier는 추출된 특징을 사용하여 이미지가 어떤 클래스에 속하는지 예측

# III. Transfer Learning(pretrained model)



# III.Transfer Learning(pretrained model)

---

TensorFlow의 주요 pretrained model :

이미지 분류, 객체 감지, 자연어 처리 등 다양한 작업에 활용

**1.VGG16/VGG19:** 이미지 분류를 위한 CNN (Convolutional Neural Network) . ImageNet 데이터셋을 기반으로 다양한 객체 및 동물의 분류를 수행.

**2.ResNet50/ResNet101:** 대규모 데이터셋에서 다양한 객체 및 동물의 분류.

**3.MobileNet:** 경량화된 CNN 아키텍처로, 모바일 및 임베디드 장치에서의 실시간 이미지 분류 작업에 특화. 작은 크기와 빠른 속도로 객체 분류를 수행.

**4.SSD (Single Shot MultiBox Detector):** 객체 감지 및 객체 기반 작업에 사용.

**5.BERT (Bidirectional Encoder Representations from Transformers):** Transformer 기반 아키텍처로, 문장의 의미와 문맥을 이해하는 데에 탁월한 성능. 텍스트 분류, 문장 유사도 측정, 개체명 인식 등 다양한 자연어 처리 작업에 활용.

**= > 적용할 데이터에 대해 다양한 모델의 사용하여 최적의 모델과 환경을 선택 !!**

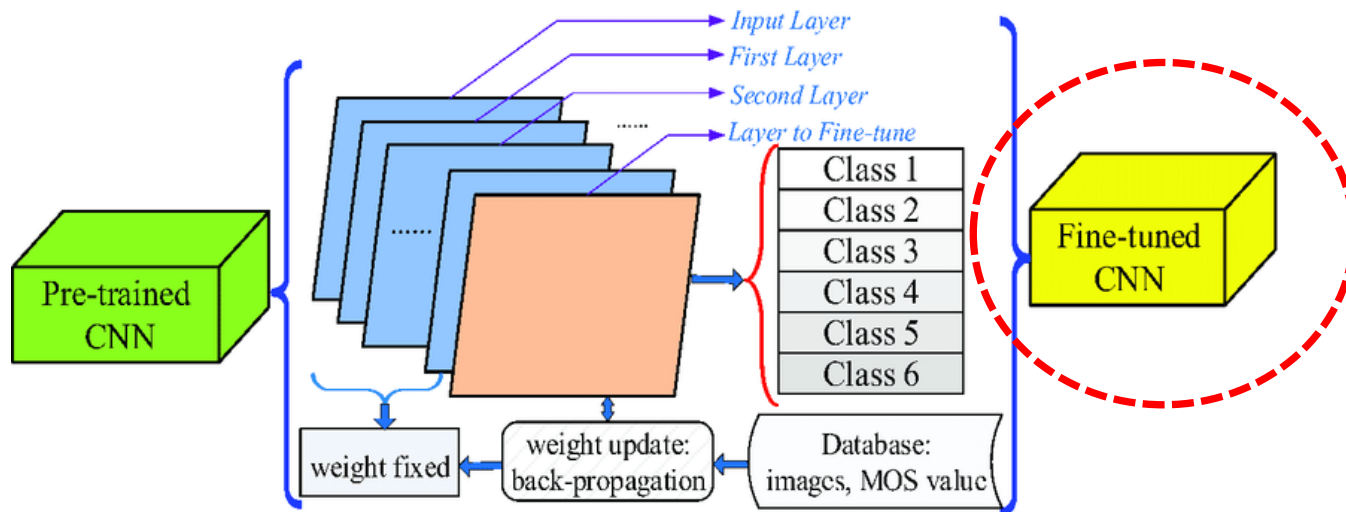
# III. Transfer Learning (pretrained model)

| Model                  | TF-Slim File         | Checkpoint                                            | Top-1 Accuracy | Top-5 Accuracy |
|------------------------|----------------------|-------------------------------------------------------|----------------|----------------|
| Inception V1           | <a href="#">Code</a> | <a href="#">inception_v1_2016_08_28.tar.gz</a>        | 69.8           | 89.6           |
| Inception V2           | <a href="#">Code</a> | <a href="#">inception_v2_2016_08_28.tar.gz</a>        | 73.9           | 91.8           |
| Inception V3           | <a href="#">Code</a> | <a href="#">inception_v3_2016_08_28.tar.gz</a>        | 78.0           | 93.9           |
| Inception V4           | <a href="#">Code</a> | <a href="#">inception_v4_2016_09_09.tar.gz</a>        | 80.2           | 95.2           |
| Inception-ResNet-v2    | <a href="#">Code</a> | <a href="#">inception_resnet_v2_2016_08_30.tar.gz</a> | 80.4           | 95.3           |
| ResNet V1 50           | <a href="#">Code</a> | <a href="#">resnet_v1_50_2016_08_28.tar.gz</a>        | 75.2           | 92.2           |
| ResNet V1 101          | <a href="#">Code</a> | <a href="#">resnet_v1_101_2016_08_28.tar.gz</a>       | 76.4           | 92.9           |
| ResNet V1 152          | <a href="#">Code</a> | <a href="#">resnet_v1_152_2016_08_28.tar.gz</a>       | 76.8           | 93.2           |
| ResNet V2 50^          | <a href="#">Code</a> | <a href="#">resnet_v2_50_2017_04_14.tar.gz</a>        | 75.6           | 92.8           |
| ResNet V2 101^         | <a href="#">Code</a> | <a href="#">resnet_v2_101_2017_04_14.tar.gz</a>       | 77.0           | 93.7           |
| ResNet V2 152^         | <a href="#">Code</a> | <a href="#">resnet_v2_152_2017_04_14.tar.gz</a>       | 77.8           | 94.1           |
| ResNet V2 200          | <a href="#">Code</a> | TBA                                                   | 79.9*          | 95.2*          |
| VGG 16                 | <a href="#">Code</a> | <a href="#">vgg_16_2016_08_28.tar.gz</a>              | 71.5           | 89.8           |
| VGG 19                 | <a href="#">Code</a> | <a href="#">vgg_19_2016_08_28.tar.gz</a>              | 71.1           | 89.8           |
| MobileNet_v1_1.0_224   | <a href="#">Code</a> | <a href="#">mobilenet_v1_1.0_224.tgz</a>              | 70.9           | 89.9           |
| MobileNet_v1_0.50_160  | <a href="#">Code</a> | <a href="#">mobilenet_v1_0.50_160.tgz</a>             | 59.1           | 81.9           |
| MobileNet_v1_0.25_128  | <a href="#">Code</a> | <a href="#">mobilenet_v1_0.25_128.tgz</a>             | 41.5           | 66.3           |
| MobileNet_v2_1.4_224^* | <a href="#">Code</a> | <a href="#">mobilenet_v2_1.4_224.tgz</a>              | 74.9           | 92.5           |
| MobileNet_v2_1.0_224^* | <a href="#">Code</a> | <a href="#">mobilenet_v2_1.0_224.tgz</a>              | 71.9           | 91.0           |

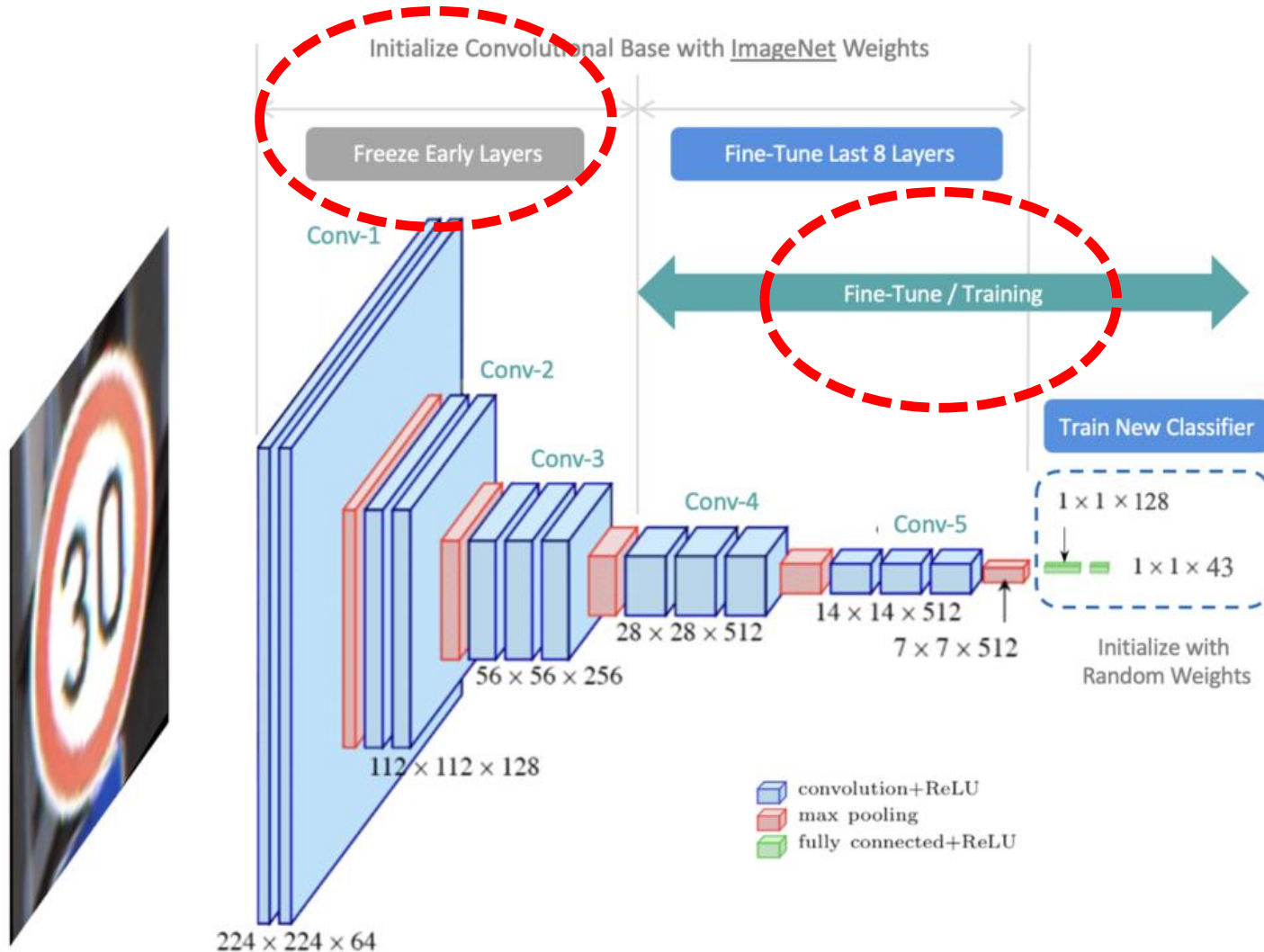
# III. Transfer Learning (pretrained model)

**Fine-tuning** : pretrained model의 일부 레이어를 고정하고  
일부 레이어를 재훈련하여 새로운 작업에 적합하도록 조정

1. 기존 모델 가져오기: ImageNet에서 사전 훈련된 모델을 다운로드/API
2. 레이어 동결(**freeze**): low level feature trained model의 레이어 동결, 가중치 고정
3. 새로운 레이어 추가(**fine tuning**): 새로운 작업에 맞게 추가적인 레이어를 모델에 추가.
4. 재훈련(**Training**): 새로운 레이어를 초기화, 모델을 재훈련. 새로운 레이어들의 가중치를 학습.



# III. Transfer Learning (pretrained model)



# III. Transfer Learning

## How to use pretrained model

```
[4] 1 import tensorflow as tf
    2 from tensorflow.keras.models import Sequential
    3 from tensorflow.keras.layers import Dense, Dropout, Flatten
    4 from tensorflow.keras.applications import VGG16, ResNet50, MobileNet, InceptionV3
    5
    6 base_model = VGG16(weights = 'imagenet', include_top=True, input_shape=(224,224,3))
    7 #include_top= True: extractor와 classifier 동시에 업로드
    8 #           False: feature extractor만 업로드
    9
   10 base_model.summary()
```

**include\_top** : False는 feature extractor만 업로드, **True는 classifier도 업로드**  
**input\_shape** : 학습시킬 새로운 이미지의 크기

**VGG16/VGG19**: 이미지 분류를 위한 CNN (Convolutional Neural Network) .  
ImageNet 데이터셋을 기반으로 다양한 객체 및 동물의 분류를 수행.

| Layer (type)               | Output Shape          | Param # |
|----------------------------|-----------------------|---------|
| =====                      |                       |         |
| input_1 (InputLayer)       | [(None, 224, 224, 3)] | 0       |
| block1_conv1 (Conv2D)      | (None, 224, 224, 64)  | 1792    |
| block1_conv2 (Conv2D)      | (None, 224, 224, 64)  | 36928   |
| block1_pool (MaxPooling2D) | (None, 112, 112, 64)  | 0       |
| block2_conv1 (Conv2D)      | (None, 112, 112, 128) | 73856   |
| block2_conv2 (Conv2D)      | (None, 112, 112, 128) | 147584  |
| block2_pool (MaxPooling2D) | (None, 56, 56, 128)   | 0       |
| block3_conv1 (Conv2D)      | (None, 56, 56, 256)   | 295168  |
| block3_conv2 (Conv2D)      | (None, 56, 56, 256)   | 590080  |
| block3_conv3 (Conv2D)      | (None, 56, 56, 256)   | 590080  |
| block3_pool (MaxPooling2D) | (None, 28, 28, 256)   | 0       |
| block4_conv1 (Conv2D)      | (None, 28, 28, 512)   | 1180160 |
| block4_conv2 (Conv2D)      | (None, 28, 28, 512)   | 2359808 |
| block4_conv3 (Conv2D)      | (None, 28, 28, 512)   | 2359808 |
| block4_pool (MaxPooling2D) | (None, 14, 14, 512)   | 0       |
| block5_conv1 (Conv2D)      | (None, 14, 14, 512)   | 2359808 |
| block5_conv2 (Conv2D)      | (None, 14, 14, 512)   | 2359808 |
| block5_conv3 (Conv2D)      | (None, 14, 14, 512)   | 2359808 |
| block5_pool (MaxPooling2D) | (None, 7, 7, 512)     | 0       |

|                               |               |           |
|-------------------------------|---------------|-----------|
| flatten (Flatten)             | (None, 25088) | 0         |
| fc1 (Dense)                   | (None, 4096)  | 102764544 |
| fc2 (Dense)                   | (None, 4096)  | 16781312  |
| predictions (Dense)           | (None, 1000)  | 4097000   |
| =====                         |               |           |
| Total params: 138,357,544     |               |           |
| Trainable params: 138,357,544 |               |           |
| Non-trainable params: 0       |               |           |
| =====                         |               |           |

**VGG16 : 138,357,544개의 파라미터를 가진 모델**  
**(extractor + classifier)**

# III. Transfer Learning

## How to use pretrained model



```
1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense, Dropout, Flatten
4 from tensorflow.keras.applications import VGG16, ResNet50, MobileNet, InceptionV3
5
6 base_model = VGG16(weights = 'imagenet', include_top=False, input_shape=(240,240,3))
7 #include_top= True: extractor와 classifier 동시에 업로드,
8 #           False는 feature extractor만 업로드
9
10 base_model.summary()
```

**include\_top : False는 feature extractor만 업로드, True는 classifier도 업로드**  
**input\_shape : 학습시킬 새로운 이미지의 크기**

**VGG16/VGG19:** 이미지 분류를 위한 CNN (Convolutional Neural Network) .  
ImageNet 데이터셋을 기반으로 다양한 객체 및 동물의 분류를 수행.

| Layer (type)               | Output Shape          | Param # |
|----------------------------|-----------------------|---------|
| input_2 (InputLayer)       | [(None, 240, 240, 3)] | 0       |
| block1_conv1 (Conv2D)      | (None, 240, 240, 64)  | 1792    |
| block1_conv2 (Conv2D)      | (None, 240, 240, 64)  | 36928   |
| block1_pool (MaxPooling2D) | (None, 120, 120, 64)  | 0       |
| block2_conv1 (Conv2D)      | (None, 120, 120, 128) | 73856   |
| block2_conv2 (Conv2D)      | (None, 120, 120, 128) | 147584  |
| block2_pool (MaxPooling2D) | (None, 60, 60, 128)   | 0       |
| block3_conv1 (Conv2D)      | (None, 60, 60, 256)   | 295168  |
| block3_conv2 (Conv2D)      | (None, 60, 60, 256)   | 590080  |
| block3_conv3 (Conv2D)      | (None, 60, 60, 256)   | 590080  |
| block3_pool (MaxPooling2D) | (None, 30, 30, 256)   | 0       |
| block4_conv1 (Conv2D)      | (None, 30, 30, 512)   | 1180160 |
| block4_conv2 (Conv2D)      | (None, 30, 30, 512)   | 2359808 |
| block4_conv3 (Conv2D)      | (None, 30, 30, 512)   | 2359808 |
| block4_pool (MaxPooling2D) | (None, 15, 15, 512)   | 0       |
| block5_conv1 (Conv2D)      | (None, 15, 15, 512)   | 2359808 |
| block5_conv2 (Conv2D)      | (None, 15, 15, 512)   | 2359808 |
| block5_conv3 (Conv2D)      | (None, 15, 15, 512)   | 2359808 |
| block5_pool (MaxPooling2D) | (None, 7, 7, 512)     | 0       |
| =====                      |                       |         |
| Total params: 14,714,688   |                       |         |

**VGG16 : 14,714,688개의  
파라미터를 가진 모델  
(**extractor only**)**

# III. Transfer Learning

```
1 model = Sequential()
2
3 model.add(base_model)
4
5 model.add(Flatten())
6
7 model.add(Dense(64, activation='relu'))
8 model.add(Dropout(0.25))
9 model.add(Dense(4, activation='softmax'))
10
11 model.compile(loss='sparse_categorical_crossentropy',
12               optimizer=tf.keras.optimizers.Adam(),
13               metrics=['accuracy'])
14
15 model.summary()
```

```
Model: "sequential_1"
-----
Layer (type)                 Output Shape              Param #
-----
vgg16 (Functional)          (None, 7, 7, 512)        14714688
flatten_1 (Flatten)         (None, 25088)             0
dense_2 (Dense)              (None, 64)                1605696
dropout_1 (Dropout)         (None, 64)                0
dense_3 (Dense)              (None, 4)                 260
-----
Total params: 16,320,644
Trainable params: 16,320,644
Non-trainable params: 0
-----
```

**VGG16 모델에 fine tuning :**

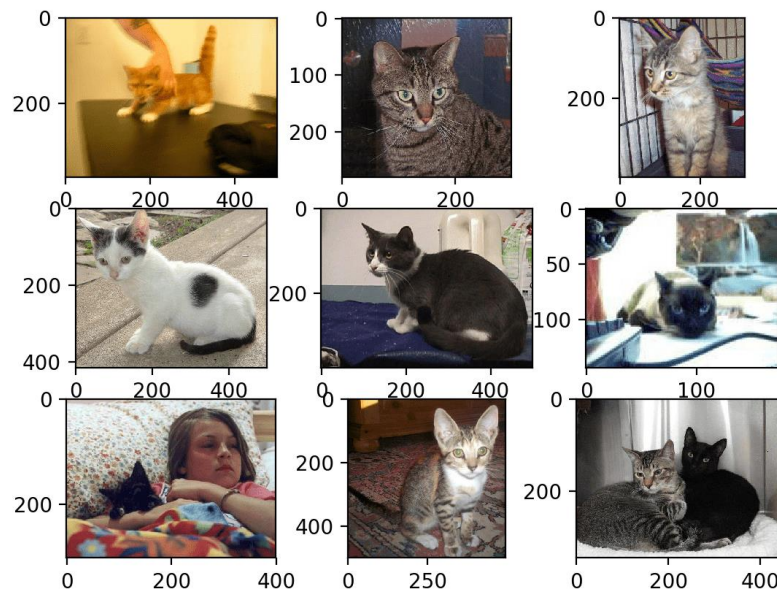
**Flatten > Dense(64, relu) > Dropout(25%) > Dense(4, softmax) > Compile( )**

**14,714,688개의 파라미터 > 16,320,644개의 파라미터**

# III. Transfer Learning

---

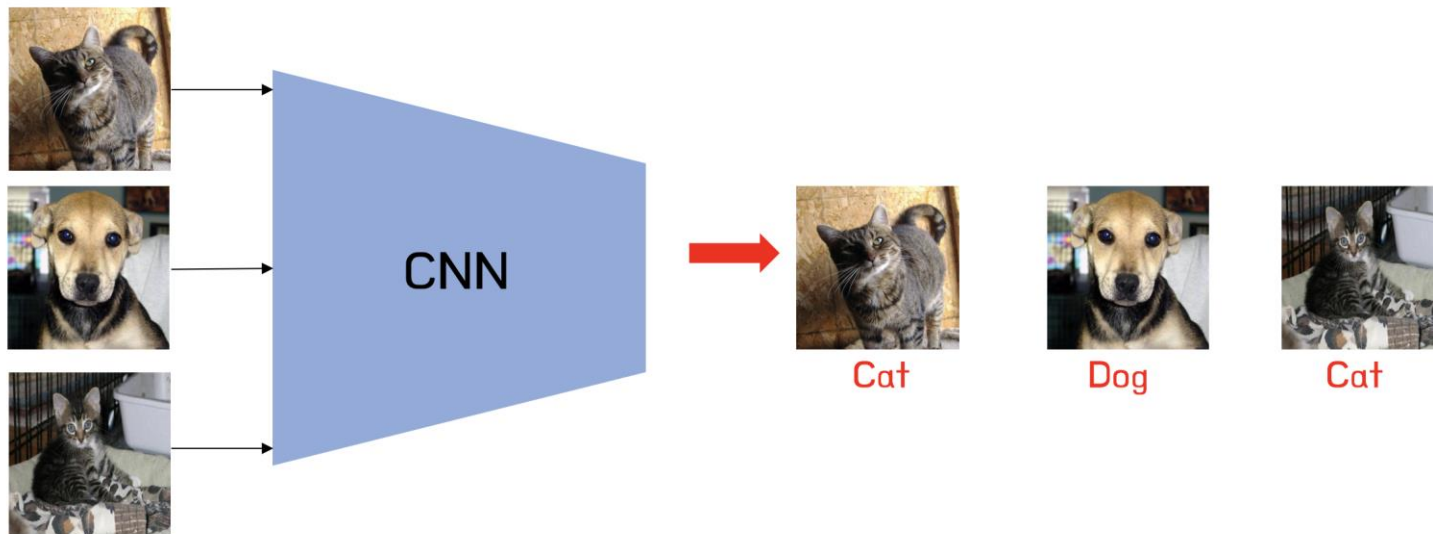
## Transfer Learning with cat&dog



# III. Transfer Learning (cat&dog classification)

## Cats and Dogs Filtered 구조 (TensorFlow 예제 기준)

컴퓨터 비전 분야에서 널리 사용되는 데이터셋 중 하나.  
고양이와 개의 이미지로 구성되어 있으며, 주로 이미지 분류 작업을 수행하기 위해 사용.  
2000개의 학습 데이터와 1000개의 테스트 데이터  
원래는 컬러 이미지 (크기와 해상도는 제각각).  
데이터의 제한적 수량 학습으로 데이터의 증강(augmentation) 필요 !



# III. Transfer Learning (cat&dog classification)

## Cats and Dogs data set

**Kaggle Cats and Dogs**로 불리며, Microsoft에서 공개.  
두 개의 클래스: 고양이(Cat), 개(Dog).  
이미지 개수: 약 25,000장 (각 클래스 12,500장 )

The screenshot shows the Kaggle interface for the 'Dogs vs. Cats' competition. On the left is a sidebar with navigation links: Home, Competitions (selected), Datasets, Models, Code, Discussions, Learn, More, Your Work, and a 'VIEWED' section with 'Titanic - Machine L...' and 'ARC Prize 2025'. The main content area has a search bar at the top. Below it, the competition title 'Dogs vs. Cats' is displayed with the subtitle 'Create an algorithm to distinguish dogs from cats'. A 'Late Submission' button is in the top right. A blue banner with book icons is also present. Below the title, there are tabs for Overview, Data (selected), Code, Models, Discussion, Leaderboard, and Rules. The 'Dataset Description' section states: 'The training archive contains 25,000 images of dogs and cats. Train your algorithm on these files and predict the labels for test1.zip (1 = dog, 0 = cat).' A note on hand labeling advises against manual labeling. On the right, a summary box lists: Files (3 files), Size (853.96 MB), Type (zip, csv), and License (Subject to Competition Rules).

**Files**  
3 files

**Size**  
853.96 MB

**Type**  
zip, csv

**License**  
Subject to Competition Rules

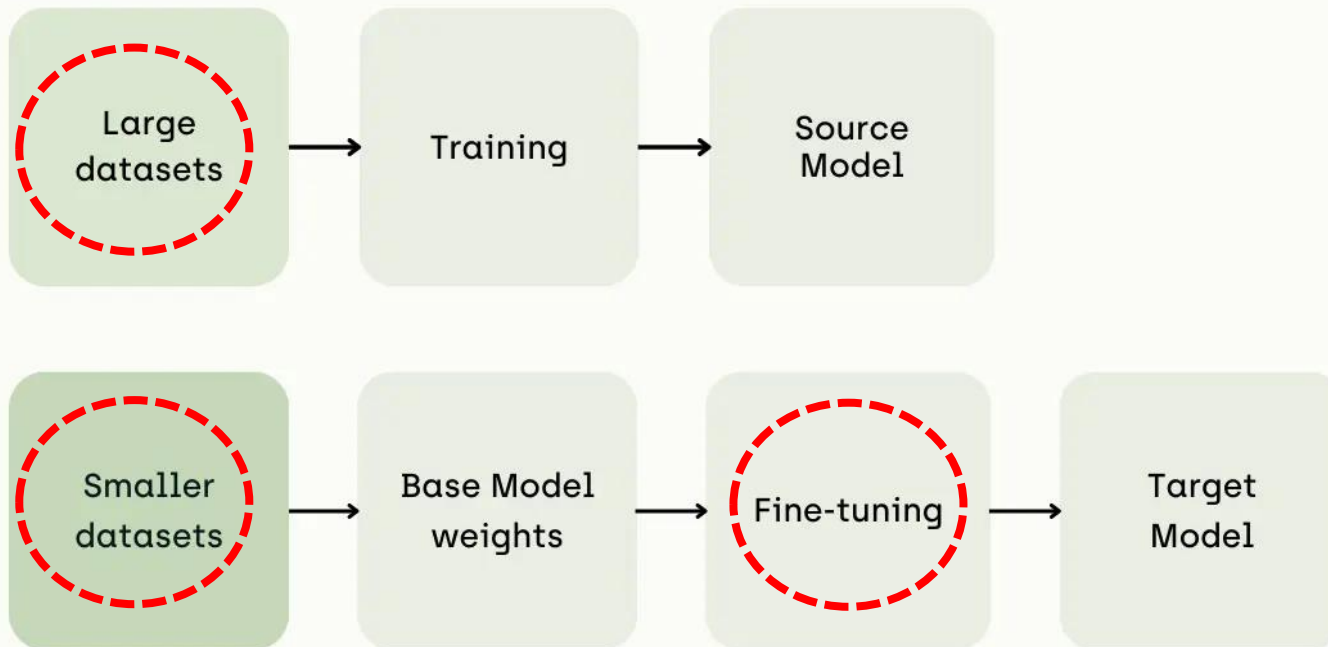
# III.Transfer Learning

| 구분       | Original Dataset             | Filtered (Google)                |
|----------|------------------------------|----------------------------------|
| 공식 명칭    | Dogs vs. Cats                | Cats and Dogs Filtered           |
| 전체 이미지 수 | 25,000장                      | 3,000장 (약 1/8 수준)                |
| 데이터 구성   | Train: 25,000 / Test: 12,500 | Train: 2,000 / Validation: 1,000 |
| 클래스당 개수  | 각 12,500장 (균형)               | 각 1,000장 (Train), 각 500장 (Test)  |
| 파일 용량    | 약 800MB ~ 2GB 이상             | 약 68MB (경량화)                     |
| 주요 목적    | 실전 경진대회 및 대규모 모델 학습          | 교육용, 전이 학습 실습, 빠른 학습             |
| 사용 난이도   | 높음 (데이터 전처리에 시간 소요)          | 낮음 (Colab 등에서 즉시 실행 가능)          |

# III. Transfer Learning

---

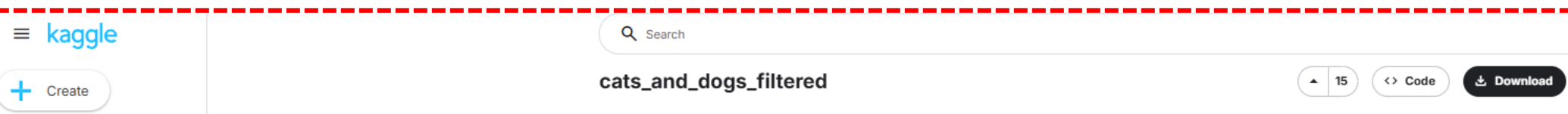
## Steps of Model Fine-Tuning



# III. Transfer Learning

```
1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.applications import Xception
4 from tensorflow.keras.layers import Flatten, Dense, Dropout, GlobalAveragePooling2D
5 from tensorflow.keras.preprocessing.image import ImageDataGenerator
6
```

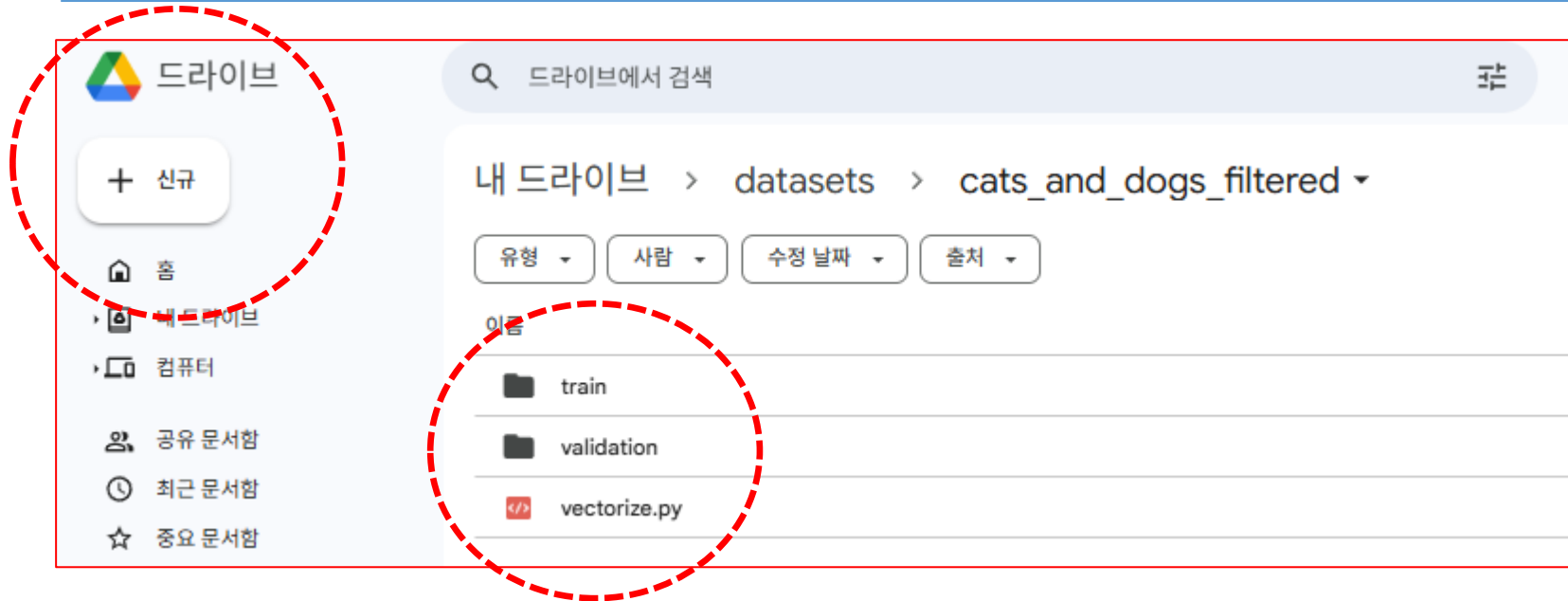
```
1 IMG_WIDTH = 224
2 IMG_HEIGHT = 224
```



**Kaggle competition에서 데이터셋 다운로드 !!**

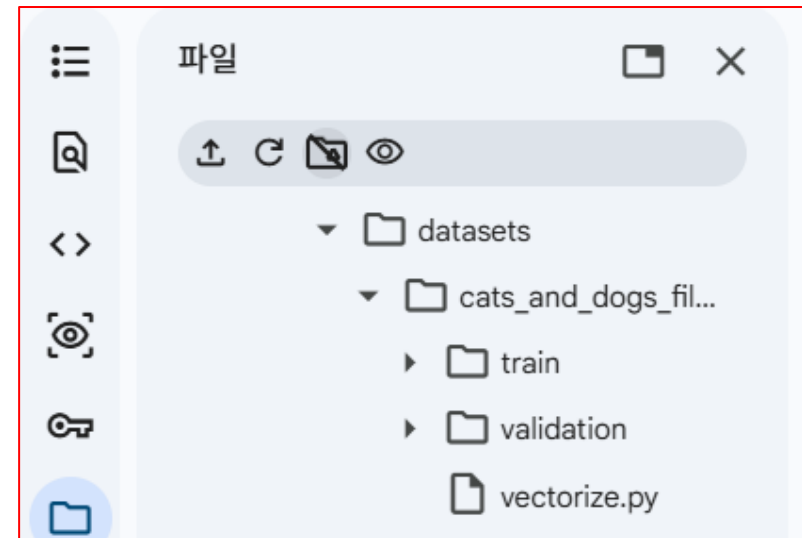
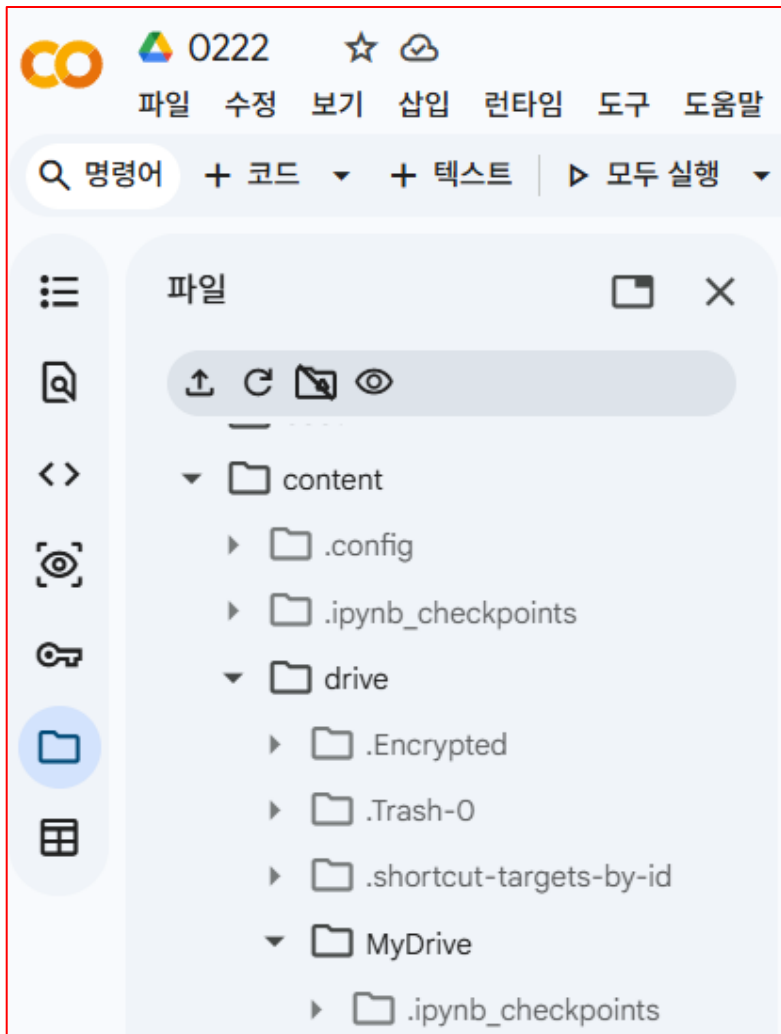
<https://www.kaggle.com/datasets/birajsth/cats-and-dogs-filtered>

# III. Transfer Learning



구글 드라이브에 압축이 해제된 데이터 셋을 업로드 !!

# III. Transfer Learning



**코랩에서 구글 드라이브를 연결 후  
데이터세트의 위치를 확인 !!!**

# III. Transfer Learning

---



**압축 해제 된 뒤의 폴더구조 및 샘플 데이터**

# III. Transfer Learning

**Kaggle notebook을 사용한다면 경로 변경!!** ('/kaggle/working/')

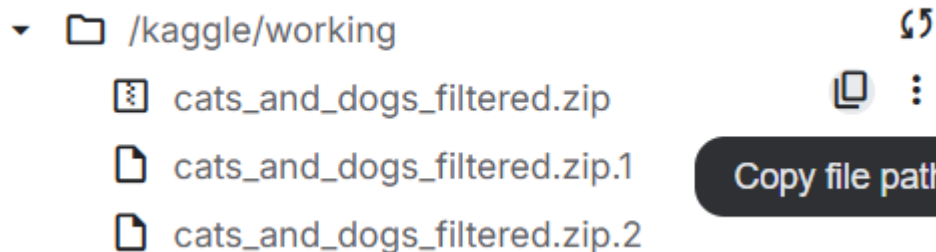
# 2. 데이터셋 다운로드 및 압축 해제

!wget [https://storage.googleapis.com/mledu-datasets/cats\\_and\\_dogs\\_filtered.zip](https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip)

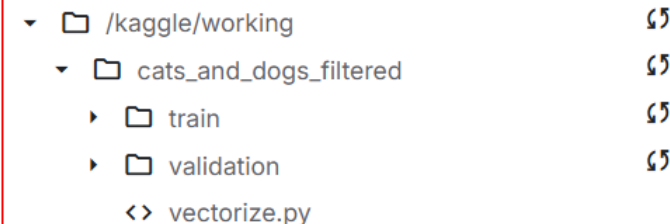
```
if os.path.exists('/kaggle/working/cats_and_dogs_filtered/'):
    shutil.rmtree('/kaggle/working/cats_and_dogs_filtered/')
    print('/kaggle/working/cats_and_dogs_filtered/ is removed !!!')
```

```
with zipfile.ZipFile('/kaggle/working/cats_and_dogs_filtered.zip', 'r') as target_file:
    target_file.extractall('/kaggle/working/')
```

IMG\_WIDTH = 224 IMG\_HEIGHT = 224



Output (398.5MiB / 19.5GiB)



# III. Transfer Learning

```
1 base_model = Xception(weights='imagenet', include_top=False, input_shape=(IMG_WIDTH, IMG_HEIGHT, 3))
2
3 base_model.summary()
```

Downloading data from [https://storage.googleapis.com/tensorflow/keras-applications/xception/xception\\_weights\\_tf\\_dim\\_ordering\\_tf\\_data\\_format.h5](https://storage.googleapis.com/tensorflow/keras-applications/xception/xception_weights_tf_dim_ordering_tf_data_format.h5) 83683744/83683744 [=====] - 0s 0us/step  
Model: "xception"

| Layer (type)                          | Output Shape          | Param # | Connected to               |
|---------------------------------------|-----------------------|---------|----------------------------|
| input_3 (InputLayer)                  | [(None, 224, 224, 3)] | 0       | []                         |
| block1_conv1 (Conv2D)                 | (None, 111, 111, 32)  | 864     | ['input_3[0][0]']          |
| block1_conv1_bn (Batch Normalization) | (None, 111, 111, 32)  | 128     | ['block1_conv1[0][0]']     |
| block1_conv1_act (Activation)         | (None, 111, 111, 32)  | 0       | ['block1_conv1_bn[0][0]']  |
| block1_conv2 (Conv2D)                 | (None, 109, 109, 64)  | 18432   | ['block1_conv1_act[0][0]'] |

## Xception모델의 호출

# III. Transfer Learning

---

**base\_model = Xception(weights='imagenet', include\_top=False,  
input\_shape=(IMG\_WIDTH,IMG\_HEIGHT,3))**

**weights='imagenet':** weights='imagenet' 옵션을 사용하면 Xception 모델이  
ImageNet 데이터셋에서 사전 훈련된 가중치를 사용하여 초기화

**include\_top=False:** 모델의 상단에 있는 classifier를 포함시키지 않는다.  
이를 통해 모델을 특정 작업에 맞게 수정하고 재사용할 수 있다.

**input\_shape=(IMG\_WIDTH,IMG\_HEIGHT,3):** 입력 이미지의 형태를 정의.  
IMG\_WIDTH와 IMG\_HEIGHT는 이미지의 가로와 세로 크기이며,  
3은 컬러 채널 수(RGB).  
입력 이미지는 이 크기로 사전에 전처리되어야 한다.

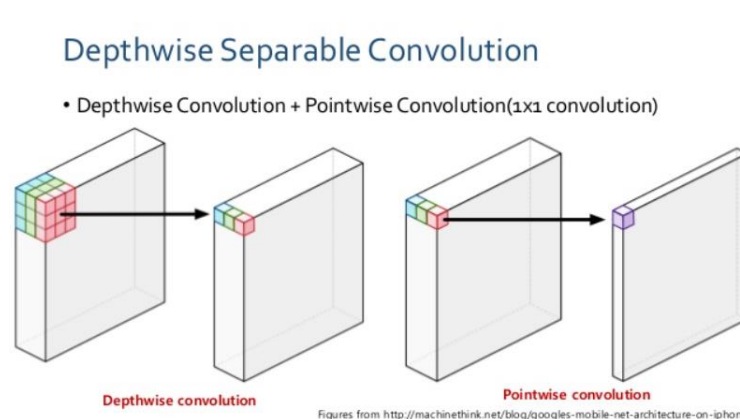
# III. Transfer Learning

## Xception :

"Extreme Inception"의 약어로, Inception 모듈을 확장한 형태.  
이미지 내에서 다양한 크기와 비율의 객체를 잘 인식할 수 있도록 설계.

공간의 특징을 추출하는 Depthwise Convolution은 입력 채널별로 컨볼루션을 수행하고,  
채널간의 상호작용을 조절하고 특성을 결합

\* Pointwise Convolution 은 1x1 컨볼루션을 통해 채널 간의 선형 결합을 수행



# III. Transfer Learning

---

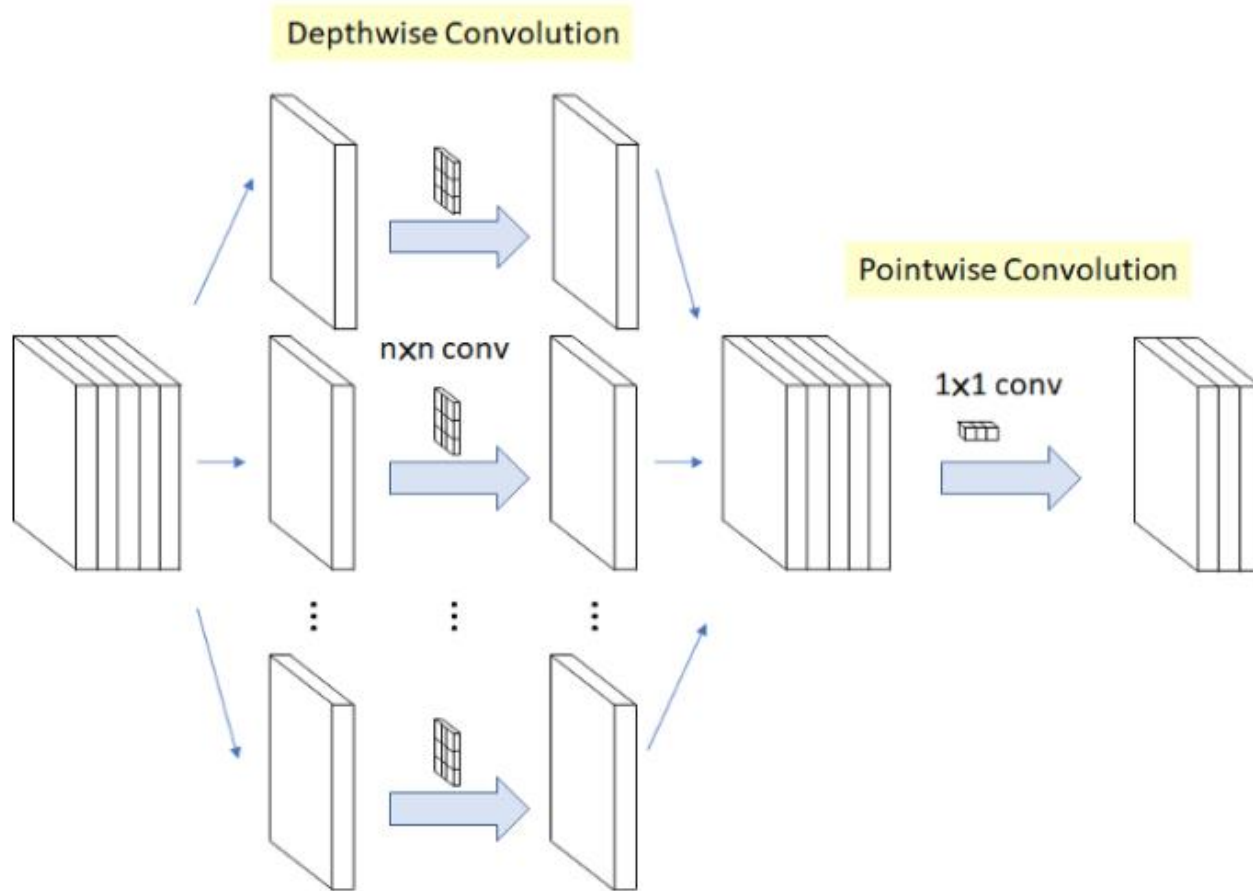
## Xception :

이미지넷(ImageNet) 등 대규모 데이터셋에서 사전훈련된(pre-trained) 모델

- 1.극도의 인셉션 구조:** Xception은 Inception 모델의 아이디어를 발전. Inception 모델은 다양한 커널 크기를 가진 **병렬적인 컨볼루션 계층을 사용하여 효율적인 특징 추출을 수행.** Xception은 이를 더욱 극단적으로 발전시켜 모든 컨볼루션 계층에 대해 1x1 컨볼루션을 사용하고, 병렬적인 컨볼루션 연산을 연결한 깊은 구조를 채택.
- 2.깊은 네트워크 아키텍처:** 깊은 아키텍처는 더욱 복잡한 시각적 패턴을 학습할 수 있어 다양한 이미지 분류와 관련된 작업에서 좋은 성능을 발휘할 수 있다.
- 3.파라미터 효율성:** 1x1 컨볼루션을 사용하여 입력 텐서의 차원을 줄이고, 파라미터 수를 크게 감소. 이는 모델의 학습 속도를 향상시키고, 메모리 사용량을 감소.
- 4.사전훈련된 가중치:** 이미지넷과 같은 대규모 데이터셋에서 미리 학습된 가중치를 가지고 전이학습을 수행할 수 있다.

# III. Transfer Learning

---





# III. Transfer Learning

---



14,197,122 images, 21841 synsets indexed

[Home](#) [Download](#) [Challenges](#) [About](#)

Not logged in. [Login](#) | [Signup](#)

**ImageNet** is an image database organized according to the **WordNet** hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. The project has been **instrumental** in advancing computer vision and deep learning research. The data is available for free to researchers for non-commercial use.

Mar 11 2021. ImageNet website update.



# III. Transfer Learning

---

## ImageNet :

**약 1,000개의 다양한 클래스에 대해 약 1,400만장의 이미지로 구성.**

ImageNet은 **사전 훈련된 모델의 가중치를 제공**  
사전 훈련된 모델을 기반으로 새로운 작업에 대해 **전이 학습(Transfer Learning)** 수행가능

- 1.데이터셋 크기:** 약 1,400만장의 이미지로 구성되어 있어 많은 컴퓨팅 자원과 시간이 필요..
- 2.클래스 불균형:** 일부 클래스는 많은 이미지를 가지고 있고, 다른 클래스는 적은 이미지를 가지고 있을 수 있다. 이로 인해 모델이 불균형한 클래스에 대해 성능이 떨어질 수 있다.
- 3.정확도 한계:** 일부 이미지에 대한 레이블이 부정확, 이는 모델의 성능을 제한할 수 있다.
- 4.새로운 도메인 적용의 어려움:** 주로 자연 이미지에 대해 좋은 성능. 하지만 특정 도메인의 이미지에 적용할 때는 추가적인 조정이 필요할 수 있다.

# III. Transfer Learning

```
1 model = Sequential()  
2  
3 model.add(base_model)  
4  
5 model.add(GlobalAveragePooling2D())  
6  
7 model.add(Dense(16, activation='relu'))  
8 model.add(Dropout(0.25))  
9  
10 model.add(Dense(2, activation='softmax')) # 예측 결과가 2개(cat, dog)이므로 최종 출력은 2  
11  
12 model.summary()
```

Model: "sequential\_2"

| Layer (type)                                      | Output Shape       | Param #  |
|---------------------------------------------------|--------------------|----------|
| exception (Functional)                            | (None, 7, 7, 2048) | 20861480 |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 2048)       | 0        |
| dense_4 (Dense)                                   | (None, 16)         | 32784    |
| dropout_2 (Dropout)                               | (None, 16)         | 0        |
| dense_5 (Dense)                                   | (None, 2)          | 34       |
| =====                                             |                    |          |
| Total params: 20,894,298                          |                    |          |
| Trainable params: 20,839,770                      |                    |          |
| Non-trainable params: 54,528                      |                    |          |
| =====                                             |                    |          |

**Xception의 extractor에 classifier를 fine tuning하여  
예측결과가 2개(cat, dog)인 모델을 완성**

# III. Transfer Learning

```
1 model = Sequential()  
2  
3 model.add(base_model)  
4  
5 model.add(GlobalAveragePooling2D())
```

Model: "sequential\_2"

| Layer (type)                                      | Output Shape       | Param #  |
|---------------------------------------------------|--------------------|----------|
| exception (Functional)                            | (None, 7, 7, 2048) | 20861480 |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 2048)       | 0        |

`model.add(GlobalAveragePooling2D())`

**Global Average Pooling** : 신경망(CNN) 모델의 마지막 부분에 추가되는 층

이전 합성곱 및 풀링 층을 통해 추출된 특성 맵에 대한 공간 정보를 압축하여  
글로벌한 컨텍스트를 나타내는 특성을 얻을 수 있다.

Global Average Pooling에 Fully Connected Layer를 추가하여 최종 분류나 회귀를 수행.

# III. Transfer Learning

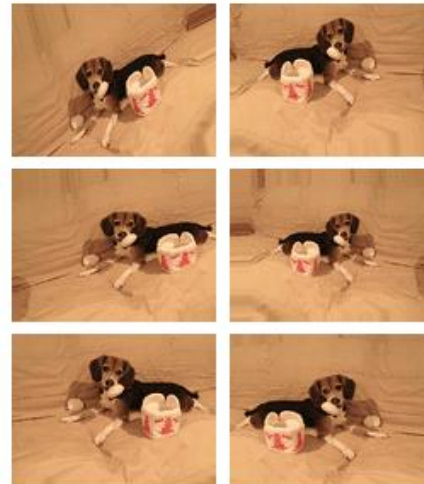
---

```
1 train_data_gen = ImageDataGenerator(rescale=1./255, # 정규화
2                                     rotation_range=10, width_shift_range=0.1,
3                                     height_shift_range=0.1, shear_range=0.1, zoom_range=0.1)
4
5 test_data_gen = ImageDataGenerator(rescale=1./255) # 정규화
```

Input Image



Augmented Images



# III.Transfer Learning

---

## Keras의 ImageDataGenerator

데이터 증강(data augmentation)을 수행.  
데이터 증강은 기존의 데이터를 변형하여 데이터의 다양성을 높이는 기법,  
모델의 성능과 일반화 능력을 향상시키는 데 도움.

rescale=1./255: 픽셀값을 0과 1 사이값으로 정규화.  
rotation\_range=10: 이미지를 최대 10도까지 회전.  
width\_shift\_range=0.1: 이미지를 가로 방향으로 최대 10%까지 이동.  
height\_shift\_range=0.1: 이미지를 세로 방향으로 최대 10%까지 이동.  
shear\_range=0.1: 이미지를 최대 10%까지 변형(비틀기).  
zoom\_range=0.1: 이미지를 최대 10%까지 확대 또는 축소.

**데이터 증강을 통해 다양한 상황에 대처할 수 있는 능력을 키우고, 과적합 방지, 일반화 성능 향상.**

# III. Transfer Learning

Keras Documentation

Search docs

Home

Why use Keras

Getting started

Models

Layers

Preprocessing

Losses

Metrics

Optimizers

Activations

Callbacks

Datasets

Applications

Backend

Initializers

Regularizers

Constraints

Visualization

Scikit-learn API

Docs » Preprocessing » Image Preprocessing

Edit on GitHub

이미지 전처리

ImageDataGenerator 클래스 [source]

```
keras.preprocessing.image.ImageDataGenerator(featurewise_center=False, samplewise_center=False, featurewise_std_normalization=False, samplewise_std_normalization=False, zca_epsilon=1e-6, zca_whitening=False, rotation_range=0, width_shift_range=0, height_shift_range=0, shear_range=0, zoom_range=1, channel_shift_range=0, validation_split=0.0)
```

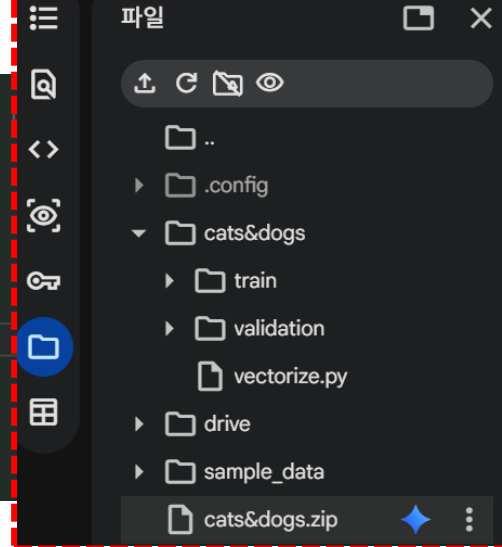
실시간 데이터 증강을 사용해서 텐서 이미지 데이터 배치를 생성합니다. 데이터에 대해 (배치 단위로) 루프가 순환됩니다.

인수

- featurewise\_center**: 불리언. 데이터셋에 대해 특성별로 인풋의 평균이 0이 되도록 합니다.
- samplewise\_center**: 불리언. 각 샘플의 평균이 0이 되도록 합니다.
- featurewise\_std\_normalization**: 불리언. 인풋을 각 특성 내에서 데이터셋의 표준편차로 나눕니다.
- samplewise\_std\_normalization**: 불리언. 각 인풋을 표준편차로 나눕니다.
- zca\_epsilon**: 영위상 성분분석 백색화의 엡실론 값. 디폴트 값은 1e-6입니다.
- zca\_whitening**: 불리언. 영위상 성분분석 백색화를 적용할지 여부입니다.
- rotation\_range**: 정수. 무작위 회전의 각도 범위입니다.
- width\_shift\_range**: 부동소수점, 1D 형태의 유사배열 혹은 정수

```
1 # 로컬 경로에서 압축 해제, 로컬에서 데이터를 로드해야 학습시간이 단축된다 !!
2 import zipfile
3 with zipfile.ZipFile('/content/cats&dogs.zip', 'r') as target_file:
4     target_file.extractall('/content/cats&dogs')
```

```
1 train_dir = '/content/cats&dogs/train'
2 test_dir = '/content/cats&dogs/validation'
```



# 로컬 경로에서 압축 해제, 로컬에서 데이터를 로드해야 학습시간이 단축된다 !!!

```
1 train_data = train_data_gen.flow_from_directory(train_dir, batch_size=32,
2                                                  color_mode='rgb', shuffle=True, class_mode='categorical',
3                                                  target_size=(IMG_WIDTH, IMG_HEIGHT))
4
5
6 test_data = test_data_gen.flow_from_directory(test_dir, batch_size=32,
7                                                color_mode='rgb', shuffle=True, class_mode='categorical',
8                                                target_size=(IMG_WIDTH, IMG_HEIGHT))
```

구글 드라이브의 Train, validation 데이터의 위치를 확인하고 설정 !!

**flow\_from\_directory** 함수

디렉토리 내의 이미지를 자동으로 **레이블링**하고,

배치 단위로 이미지를 반환하는 **제너레이터**를 생성.

모델의 fit 함수에 입력으로 사용되며 이 때 증강데이터가 발생하며 학습 및 평가에 활용.

# III. Transfer Learning

---

`flow_from_directory` :

디렉토리 내의 이미지를 자동으로 레이블링하고, 배치 단위로 이미지를 반환하는 제너레이터.  
모델의 `fit` 함수에 입력으로 사용될 수 있으며, 모델 학습 및 평가에 활용.

`flow_from_directory` 메서드는 실제로 `flow`와 `gen` 메서드를 합쳐놓은 기능

`batch_size=32`: 한 번에 처리되는 이미지의 개수.

`color_mode='rgb'`: 이미지의 컬러 모드.

`shuffle=True`: 데이터를 섞을지 여부. `True`로 설정하면 매 에포크마다 데이터가 섞는다.

`class_mode='categorical'`: 클래스 모드를 지정.

`class_mode='categorical'`로 설정된 경우,

`flow_from_directory` 함수는 레이블을 원-핫 인코딩 형식으로 반환.

모델의 출력 레이어에 적합한 형식의 레이블을 생성하여 모델의 학습과 예측을 용이하게

# III. Transfer Learning

Keras Documentation

Search docs

Home

Why use Keras

Getting started

Models

Layers

Preprocessing

Losses

Metrics

Optimizers

Activations

Callbacks

Datasets

Applications

Backend

Initializers

Regularizers

Constraints

Visualization

Scikit-learn API

Utils

## flow\_from\_directory

```
flow_from_directory(directory, target_size=(256, 256), color_mode='rgb', classes=None, class_mode='categorical')
```

디렉토리에의 경로를 전달받아 증강된 데이터의 배치를 생성합니다.

인수

- **directory**: string, 표적 디렉토리에의 경로. 반드시 한 클래스 당 하나의 하위 디렉토리가 있어야 합니다. 각 하위 디렉토리 내에 위치한 어떤 PNG, JPG, BMP, PPM 혹은 TIF 이미지도 생성자에 포함됩니다. 세부사항은 [이 스크립트](#) 를 참조하십시오.
- **target\_size**: 정수 튜플 (높이, 넓이), 디폴트 값: (256, 256). 모든 이미지의 크기를 재조정할 치수.
- **color\_mode**: "grayscale", "rgb", "rgba" 중 하나. 디폴트 값: "rgb". 변환될 이미지가 1개, 3개, 혹은 4개의 채널을 가질지 여부.
- **classes**: 클래스 하위 디렉토리의 선택적 리스트 (예. ['dogs', 'cats']). 디폴트 값: None. 특별히 값을 지정하지 않으면, 각 하위 디렉토리를 각기 다른 클래스로 대하는 방식으로 클래스의 리스트가 `directory` 내 하위 디렉토리의 이름/구조에서 자동으로 유추됩니다 (그리고 라벨 색인에 대응되는 클래스의 순서는 영숫자 순서를 따릅니다). `class_indices` 속성을 통해서 클래스 이름과 클래스 색인 간 매핑을 담은 딕셔너리를 얻을 수 있습니다.
- **class\_mode**: "categorical", "binary", "sparse", "input", 혹은 None 중 하나. 디폴트 값: "categorical". 반환될 라벨 배열의 종류를 결정합니다:
  - "categorical"은 2D형태의 원-핫 인코딩된 라벨입니다,
  - "binary"는 1D 형태의 이진 라벨입니다, "sparse"는 1D 형태의 정수 라벨입니다,
  - "input"은 인풋 이미지와 동일한 이미지입니다 (주로 자동 인코더와 함께 사용합니다).

# III. Transfer Learning

---

```
1 print(train_data.class_indices.items())
2 print(test_data.class_indices.items())
3
4 print(len(train_data.classes))
5 print(len(test_data.classes))
6
7 print(train_data.num_classes)
8 print(test_data.num_classes)
9
```

```
dict_items([('cats', 0), ('dogs', 1)])
dict_items([('cats', 0), ('dogs', 1)])
2000
1000
2
2
```

**train\_data.class\_indices,**

**test\_data.class\_indices :**

각각 훈련 데이터와 테스트 데이터의 클래스 레이블과 해당 인덱스를 매핑한 딕셔너리.  
이를 출력하여 각 클래스 레이블과 인덱스를 확인할 수 있다.

# III. Transfer Learning

---

```
1 # 모델 컴파일
2
3 model.compile(loss='categorical_crossentropy',
4               optimizer=tf.keras.optimizers.Adam(0.00002), metrics=['accuracy'])
```

```
model.compile(loss='categorical_crossentropy',
              optimizer=tf.keras.optimizers.Adam(0.00002), metrics=['accuracy'])
```

`optimizer=tf.keras.optimizers.Adam(0.00002) :`

매우 작은 학습률을 사용하면 모델이 더 정확하고 안정적으로 최적화,  
경사 소실 문제를 완화

**전이 학습에서는 새로운 작업에 대한 가중치 업데이트를 조절하기 위해 작은 학습률을 사용.**

미세한 조정을 통해 새로운 작업에 미치는 영향을 최소화, 기존의 가중치를 보존  
이전에 학습한 특성을 유지하면서 새로운 작업에 적응할 수 있도록 한다.

```

1 from datetime import datetime
2 from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
3
4 save_file_name = './cats_and_dogs_filtered_Xception_Colab.keras'
5
6 checkpoint = ModelCheckpoint(save_file_name,
7                               monitor='val_loss',
8                               verbose=1,
9                               save_best_only=True,
10                              mode='auto'
11                              )
12
13 earlystopping = EarlyStopping(monitor='val_loss',
14                               patience=5 )
15
16
17 start_time = datetime.now()
18
19 hist = model.fit(train_data,
20                  epochs=30,
21                  validation_data=test_data,
22                  callbacks=[checkpoint, earlystopping])
23
24 end_time = datetime.now()
25
26 print('elapsed time = ', end_time-start_time)

```

**patience 5번(학습결과가 개선되지 않은 5번째에 early stopping)**

# III. Transfer Learning

---

```
Epoch 6: val_loss improved from 0.03241 to 0.02958, saving model to ./cats_and_dogs_filtered_Xception_Colab.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
63/63 ----- 43s 673ms/step - accuracy: 0.9923 - loss: 0.0412 - val_accuracy: 0.9900 - val_loss: 0.0296
Epoch 7/30
63/63 ----- 0s 576ms/step - accuracy: 0.9928 - loss: 0.0313
Epoch 7: val_loss improved from 0.02958 to 0.02816, saving model to ./cats_and_dogs_filtered_Xception_Colab.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
63/63 ----- 42s 670ms/step - accuracy: 0.9928 - loss: 0.0313 - val_accuracy: 0.9900 - val_loss: 0.0282
Epoch 8/30
63/63 ----- 0s 575ms/step - accuracy: 0.9988 - loss: 0.0208
Epoch 8: val_loss did not improve from 0.02816
63/63 ----- 42s 659ms/step - accuracy: 0.9988 - loss: 0.0208 - val_accuracy: 0.9880 - val_loss: 0.0294
Epoch 9/30
63/63 ----- 0s 574ms/step - accuracy: 0.9961 - loss: 0.0245
Epoch 9: val_loss did not improve from 0.02816
63/63 ----- 42s 655ms/step - accuracy: 0.9961 - loss: 0.0245 - val_accuracy: 0.9870 - val_loss: 0.0294
Epoch 10/30
63/63 ----- 0s 578ms/step - accuracy: 0.9993 - loss: 0.0150
Epoch 10: val_loss did not improve from 0.02816
63/63 ----- 42s 659ms/step - accuracy: 0.9992 - loss: 0.0150 - val_accuracy: 0.9890 - val_loss: 0.0301
Epoch 11/30
63/63 ----- 0s 572ms/step - accuracy: 0.9933 - loss: 0.0289
Epoch 11: val_loss did not improve from 0.02816
63/63 ----- 41s 654ms/step - accuracy: 0.9934 - loss: 0.0288 - val_accuracy: 0.9850 - val_loss: 0.0382
Epoch 12/30
63/63 ----- 0s 572ms/step - accuracy: 0.9998 - loss: 0.0097
Epoch 12: val_loss did not improve from 0.02816
63/63 ----- 41s 652ms/step - accuracy: 0.9998 - loss: 0.0097 - val_accuracy: 0.9860 - val_loss: 0.0366
```

**val\_loss improved from 0.02958 to 0.02816**

**Epoch 12: val\_loss did not improve from 0.02816**

# III. Transfer Learning

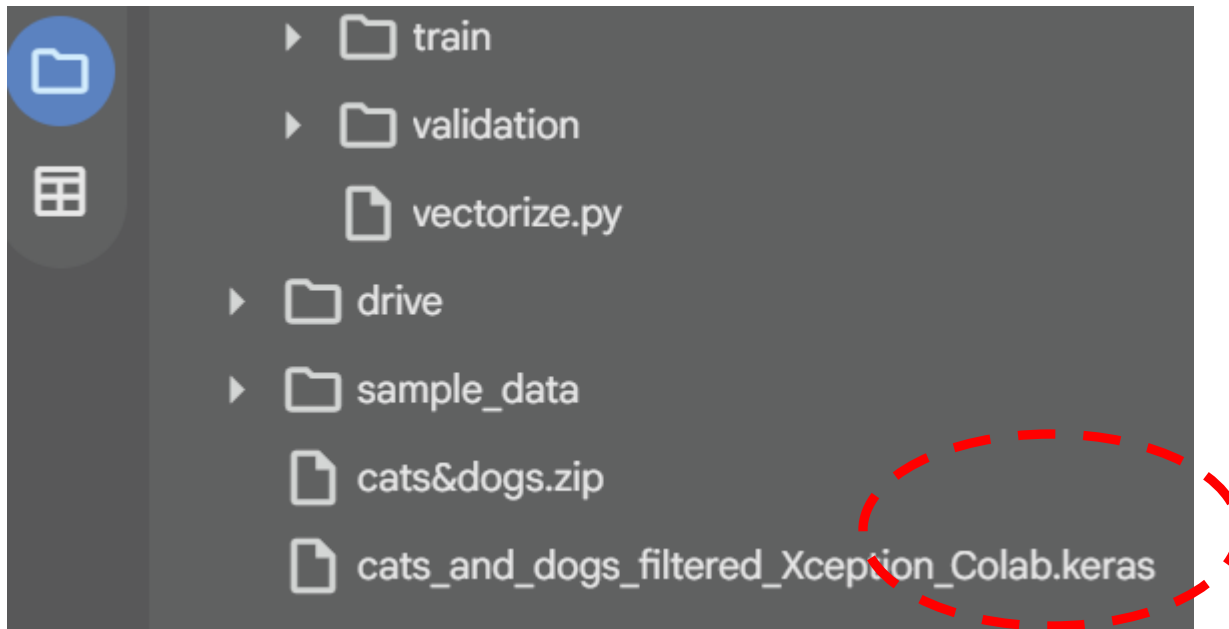
---

**확장자 ".keras" 를 모델의 저장 포맷으로 추천**

We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')`

**저장된 모델을 로드하고 예측을 수행하는 코드는**

```
from tensorflow.keras.models import load_model  
model = load_model('./cats_and_dogs_filtered_Xception_Colab.keras')  
predictions = model.predict(test_data)
```



# III. Transfer Learning

---

**ModelCheckpoint(save\_file\_name, monitor='val\_loss', verbose=1, save\_best\_only=True, mode='auto' )**

**ModelCheckpoint**는 **최적의 모델 가중치를 저장하는 콜백(callback) 함수**.

**save\_file\_name:** 저장할 모델 파일의 이름 또는 경로,  
확장자가 .keras

**monitor:** 기본값은 'val\_loss'로 검증 손실(validation loss)을 모니터링.  
'val\_accuracy'로 검증 정확도(validation accuracy)를 모니터링할 수도 있다.

**verbose:** 저장 시 메시지를 출력.

**save\_best\_only:** 기본값은 True로, 가장 좋은 성능을 보인 모델만 저장.  
False로 설정하면 모든 에포크(epoch)에서 모델이 저장.

**mode:** 모델을 저장할 지표의 방향을 지정. 'auto', 'min', 'max' .

# III.Transfer Learning

---

**earlystopping = EarlyStopping(monitor='val\_loss', patience=5 )**

EarlyStopping은 모델 학습 중에 조기 종료를 수행하는 콜백(callback) 함수.

- **monitor:** 모니터링할 지표를 지정. 기본값은 'val\_loss'로, 검증 손실(validation loss) 'val\_accuracy'로 검증 정확도(validation accuracy)를 모니터링할 수도 있다.
- **patience:** 기본값은 0으로, 한 번의 에포크(epoch)에서도 개선이 없으면 학습을 중단. 이 값이 5로 설정되면, 5번의 에포크 동안 성능이 개선않되면 학습을 중단.

## **\*\*콜백(callback) 함수**

모델의 학습 중에 특정 이벤트가 발생할 때 호출되는 함수.

학습 과정에서 정의된 지점에서 호출되어 원하는 동작을 수행할 수 있다.

콜백 함수를 사용하면 학습 중에 다양한 작업을 수행할 수 있으며,

주로 모델의 성능 모니터링, 학습 중단, 모델 저장 등에 활용

# III.Transfer Learning

---

```
1 print("Min Validation Loss:", min(hist.history['val_loss']))  
2 print("Max Validation Accuracy:", max(hist.history['val_accuracy']))
```

```
Min Validation Loss: 0.03553015738725662  
Max Validation Accuracy: 0.9879999756813049
```

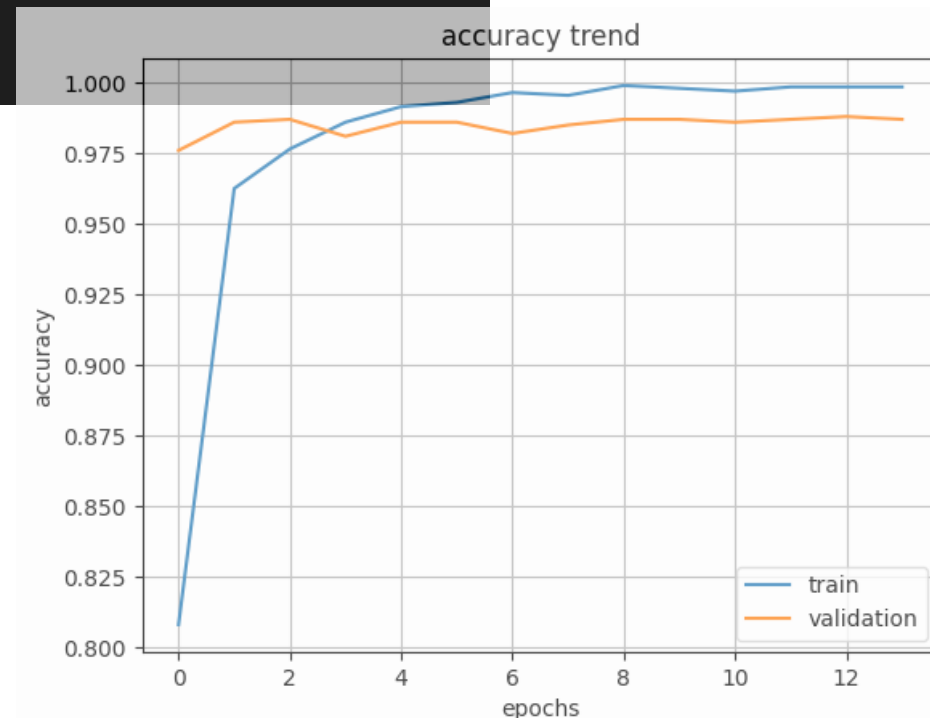
**Min Validation loss : 3.55%**

**Max Validation accuracy : 98.79%**

# III. Transfer Learning

```
1 import matplotlib.pyplot as plt
2
3 plt.title('accuracy trend')
4 plt.grid()
5 plt.xlabel('epochs')
6 plt.ylabel('accuracy')
7 plt.plot(hist.history['accuracy'], label='train')
8 plt.plot(hist.history['val_accuracy'], label='validation')
9 plt.legend(loc='best')
10 plt.show()
```

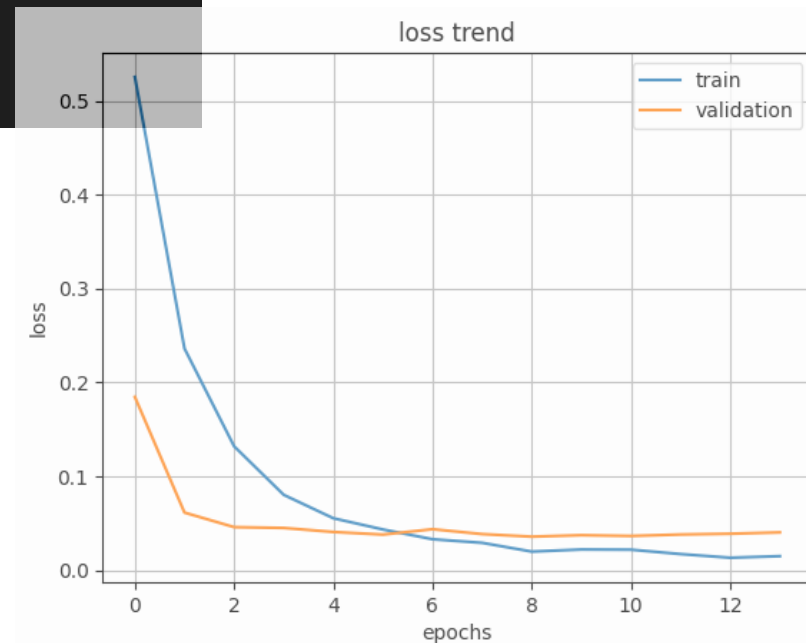
**Min Validation loss : 3.55%**  
**Max Validation accuracy : 98.79%**



# III. Transfer Learning

```
1 import matplotlib.pyplot as plt
2
3 plt.title('loss trend')
4 plt.grid()
5 plt.xlabel('epochs')
6 plt.ylabel('loss')
7 plt.plot(hist.history['loss'], label='train')
8 plt.plot(hist.history['val_loss'], label='validation')
9 plt.legend(loc='best')
10 plt.show()
```

**Min Validation loss : 3.55%**  
**Max Validation accuracy : 98.79%**



# III.Transfer Learning

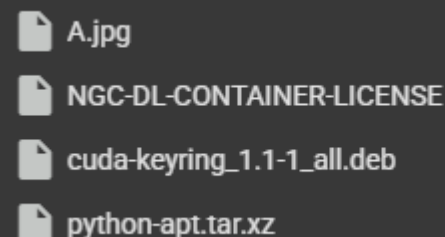
---

## Prediction

**학습모델의 저장 및 예측**

# III. Transfer Learning

```
1 import tensorflow as tf
2 from tensorflow.keras.models import load_model
3 from tensorflow.keras.preprocessing.image import load_img, img_to_array
4 import numpy as np
5
6 # 모델 불러오기
7 model = load_model('/content/cats_and_dogs_Xception_Colab.h5')
8
9 # 이미지 경로 지정
10 image_path = '/A.jpg'
11
12 # 이미지 불러오기 및 전처리
13 image = load_img(image_path, target_size=(224, 224))
14 image = img_to_array(image)
15 image = np.expand_dims(image, axis=0)
16 image = image / 255.0 # 이미지 정규화
17
18 # 예측 수행
19 class_name = ['cat', 'dog']
20 predictions = model.predict(image)
21 predicted_class = str(class_name[np.argmax(predictions)])
22
23 # 예측 결과 출력
24 print(f"Predicted class: {predicted_class}")
```



- A.jpg
- NGC-DL-CONTAINER-LICENSE
- cuda-keyring\_1.1-1\_all.deb
- python-apt.tar.xz

# III.Transfer Learning

---

**복수의 테스트 데이터 분류예측**

**Cat & Dog**

# III. Transfer Learning

예측용 데이터(강아지, 고양이: png, jpg)를 수집하여  
구글 드라이브에 폴더를 만들어 데이터를 저장한다

... > dataset > catdog ▾

유형 ▾

사람 ▾

수정 날짜 ▾

출처 ▾

이름 ↑

|                      |
|----------------------|
| 📁 .ipynb_checkpoints |
| 📷 cat1.jpg           |
| 📷 cat2.jpg           |
| 📷 cat3.jpg           |
| 📷 cat4.jpg           |
| 📷 cat5.jpg           |
| 📷 dog1.jpg           |
| 📷 dog2.jpg           |
| 📷 dog3.jpg           |
| 📷 dog4.jpg           |
| 📷 dog5.jpg           |

파일

📁 bin

📁 boot

📁 content

📁 drive

📁 MyDrive

📁 8266MCU

📁 Classroom

📁 Colab Notebooks

📁 CrewAI

📁 HuggingFace

📁 MachineLearning with ColabN...

📁 5674-833\_4th

📁 dataset

📁 TitanicData

📁 audio.zip (Unzipped Fil...

📁 catdog

마지막으로 수정한 날짜 ▾

파일 크기

|                |      |            |
|----------------|------|------------|
| 2023. 6. 30. 나 | —    | 🔍 ⬇️ ✎ ⭐ ⋮ |
| 2023. 6. 29. 나 | 69KB | ⋮          |
| 2023. 6. 29. 나 | 26KB | ⋮          |
| 2023. 6. 29. 나 | 29KB | ⋮          |
| 2023. 6. 29. 나 | 29KB | ⋮          |
| 2023. 6. 29. 나 | 75KB | ⋮          |
| 2023. 6. 29. 나 | 64KB | ⋮          |
| 2023. 6. 29. 나 | 29KB | ⋮          |
| 2023. 6. 29. 나 | 88KB | ⋮          |
| 2023. 6. 29. 나 | 57KB | ⋮          |
| 2023. 6. 29. 나 | 32KB | ⋮          |

## 테스트할 데이터의 위치를 지정하여 해당 파일을 불러 오고 이미지 파일에 대한 전처리 작업을 진행한다.(Resize, RGB, Standardization)

```
1 import cv2
2 import glob
3 import os
4
5 # 1. 경로 재확인 (본인의 실제 드라이브 경로로 수정 필요)
6 test_img_path = '/content/drive/MyDrive/cat_dog/*'
7 test_img_name_list = glob.glob(test_img_path)
8
9 test_img_list = [] # 처리된 이미지를 담을 리스트
10
11 for img_path in test_img_name_list:
12     # 확장자 체크 (이미지 파일만 필터링)
13     if not img_path.lower().endswith(('.jpg', '.jpeg', '.png')):
14         continue
15
16     src_img = cv2.imread(img_path, cv2.IMREAD_COLOR)
17
18     # 2. 이미지 로드 성공 여부 확인 (에러 방지 핵심)
19     if src_img is None:
20         print(f"이미지를 불러올 수 없습니다: {img_path}")
21         continue
22
23     # 3. 전처리 진행
24     src_img = cv2.resize(src_img, dsize=(IMG_WIDTH, IMG_HEIGHT))
25     dst_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
26     dst_img = dst_img / 255.0
27
28     test_img_list.append(dst_img)
29
30 print(f"총 {len(test_img_list)}개의 이미지가 성공적으로 로드되었습니다.")
```

# III.Transfer Learning

---

**glob.glob('test\_image\_dir/\*')**

**test\_image\_dir** 경로 안에 있는 모든 파일을 선택하여 변수에 저장

**glob.glob() 함수** : 파일의 이름 패턴에 대해 매칭을 수행하여 동일한 패턴만 반환.

**\***: 0개 이상의 임의의 문자열과 일치.

**?**: 정확히 한 개의 임의의 문자와 일치.

**[characters]**: 괄호 안에 있는 문자 중 하나와 일치.

**[!characters]**: 괄호 안에 있는 문자를 제외한 다른 문자와 일치.

여러 개의 패턴을 동시에 지정하려면 | (파이프) 기호를 사용하여 패턴을 구분.

예를 들어, 다음은 몇 가지 예시입:

**".txt"**: 확장자가 .txt인 모든 파일을 선택.

**"image[1-3].jpg"**: image1.jpg, image2.jpg, image3.jpg와 일치하는 파일을 선택.

**[abc].png"**: a, b, c로 시작하는 모든 .png 파일을 선택.

**"file?.pdf"**: file 다음에 정확히 하나의 임의의 문자가 오고 .pdf로 끝나는 파일을 선택

# III. Transfer Learning

---

for i in range(len(test\_img\_name\_list)):

```
src_img = cv2.imread(test_img_name_list[i], cv2.IMREAD_COLOR)
src_img = cv2.resize(src_img, dsize=(IMG_WIDTH, IMG_HEIGHT))
dst_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB)
dst_img = dst_img / 255.0
```

이미지를 리스트에서 읽어서(imread( ))  
리사이징 후(resize( ))  
**BGR에서 RGB로 변환하고(cvtColor( ))**  
표준화 후에(dst / 255)  
저장

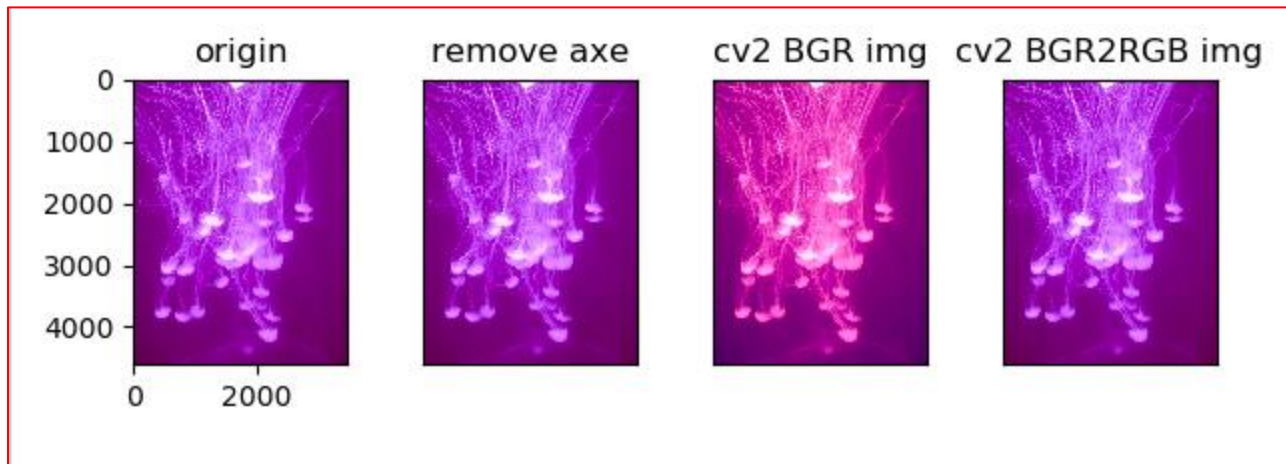


# III. Transfer Learning

---

`dst_img = cv2.cvtColor(src_img, cv2.COLOR_BGR2RGB):`

`cv2.cvtColor()` 함수를 사용하여 이미지의 색상 공간을 **BGR에서 RGB로 변환**.  
OpenCV는 기본적으로 BGR 색상 공간을 사용,  
일반적으로 사용되는 RGB 색상 공간으로 변환.



# III. Transfer Learning

```
47 # 이미지 리스트 출력
48 import matplotlib.pyplot as plt
49
50 plt.figure(figsize=(12, 5))
51 for i in range(10):
52     plt.subplot(2, 5, i+1)
53     plt.axis('off')
54     plt.imshow(test_img_list[i])
55 plt.show()
56
57 # 이미지 shape 확인
58 print(test_img_list[0].shape)
```

10



(224, 224, 3)

# III.Transfer Learning

---

**plt.figure(figsize=( 12,5)):** 새로운 Matplotlib figure를 생성하고, 그 크기를 8x6 인치로 설정.

**for i in range(10)):**

**plt.subplot(2, 5, i+1):** 2x5 그리드에서 i+1번째 서브플롯을 생성.

**plt.axis('off'):** 서브플롯의 축을 비활성화.

**plt.imshow(test\_img\_list[i]):** 현재 반복된 이미지를 서브플롯에 표시.

**plt.show():** 위에서 생성한 10개의 테스트 이미지를 보여준다.

# III. Transfer Learning

---

```
1 import tensorflow as tf
2 from tensorflow.keras.models import load_model
3 from tensorflow.keras.preprocessing.image import load_img, img_to_array
4 import numpy as np
5
6 # 모델 불러오기
7 model = load_model('/content/cats_and_dogs_Xception_Colab.h5')
8
```

# III. Transfer Learning

---

```
1 import numpy as np
2 print(np.array(test_img_list).shape)
3
4 # 모델 예측
5 class_name = ['cat', 'dog']
6 pred = model.predict(np.array(test_img_list))
7 print(pred.shape)
8 print(pred)
```

```
(10, 224, 224, 3)
```

```
1/1 ————— 0s 88ms/step
```

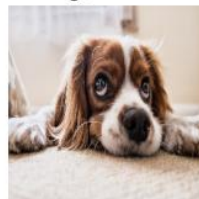
```
(10, 2)
```

```
[[1.3315387e-03 9.9866843e-01]
 [9.9851471e-01 1.4853037e-03]
 [7.8244847e-01 2.1755151e-01]
 [1.0518907e-04 9.9989486e-01]
 [2.3412757e-04 9.9976593e-01]
 [4.4362777e-01 5.5637223e-01]
 [9.9878412e-01 1.2159345e-03]
 [9.9701476e-01 2.9852916e-03]
 [9.9987864e-01 1.2137362e-04]
 [6.1383541e-03 9.9386168e-01]]
```

# III. Transfer Learning

```
1 # 예측결과 시각화
2 plt.figure(figsize=(12, 5))
3 for i in range(10):
4     plt.subplot(2, 5, i+1)
5     prediction = str(class_name[np.argmax(pred[i])])
6     probability = '{0:0.2f}'.format(100*max(pred[i]))
7     title_str = prediction + ', ' + probability + '%'
8     plt.axis('off')
9     plt.title(title_str)
10    plt.imshow(test_img_list[i])
11 plt.show()
```

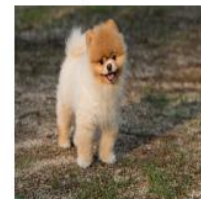
dog, 99.87%



cat, 99.85%



cat, 78.24%



dog, 99.99%



dog, 99.98%



dog, 55.64%



cat, 99.88%



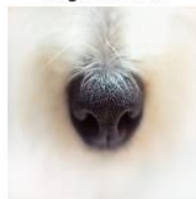
cat, 99.70%



cat, 99.99%



dog, 99.39%



# III.Transfer Learning

---

- `class_name`: 클래스의 이름을 담은 리스트
- `prediction = str(class_name[np.argmax(pred[i])])`: 예측된 결과에서 가장 높은 확률을 가진 클래스의 인덱스를 추출하고, 해당 인덱스를 사용하여 클래스 이름을 가져온다.
  - `pred[i]`는 `pred` 리스트에서 `i`번째 예측 결과.
  - `np.argmax(pred[i])`는 `pred[i]`에서 가장 높은 확률을 가진 클래스의 인덱스를 반환.
  - `Class_name[np.argmax(pred[i])]`은 `class_name` 리스트에서 해당 인덱스에 해당하는 클래스의 이름을 가져온다.
- `probability = '{0:0.2f}'.format(100*max(pred[i]))`: 예측된 결과에서 가장 높은 확률을 가져와서 백분율 형식으로 변환.
- `title_str = prediction + ', ' + probability + '%'`: 제목 문자열을 예측 클래스와 확률로 구성.

# III. Transfer Learning

## 실습) Kaggle의 cat&dog 코드를 리뷰/요약을 공유하세요

The screenshot shows the Kaggle interface for a notebook titled "Keras CNN Dog or Cat Classification" by user uysimty. The notebook has 1081 upvotes and 4515 copies. The left sidebar shows navigation options like Home, Competitions, Datasets, Models, Code, Discussions, Learn, and More. The main content area shows the notebook's title, a small image of a dog, and a description: "This Kernel for someone want to deep dive into image classification. I use CNN for classification model. If you found this Kernel helpful please up vote it. If you have some feedback and question don't forget to comment below. I have simpler model with". A link is provided: <https://www.kaggle.com/uysimty/get-start-image-classification>. The right sidebar shows the runtime (2h 18m 13s - GPU P100) and the input data (Dogs vs. Cats).

**Kaggle Interface:**

- Search:** Search
- User:** uysimty · 6Y AGO · 308,786 VIEWS
- Actions:** 1081 (upvotes), Copy & Edit 4515
- Title:** Keras CNN Dog or Cat Classification
- Tabs:** Notebook, Input, Output, Logs, Comments (169)
- Competition Notebook:** Dogs vs. Cats
- Version:** Version 32 of 32
- Runtime:** 2h 18m 13s - GPU P100
- Input:** COMPETITIONS, Dogs vs. Cats
- Output:** saasasa

**Notebook Content:**

This Kernel for someone want to deep dive into image classification. I use CNN for classification model. If you found this Kernel helpful please up vote it. If you have some feedback and question don't forget to comment below.

I have simpler model with

- <https://www.kaggle.com/uysimty/get-start-image-classification>



# III. Transfer Learning

---

```
import zipfile

with zipfile.ZipFile("/kaggle/input/dogs-vs-cats/train.zip", 'r') as zip_ref:
    zip_ref.extractall("/kaggle/working/train")

with zipfile.ZipFile("/kaggle/input/dogs-vs-cats/test1.zip", 'r') as zip_ref:
    zip_ref.extractall("/kaggle/working/test1")
```

/kaggle/working/train/train

# III. Transfer Learning

---

```
from keras.preprocessing.image import load_img
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

TensorFlow 2.0 이후로, `fit_generator()`는 단순히 `fit()`으로 통합

# III. Transfer Learning

**Classification**



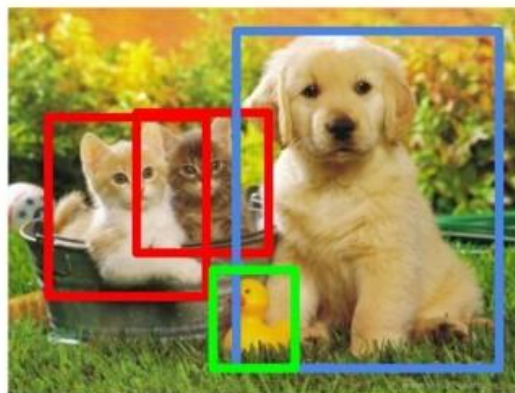
CAT

**Classification  
+ Localization**



CAT

**Object Detection**



CAT, DOG, DUCK

**Instance  
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

# III. Transfer Learning: 객체탐지의 발전사

---

## 1. CNN (Convolutional Neural Network)

– 기본 기반 기술

- 등장 배경: 이미지 분류 문제를 해결하기 위한 기본 구조
- 고정된 위치에 있는 객체를 분류만 할 수 있음

– 기술적 특이점:

정확한 위치(localization) 불가능

전체 이미지를 보고 "고양이다"는 말은 가능하지만, "어디에 고양이가 있다"는 건 못함  
기본 이미지 분류에만 사용됨

– 객체 탐지의 시작점, 하지만 "어디 있는지"는 모름

# III. Transfer Learning

---

## 2. R-CNN (Regions with CNN features, 2014)

– 등장 배경:

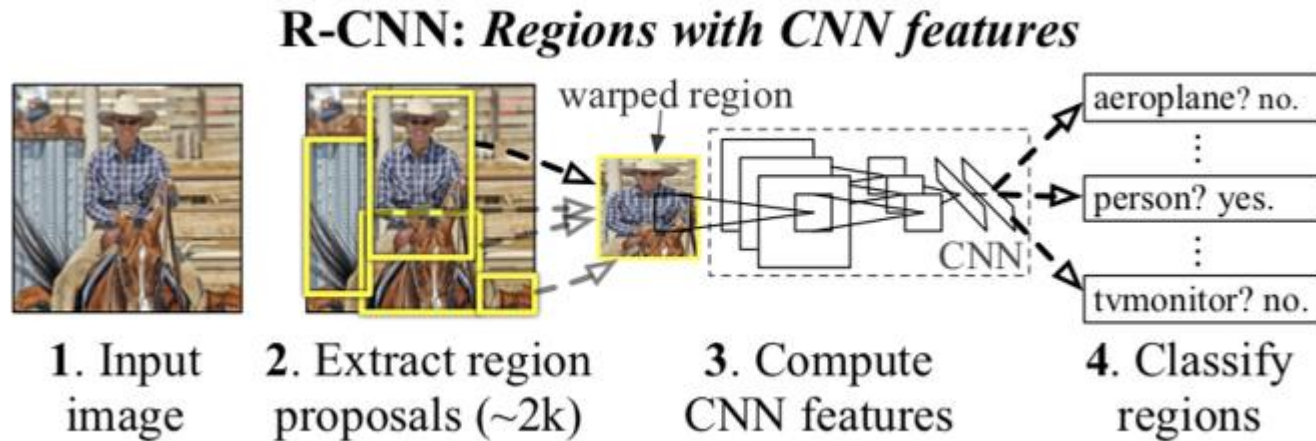
- "분류"만 하던 CNN에 위치 정보(객체 박스)를 붙이고 싶다
- Selective Search를 이용해 후보영역(region proposals)을 만든 뒤, 각 영역마다 CNN을 적용

– 기술적 특이점:

- 두 단계 방식 (Two-stage Detection)
- 후보 박스 생성 (Selective Search)
- CNN으로 각 후보 영역 분류
- 정확도는 높지만 속도가 엄청 느림 (한 이미지당 수천 번 CNN 실행)

– CNN으로 객체 위치까지 찾기 시작했지만 너무 느렸음

## III. Transfer Learning (객체탐지) **two stage detector**



**R-CNN**은 object가 있을 법한 후보 영역을 뽑아내는 "**Region proposal**" 알고리즘과 후보 영역을 분류하는 CNN을 사용

예시) 약 2,000개에 달하는 후보 이미지 각각에 대해서 convolution 연산을 수행

Proposal을 만들어내는 데에는 **Selective search**라는 비 신경망 알고리즘 사용  
>> 후보 영역의 Classification과 Bounding Box의 regression을 위해 신경망을 사용

# III. Transfer Learning

---

## 3. Fast R-CNN (2015)

- 등장 배경:

- R-CNN의 느린 속도와 중복된 CNN 연산 문제 해결 목적

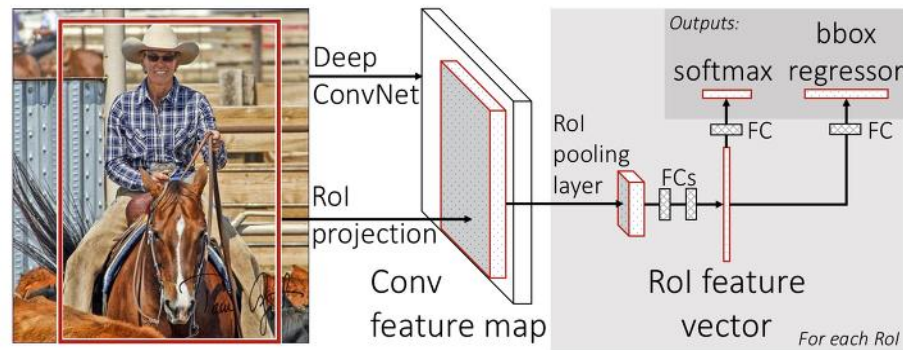
- 기술적 특이점:

- 전체 이미지에 CNN을 한 번만 적용
- 그 후, ROI(Region of Interest) Pooling을 통해 후보영역 추출
- 단일 CNN feature map을 재활용 → 속도 향상
- 학습도 end-to-end로 가능

- 이미지 한 번만 보고, 후보만 쪼개서 분류하자

# III. Transfer Learning (객체탐지) **two stage detector**

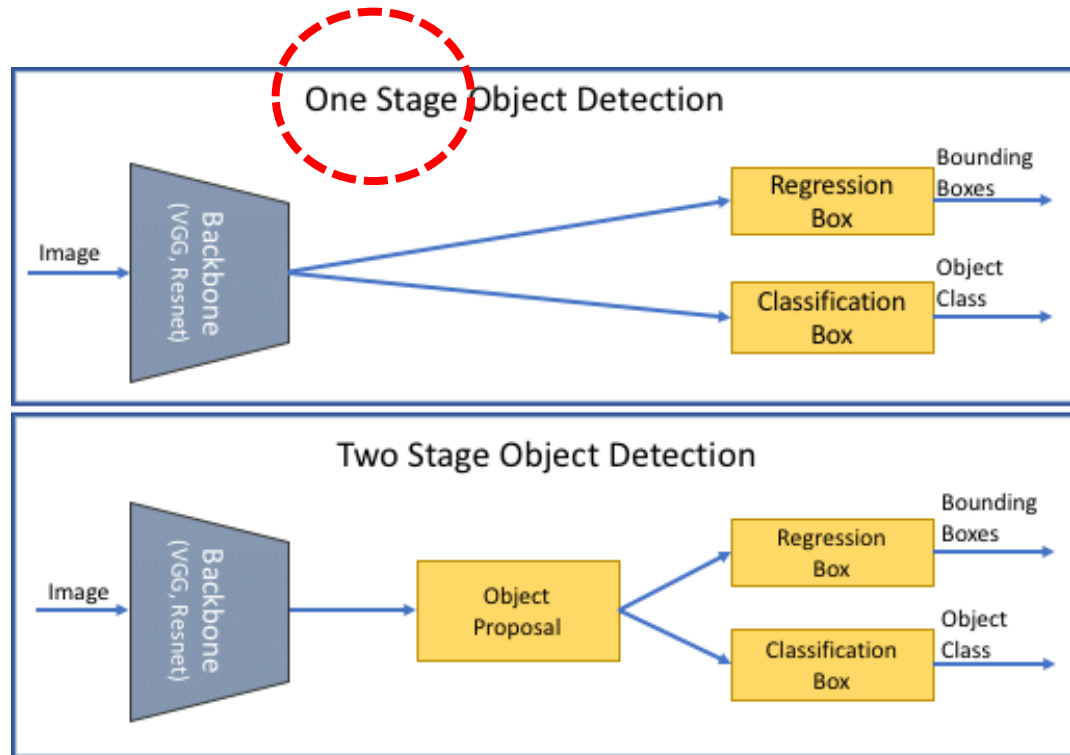
## R-CNN vs Fast R-CNN



**R-CNN의 경우,** region proposal을 selective search로 수행한 뒤 많은 수의 후보 이미지 각각에 대해서 convolution 연산을 수행. 이 경우 한 이미지에서 feature를 반복해서 추출하기 때문에 비효율적이고 느리다는 단점

**Fast R-CNN에서는** 입력 이미지를 한 번만 CNN에 통과시키고, 객체 탐지를 위한 후보 영역 추출과 특징 추출을 분리하여 수행. 앵커 박스와 특징 맵의 일부 영역을 공유하므로, RCNN보다 효율적인 속도와 좋은 탐지 성능을 제공

# III. Transfer Learning (객체탐지)

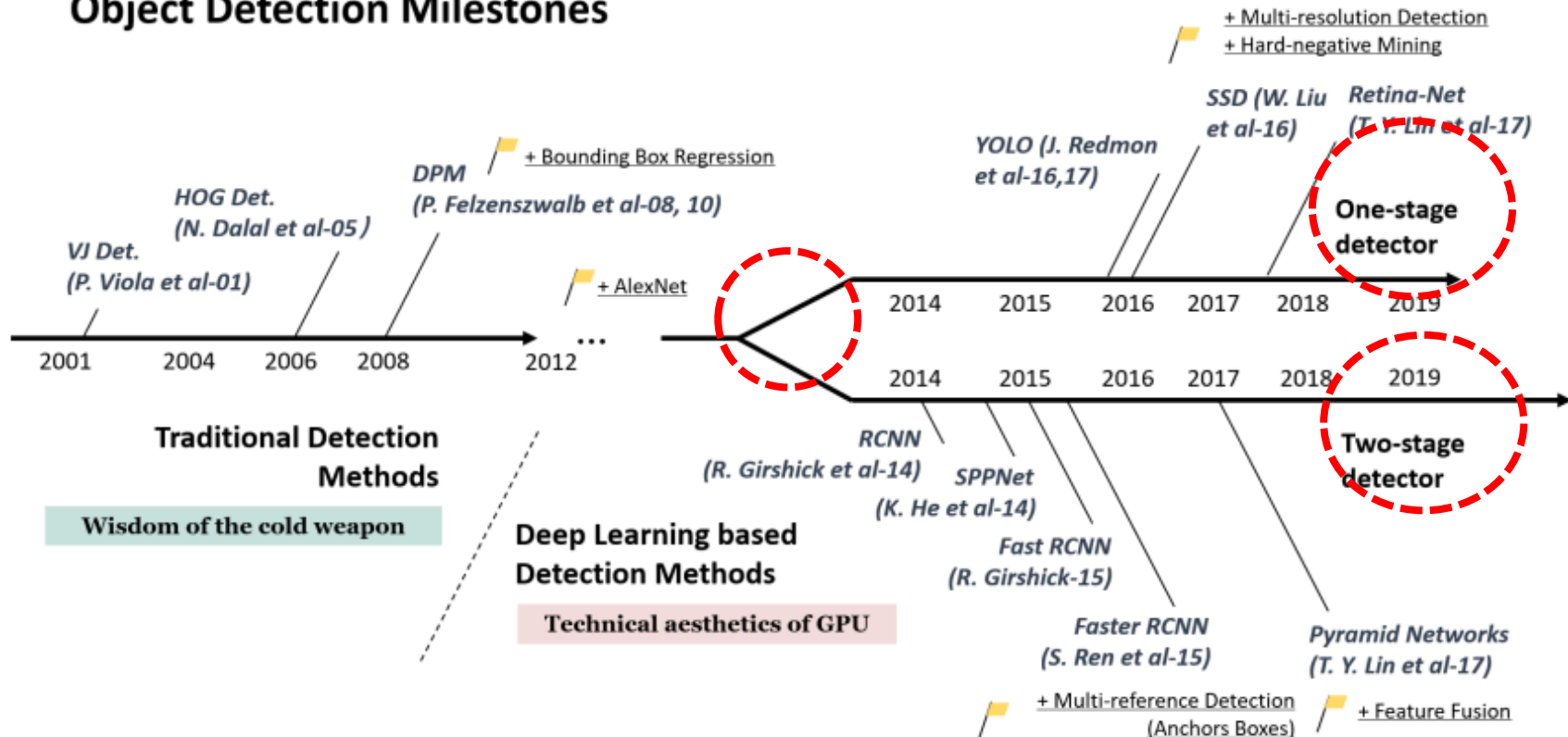


**One stage Detector :** 객체의 검출과 분류, 그리고 바운딩 박스 regression을 한 번에

**Two stage Detector :** object 위치 후보(**Region proposals**) 들을 뽑아내는 단계, 객체 검출, 이후 object가 있는지를 Classification과 정확한 바운딩 박스를 구하는 Regression을 수행하는 단계.

# III. Transfer Learning (객체탐지)

## Object Detection Milestones



# III. Transfer Learning

---

## 4. Faster R-CNN (2015)

– 등장 배경:

- Fast R-CNN은 여전히 Selective Search에 의존 (느림)
- 그래서 후보영역 생성조차도 신경망으로 대체하자

– 기술적 특이점:

- Region Proposal Network (RPN) 도입
- RPN이 직접 anchor 기반으로 후보 영역을 예측
- 완전한 end-to-end 학습 가능
- 정확도와 속도 모두 뛰어남

– “후보 영역도 CNN으로 만들자” → 진짜 완성된 객체 탐지 모델

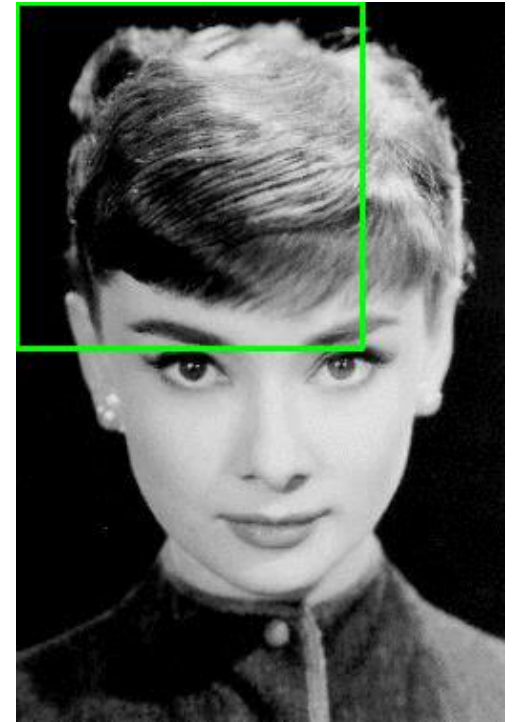
# III. Transfer Learning

---

## – Region Proposal Network (RPN) 이란?

CNN feature map 위에 슬라이딩 윈도우를 돌리면서  
Anchor Box라는 다양한 크기/비율의 박스를 겹쳐놓고

각각의 박스가 물체가 있는지 없는지, 좌표는 어디인지 예측  
즉, 후보영역 뽑기마저 CNN으로 직접 처리!



# III. Transfer Learning

---

## 5. YOLO (You Only Look Once, 2016)

- 등장 배경:
  - R-CNN 계열은 정확하지만 실시간에 부적합 (2-stage 구조)
  - 객체 탐지를 한 번에 끝내고 싶다
- 기술적 특이점:
  - 이미지를 grid로 분할 후, 각 셀에서 객체 존재 여부 + 위치 + 클래스 예측
  - 단일 CNN만으로 전체 처리
  - 실시간 처리 가능속도는 훌륭하지만, 초기 버전(YOLOv1)은 작은 객체 탐지에 약했음
- 딱 한 번 본다
- 빠르고 직관적인 객체 탐지기

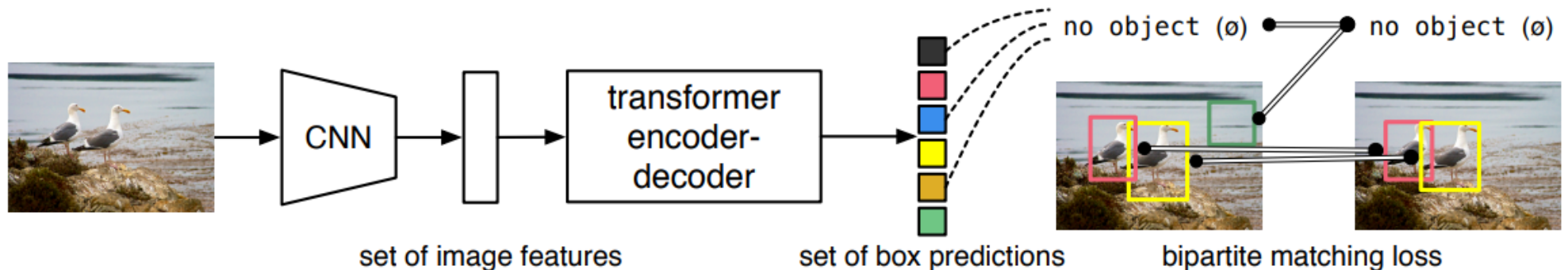
# III. Transfer Learning

## 6. DETR (DEtection TRansformer)

2020년 Facebook AI 발표, "Transformer를 이용해 객체 탐지"를 End-to-End로 처리하는 모델"

등장 배경

기존 객체 탐지 알고리즘들은 거의 다 CNN + 복잡한 후처리(Anchor, NMS) 구조였음  
anchor 설정, region proposal, NMS 등의 "핸드메이드 규칙"이 너무 많았음  
"이걸 다 없애고, 순수하게 딥러닝(Transformer)으로만 처리할 수 있을까?"



# III. Transfer Learning

---

## DETR (DEtection TRansformer)

기술적 특이점

1. Object Detection을 "Sequence Prediction"으로 봄

기존 탐지: “여기 뭔가 있어!” 박스를 예측

DETR: “이 이미지는 N개의 객체가 있고,

각각의 class와 위치는 이거야” 라고 문장처럼 예측

2. Transformer 구조 사용 (Encoder-Decoder)

Encoder: 이미지 → CNN으로 feature map 추출 후 positional encoding 추가

Decoder: 객체를 나타내는 learnable queries (예: 100개) 를 넣어

Transformer가 어떤 객체들이 있는지 예측

3. No Anchors, No NMS, No Region Proposal, NMS 전부 제거!

예측한 box들과 실제 box를 Hungarian Matching 알고리즘으로 매칭해서 학습  
이 덕분에 완전히 end-to-end 학습 가능함

# III. Transfer Learning

---

## 7. Vision Transformer (ViT) : 이미지 분류(Image Classification)

자연어 처리(NLP)에서 성공을 거둔 Transformer 구조를 컴퓨터 비전 분야에 적용하여, CNN 없이도 이미지 분류에서 높은 성능을 달성하고자 함

구조 및 특징:

입력 처리: 이미지를 고정 크기의 패치로 분할하고, 각 패치를 시퀀스로 변환

포지셔널 인코딩: 패치의 순서 정보를 보존하기 위해 위치 정보를 추가

Transformer 인코더: Self-Attention을 통해 패치 간의 관계를 학습

분류 헤드: 특별한 [CLS] 토큰을 통해 전체 이미지의 표현을 얻고, 이를 분류기로 사용

장점:

전역적인 정보 처리: Self-Attention을 통해 이미지 전체의 문맥을 고려한 특징 추출

확장성: 대규모 데이터셋에서의 학습을 통해 높은 성능 달성

단점:

데이터 효율성 부족: CNN에 비해 적은 데이터로는 성능이 저하될 수 있음

로컬 정보 처리의 한계: CNN이 갖는 지역적인 특성을 활용하지 못함

## III.Transfer Learning(객체탐지)

---

## III.Transfer Learning(객체탐지)

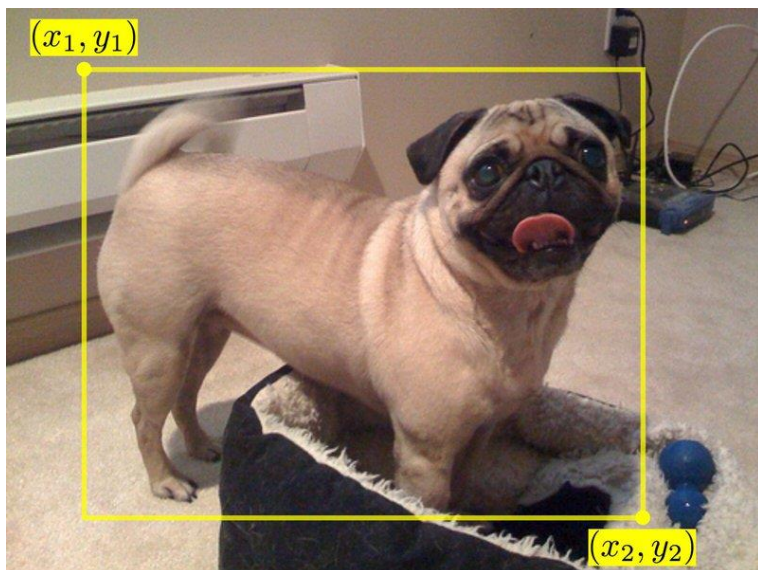
# III. Transfer Learning(객체탐지)

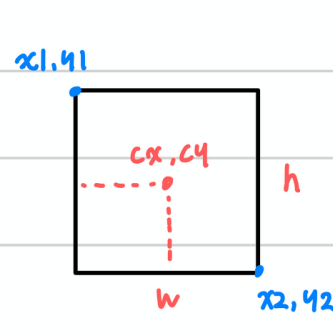
## Bounding Box

Bounding Box는 바운딩 박스, **Bbox**

corner coordinates : bounding box를 좌상점과 우하점으로 나타낸다,  
center coordinates : bounding box를 중심점과 width, height 로 나타낸다

- 코너 좌표 방식: (10, 20), (50, 70)
- 중심 좌표 방식: (30, 45), 너비 40, 높이 50




$$cx = (x1 + x2) / 2$$
$$cy = (y1 + y2) / 2$$
$$w = x2 - x1$$
$$h = y2 - y1$$

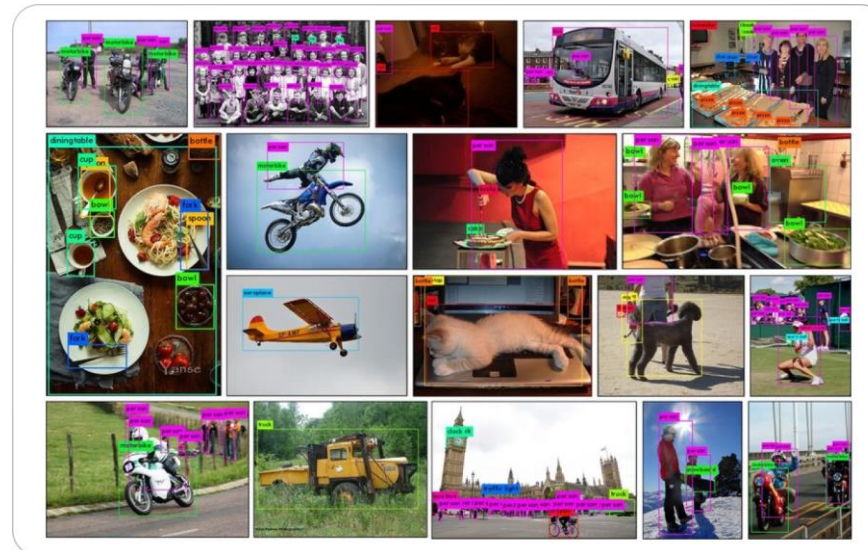
### III. Transfer Learning(객체탐지)

## COCO dataset(common objects in context)

**약 80개의 객체 카테고리**와 **약 330,000개의 이미지**로 구성.  
객체 탐지 및 분할 알고리즘의 성능 평가를 위해 많이 사용

# YOLO의 학습데이터!!

평균 정밀도(AP), 평균 재현율(AR) 등의 지표를 통해 알고리즘의 정확도와 성능을 평가



# III. Transfer Learning(객체탐지)

## mAP(means of Average Precision)

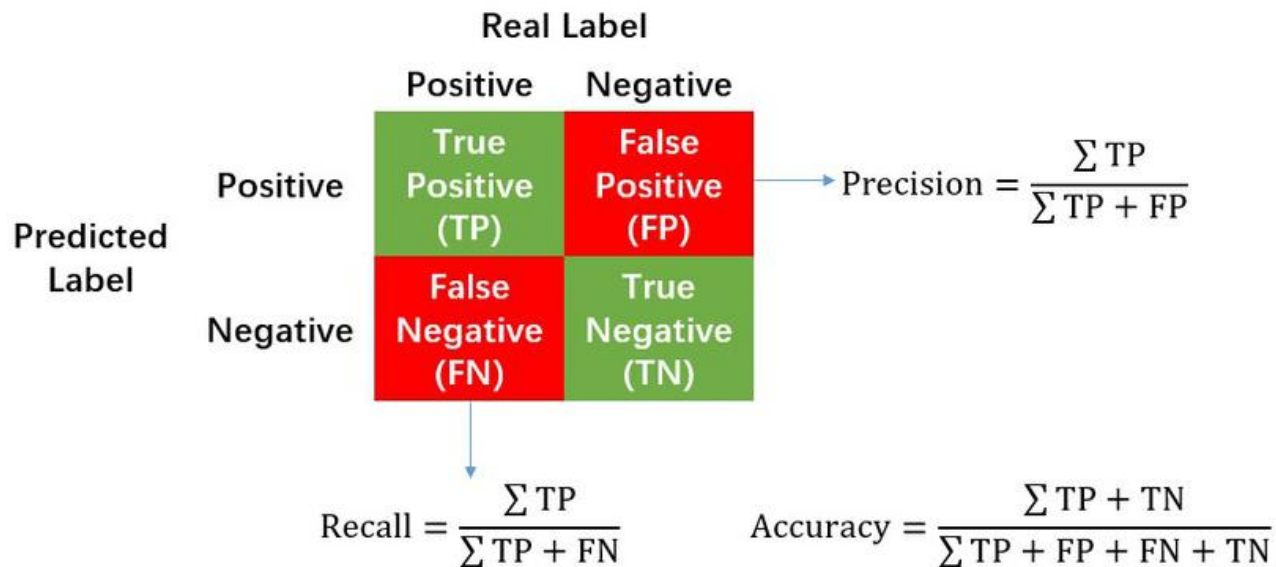
Data set내의 전체 클래스에 대해  
각 클래스에 대한 AP(Average Precision)을 계산하고 그 숫자의 평균(means)  
단일 클래스의 모델성능정보 대비 전체 Data set의 모델 성능정보 기준 !!

| Type                     | Model Name                | mAP val |
|--------------------------|---------------------------|---------|
| Two-Stage                | FRCNN - R50 FPN           | 40.2    |
| Two-Stage                | DetectorRS - R50 HTC      | 49.1    |
| Single stage w anchors   | SSD - VGG16               | 29.4    |
| Single stage w anchors   | EfficientDet - D4         | 48.3    |
| Single stage w anchors   | YOLOv5 - XL               | 49.2    |
| Single stage w/o anchors | CornetNet - Hourglass 104 | 41.8    |
| Single stage w/o anchors | CenterNet - DLA 34        | 37.4    |

$$mAP = \frac{1}{k} \sum_i^k AP_i$$

# III. Transfer Learning(객체탐지)

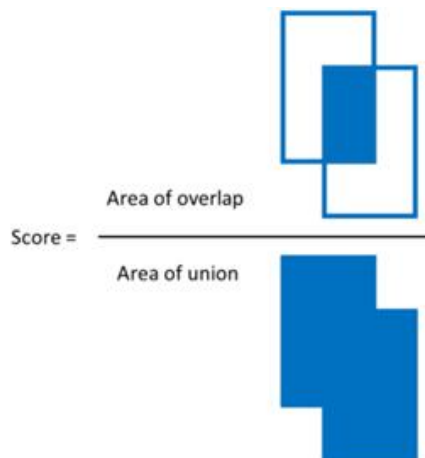
mAP(means of Average Precision)



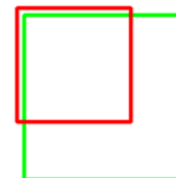
# III. Transfer Learning(객체탐지)

## IoU(Intersection over Union)

정답영역(Ground Truth bounding box)과 예측영역(predicted bounding box), 박스의 차이를 상대적으로 평가하기 위한 방법, 교차하는 영역을 합친 영역으로 나눈 값  
정답 영역과 예측 영역이 겹쳐진 부분이 클 수록 IoU의 값이 커진다

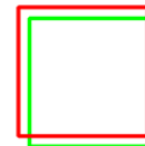


IoU: 0.4034



Poor

IoU: 0.7330



Good

IoU: 0.9264



Excellent

# III. Transfer Learning (객체탐지)

단순히 이미지를 분류하는 것이 **classification**,  
car라는 **object**의 위치를 알아내는 것이 Classification with **localization**,  
**localization**을 통해 여러 **object**를 인식하는 것을 **Detection**

Image classification



Classification with  
localization



Detection



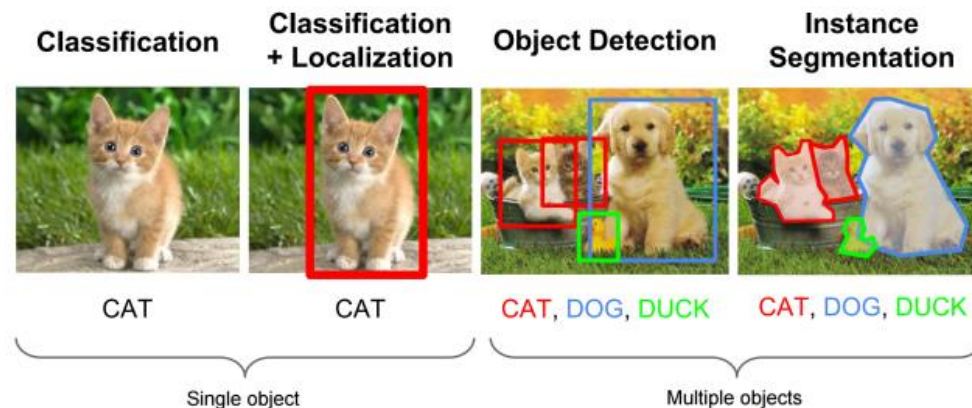
# III. Transfer Learning (객체탐지)

## 객체 탐지(Object Detection)이란? Classification + Localization

물체를 분류하는 Classification,  
물체가 박스를 통해 (Bounding box) 위치 정보 찾는 Localization

Object Detection = Classification + Localization

**Object Detection** = Multi-Labeled **Classification** + Bounding Box Regression(**Localization**)



# III. Transfer Learning (객체탐지)

---

## Image Segmentation :

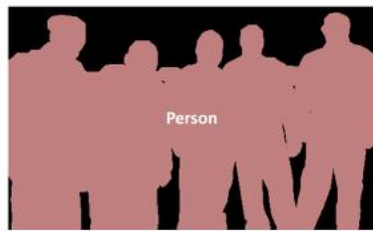
이미지를 픽셀 단위의 다양한 segments로 분할 task.  
이미지의 모든 픽셀에 라벨을 할당하는 task.

## Segmentation :

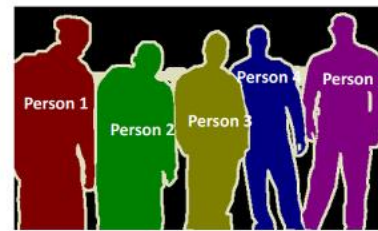
- 동일한 클래스에 해당하는 픽셀을 같은 색으로 칠하는 Semantic Segmentation.
- 동일한 클래스/다른 사물의 픽셀이면 다른 색으로 칠하는 Instance Segmentation



Object Detection



Semantic Segmentation

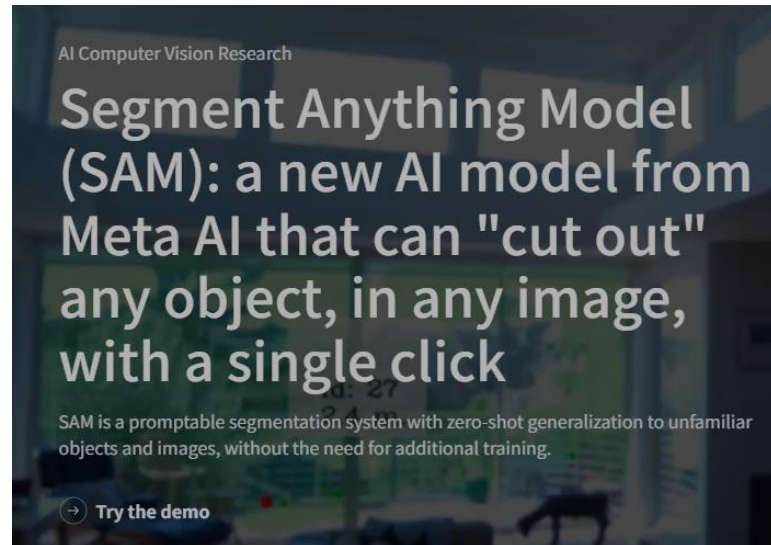


Instance Segmentation

# III. Transfer Learning (객체탐지)

---

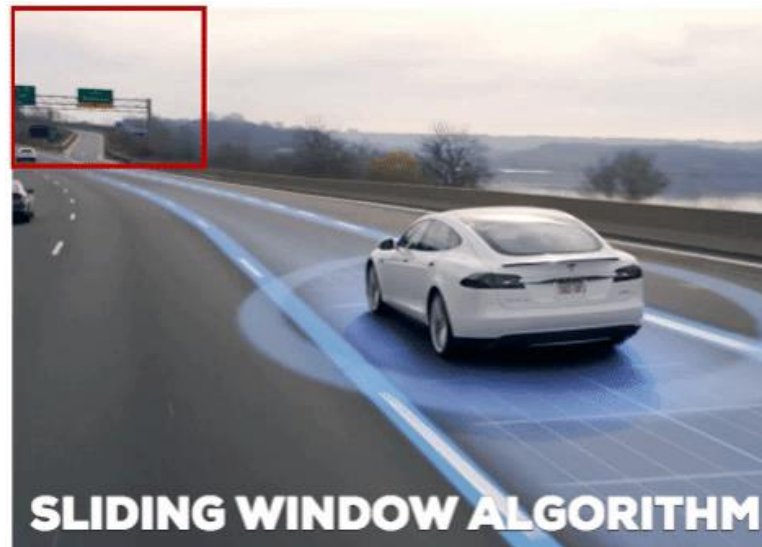
## Image Segmentation :



## III. Transfer Learning (객체탐지)

### 슬라이딩 윈도우(Sliding window)

Localization network의 입력으로 만들기 위해,  
원본 이미지에서 잘라내는 크기를 window 크기로 영역을 이동시키며(sliding)  
이미지 탐색을 수행하는 방식  
이미지 내의 오브젝트가 어떤 크기로 어떤 종횡비를 갖는지 알 수 없기 때문에 매우  
많은 종류의 Windows를 생성해야 하기 때문에 매우 비효율적인 방법일 수 있다.

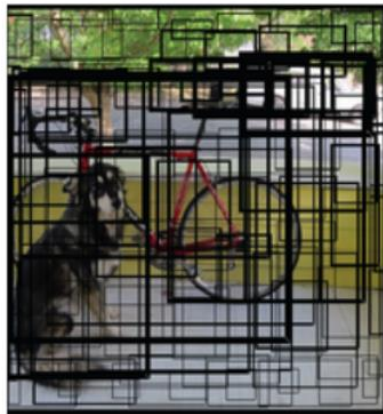


# III. Transfer Learning (객체탐지)

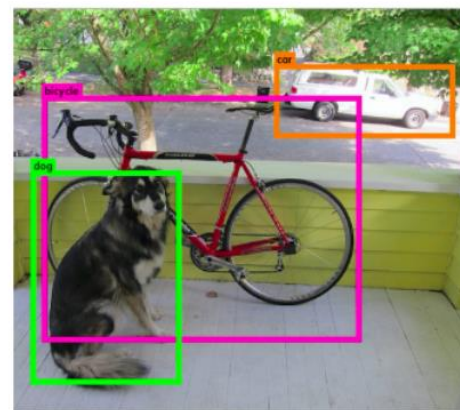
## NMS(Non-Max Suppression), 비-최대 억제

object detection 알고리즘은 탐지된 객체에 여러 개의 bounding boxes를 생성. 이 중 하나의 bounding box만을 선택해야 하는데 이때 적용하는 기법

NMS는 겹친 박스들이 있을 경우 가장 확률이 높은 박스를 기준으로 기준이 되는 IoU 이상인 것들을 없앤다.



Multiple Bounding Boxes



Final Bounding Boxes

## III. Transfer Learning (객체탐지) **one stage detector**

---

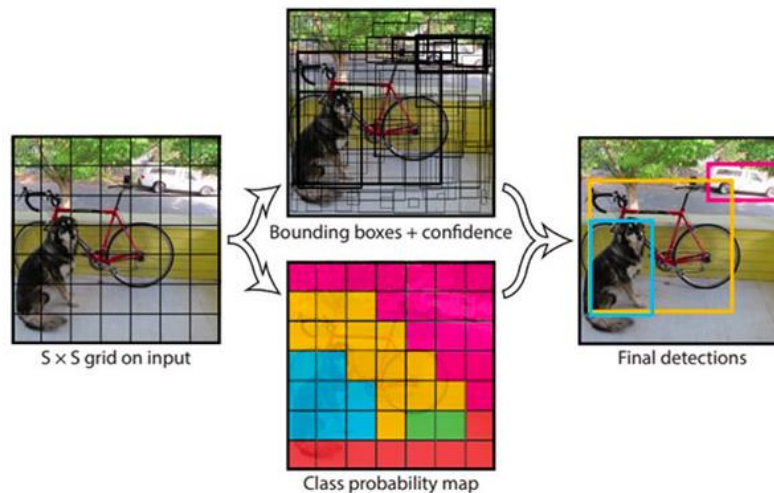


# III. Transfer Learning (객체탐지) **one stage detector**

YOLO는

이미지를 grid로 나누고, Sliding window 기법을 Convolution 연산으로 대체해 Fully Convolutional Network 연산을 통해 grid cell별로 Bbox를 얻어낸 뒤, Bbox들에 대해 NMS를 한 방식

YOLO (You Only Look Once)

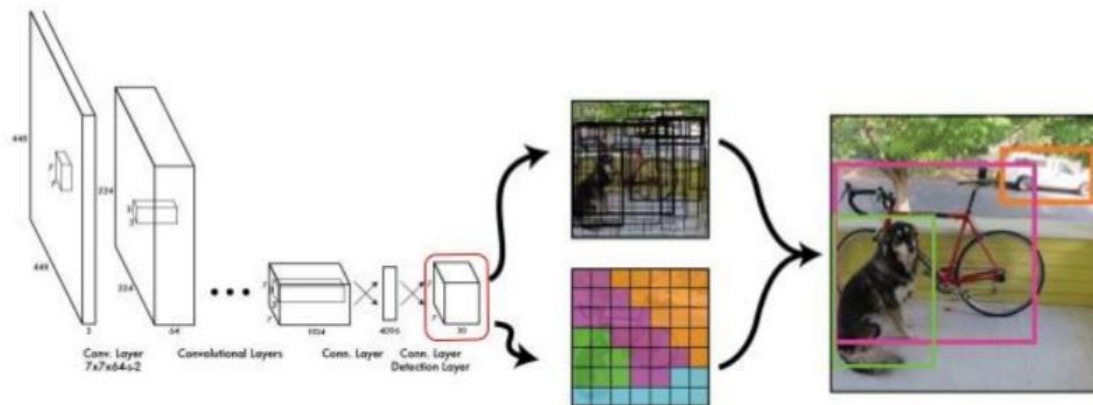


# III. Transfer Learning (객체탐지)

Bounding box 조정과 Classification을 동일 신경망 구조를 통해 동시에 실행하는 통합 인식(Unified Detection)을 구현

$S \times S$ 개의 Grid 구현 > Grid내 B개의 Bounding Box 예측 > Bbox내의 물체가 존재할 확률 계산(Confidence) > Grid마다 특정물체의 클래스 확률 계산

YOLO: You Only Look Once



# III. Transfer Learning (객체탐지)

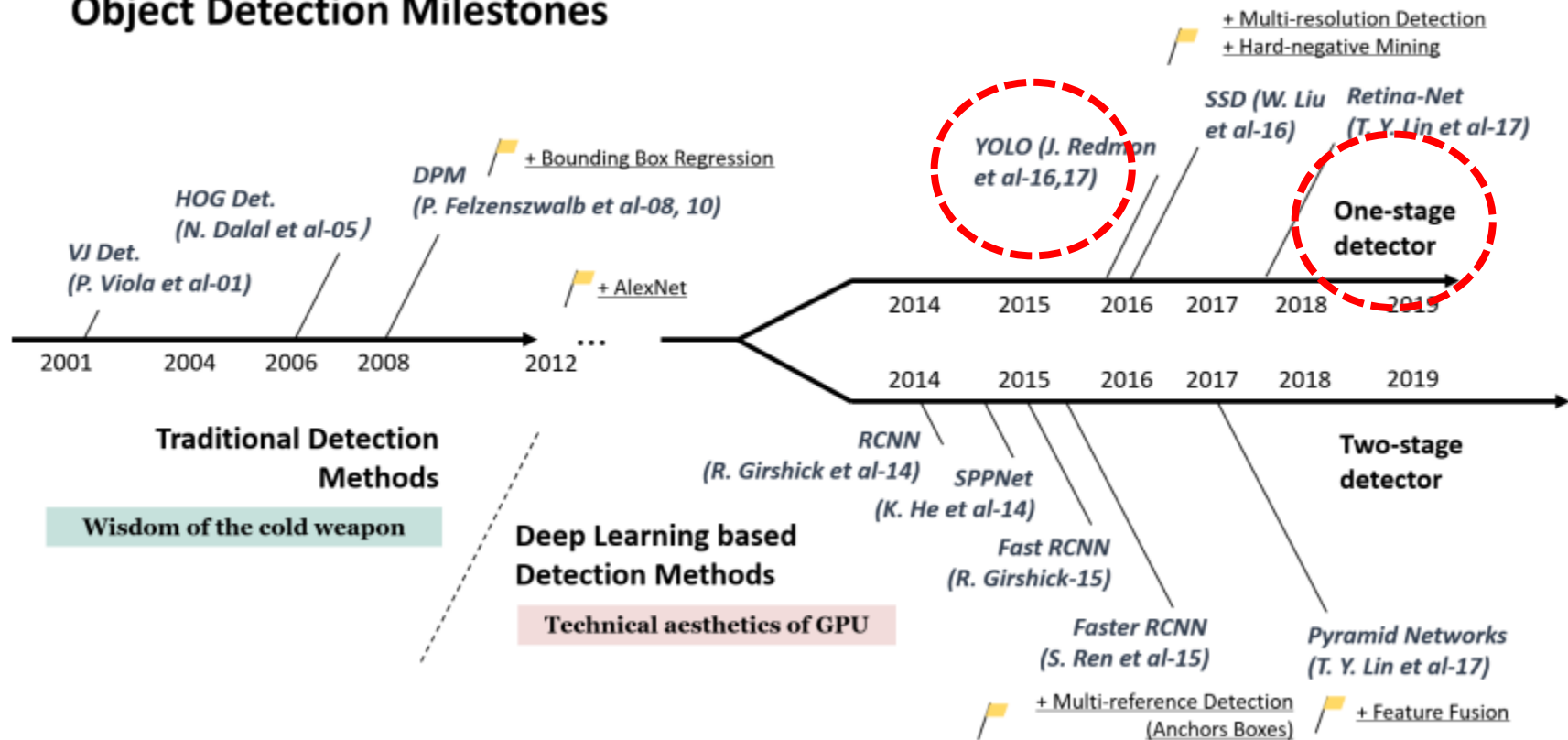
---

## YOLO 앵커박스

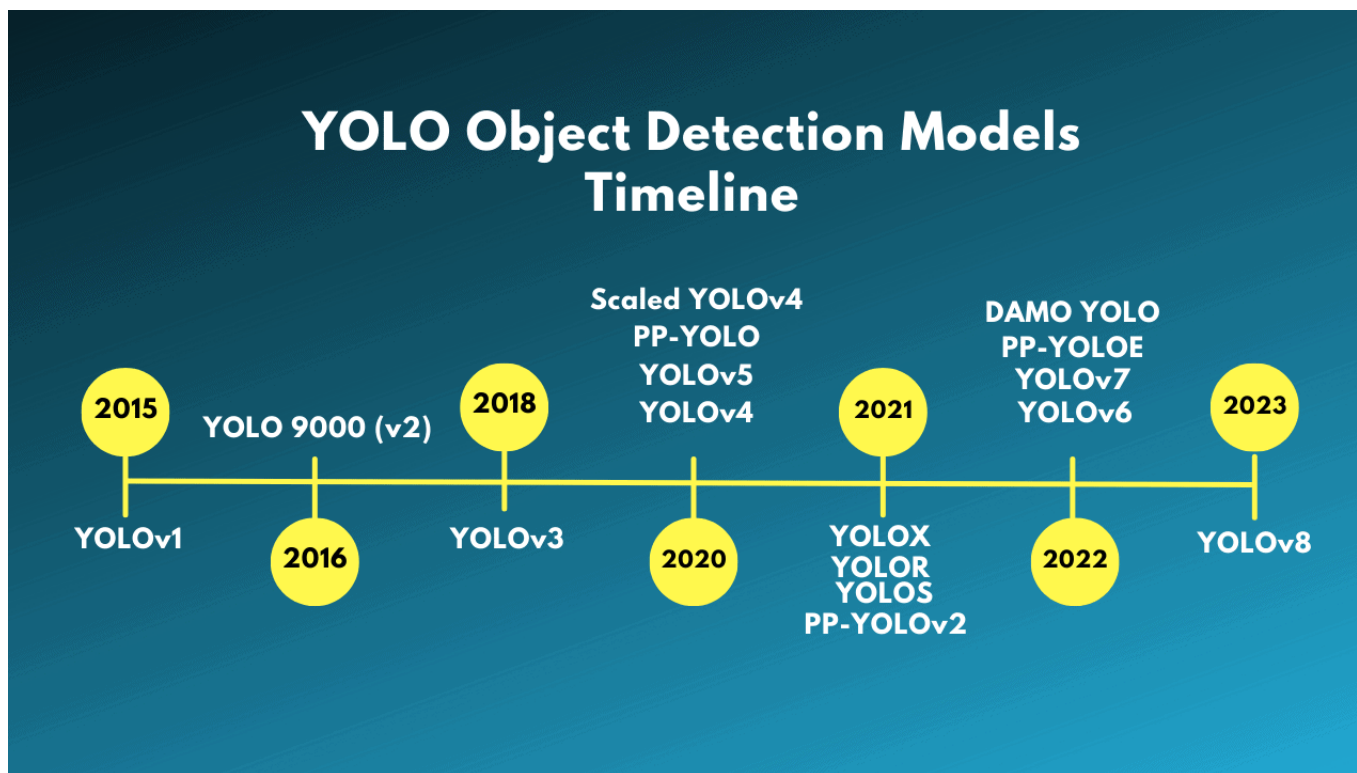
- 1 이미지를  $s \times s$  그리드로 나눔
- 2 각 셀마다 B개의 앵커박스를 던짐
- 3 각 앵커박스는  $\rightarrow (x, y, w, h, \text{confidence}, \text{class probabilities})$  예측
- 4 각 앵커박스가 예측한 위치 + 크기를 통해 진짜 객체에 맞는지 확인
- 5 실제 객체(ground truth)와 겹치는 정도를 IoU (Intersection over Union)로 평가
- 6 학습할 때는 어떤 앵커가 어떤 객체를 대표하는지 (IoU가 가장 높은 앵커박스에게만 책임 부여)

# III. Transfer Learning (객체탐지)

## Object Detection Milestones



# III. Transfer Learning (객체탐지)



|                | YOLOv5 | YOLOv6 | YOLOv7 | YOLOv8  |
|----------------|--------|--------|--------|---------|
| Inference Time | 19ms   | 22ms   | 19ms   | 16.1 ms |

# III.Transfer Learning(YOLO)

## mAP (mean Average Precision) @0.5

모델의 \*\*정확도(Precision)\*\*와 \*\*재현율(Recall)\*\*의 조합을 종합적으로 측정하는 지표 "모델이 객체를 얼마나 잘 탐지하고, 정확히 위치시켰는가?".

## "IoU ≥ 0.5일 때" 정확도(Precision)의 평균 값을 의미

예측 박스와 정답 박스가 50% 이상 겹치면 성공(True Positive), 모든 클래스의 평균 정확도.

| 모델      | mAP@0.5 | 추론 속도(ms) | 주요 특징                         |
|---------|---------|-----------|-------------------------------|
| YOLOv5  | 93.4%   | 7.1       | 빠른 추론 속도와 높은 정밀도              |
| YOLOv8  | 94.5%   | 15.9      | 높은 리콜과 다양한 비전 작업 지원           |
| YOLOv11 | 95.3%   | 7.7       | 정확도와 속도의 균형, 다양한 환경에 적합       |
| YOLOv12 | 96.1%   | 6.5       | 향상된 정확도와 추론 속도, 고정밀 객체 탐지에 적합 |

# III. Transfer Learning(YOLO)

---

## 1. 학습 출력 관련 주요 용어

**Bounding Box (바운딩 박스):** 객체를 감싸는 사각형 영역  
바운딩 박스는 (x, y) 좌표와 (width, height)로 정의.

**Confidence Score (신뢰도 점수):** 모델이 특정 객체를 탐지했을 때 해당 객체가 있을 확률

**Class Probability (클래스 확률):** 바운딩 박스 안에서 탐지된 객체가 특정 클래스에 속할 확률

**IoU (Intersection over Union):** 예측한 바운딩 박스와 실제 정답 바운딩 박스의 겹치는 부분의 비율. IoU는 (겹치는 영역의 면적) / (두 박스의 합집합 면적)으로 계산.

**IoU Threshold (IoU 임계값):** IoU가 일정 임계값 이상인 경우, 객체 탐지가 정확하다고 간주  
예를 들어, IoU가 0.5 이상이면 예측된 객체가 실제 객체와 일치한다고 판단

**AP (Average Precision):** 모델의 정확도를 평가하는 지표로, Precision-Recall Curve(정밀도-재현율 곡선)를 기반으로 계산됩니다. AP는 여러 IoU 임계값에 대해 계산된 평균값.

**mAP (mean Average Precision):** AP를 여러 클래스에 대해 평균화한 값

# III. Transfer Learning(YOLO)

---

## 2. 모델 평가 관련 주요 용어

- Precision (정밀도): 모델이 예측한 객체 중에서 실제로 객체가 맞는 비율.  
 $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$  TP(True Positive): 실제 객체를 정확히 탐지한 경우
- FP(False Positive): 객체가 없는데 객체가 있다고 잘못 예측한 경우
- Recall (재현율): 실제 객체 중에서 모델이 정확히 탐지한 비율.  
 $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- FN(False Negative): 실제 객체가 있는데 모델이 이를 탐지하지 못한 경우
- F1-Score: Precision과 Recall의 조화 평균으로, 두 지표의 균형을 평가  
두 값이 매우 다르면 F1-Score가 낮습니다.  
 $\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

# III. Transfer Learning(YOLO)

---

## YOLO에서 중요하게 여기는 평가 기준

- **inference Speed (추론 속도):**  
모델이 실제 환경에서 얼마나 빠르게 동작하는지를 평가하는 지표
- **mAP@0.5 (mean Average Precision at IoU threshold 0.5):**  
일반적으로 객체 탐지 모델의 성능을 평가하는 주요 지표로,  
IoU 임계값을 0.5로 설정하고 평균 정밀도를 계산.

# III. Transfer Learning(YOLO 8)

---

**YOLO 8**

## III. Transfer Learning(YOLO 8)

---

### ▼ Install

Pip install the ultralytics package including all [requirements](#) in a [Python>=3.7](#) environment with [PyTorch>=1.7](#).

```
pip install ultralytics
```

### ▼ Usage

#### CLI

YOLOv8 may be used directly in the Command Line Interface (CLI) with a `yolo` command:

```
yolo predict model=yolov8n.pt source='https://ultralytics.com/images/bus.jpg'
```

`yolo` can be used for a variety of tasks and modes and accepts additional arguments, i.e. `imgsz=640`. See the YOLOv8 [CLI Docs](#) for examples.

**Ultralytics 패키지를 설치** 후 모델을 지정하여 예측 시행

# III. Transfer Learning(YOLO 8)

---

## Python

YOLOv8 may also be used directly in a Python environment, and accepts the same [arguments](#) as in the CLI example above:

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolov8n.yaml") # build a new model from scratch
model = YOLO("yolov8n.pt") # load a pretrained model (recommended for training)

# Use the model
model.train(data="coco128.yaml", epochs=3) # train the model
metrics = model.val() # evaluate model performance on the validation set
results = model("https://ultralytics.com/images/bus.jpg") # predict on an image
path = model.export(format="onnx") # export the model to ONNX format
```

# III. Transfer Learning(YOLO 8)

## ▼ Detection

See [Detection Docs](#) for usage examples with these models.

| Model                   | size<br>(pixels) | mAP <sup>val</sup><br>50-95 | Speed<br>CPU ONNX<br>(ms) | Speed<br>A100 TensorRT<br>(ms) | params<br>(M) | FLOPs<br>(B) |
|-------------------------|------------------|-----------------------------|---------------------------|--------------------------------|---------------|--------------|
| <a href="#">YOLOv8n</a> | 640              | 37.3                        | 80.4                      | 0.99                           | 3.2           | 8.7          |
| <a href="#">YOLOv8s</a> | 640              | 44.9                        | 128.4                     | 1.20                           | 11.2          | 28.6         |
| <a href="#">YOLOv8m</a> | 640              | 50.2                        | 234.7                     | 1.83                           | 25.9          | 78.9         |
| <a href="#">YOLOv8l</a> | 640              | 52.9                        | 375.2                     | 2.39                           | 43.7          | 165.2        |
| <a href="#">YOLOv8x</a> | 640              | 53.9                        | 479.1                     | 3.53                           | 68.2          | 257.8        |

## ▼ Segmentation

See [Segmentation Docs](#) for usage examples with these models.

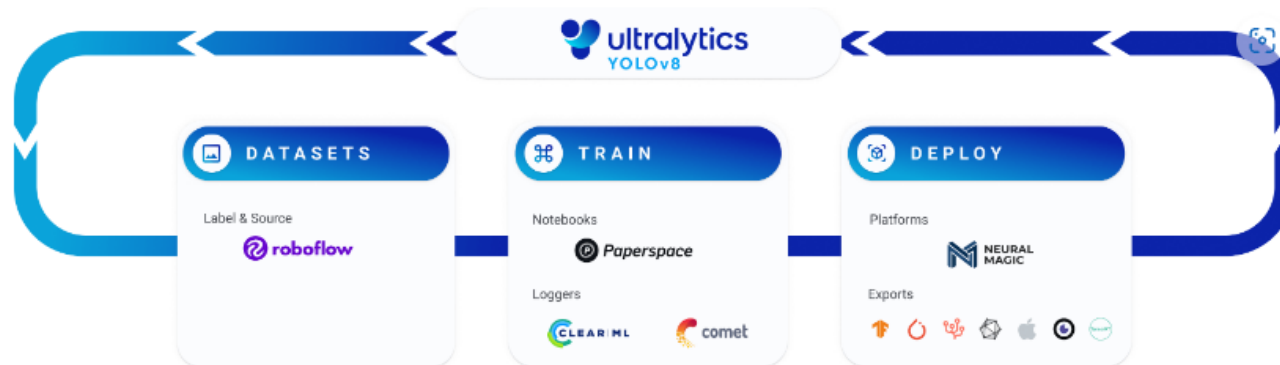
| Model                       | size<br>(pixels) | mAP <sup>box</sup><br>50-95 | mAP <sup>mask</sup><br>50-95 | Speed<br>CPU ONNX<br>(ms) | Speed<br>A100 TensorRT<br>(ms) | params<br>(M) | FLOPs<br>(B) |
|-----------------------------|------------------|-----------------------------|------------------------------|---------------------------|--------------------------------|---------------|--------------|
| <a href="#">YOLOv8n-seg</a> | 640              | 36.7                        | 30.5                         | 96.1                      | 1.21                           | 3.4           | 12.6         |
| <a href="#">YOLOv8s-seg</a> | 640              | 44.6                        | 36.8                         | 155.7                     | 1.47                           | 11.8          | 42.6         |
| <a href="#">YOLOv8m-seg</a> | 640              | 49.9                        | 40.8                         | 317.0                     | 2.18                           | 27.3          | 110.2        |
| <a href="#">YOLOv8l-seg</a> | 640              | 52.3                        | 42.6                         | 572.4                     | 2.79                           | 46.0          | 220.5        |
| <a href="#">YOLOv8x-seg</a> | 640              | 53.4                        | 43.4                         | 712.1                     | 4.02                           | 71.8          | 344.1        |

## ▼ Classification

See [Classification Docs](#) for usage examples with these models.

| Model                       | size<br>(pixels) | acc<br>top1 | acc<br>top5 | Speed<br>CPU ONNX<br>(ms) | Speed<br>A100 TensorRT<br>(ms) | params<br>(M) | FLOPs<br>(B) at 640 |
|-----------------------------|------------------|-------------|-------------|---------------------------|--------------------------------|---------------|---------------------|
| <a href="#">YOLOv8n-cls</a> | 224              | 66.6        | 87.0        | 12.9                      | 0.31                           | 2.7           | 4.3                 |
| <a href="#">YOLOv8s-cls</a> | 224              | 72.3        | 91.1        | 23.4                      | 0.35                           | 6.4           | 13.5                |
| <a href="#">YOLOv8m-cls</a> | 224              | 76.4        | 93.2        | 85.4                      | 0.62                           | 17.0          | 42.7                |
| <a href="#">YOLOv8l-cls</a> | 224              | 78.0        | 94.1        | 163.0                     | 0.87                           | 37.5          | 99.7                |
| <a href="#">YOLOv8x-cls</a> | 224              | 78.4        | 94.3        | 232.0                     | 1.01                           | 57.4          | 154.8               |

# III. Transfer Learning (YOLO 8)



| Roboflow                                                                                            | ClearML ★ NEW                                                                                              | Comet ★ NEW                                                                                                                         | Neural Magic ★ NEW                                                                |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Label and export your custom datasets directly to YOLOv8 for training with <a href="#">Roboflow</a> | Automatically track, visualize and even remotely train YOLOv8 using <a href="#">ClearML</a> (open-source!) | Free forever, <a href="#">Comet</a> lets you save YOLOv8 models, resume training, and interactively visualize and debug predictions | Run YOLOv8 inference up to 6x faster with <a href="#">Neural Magic DeepSparse</a> |

# III. Transfer Learning (YOLO 8)

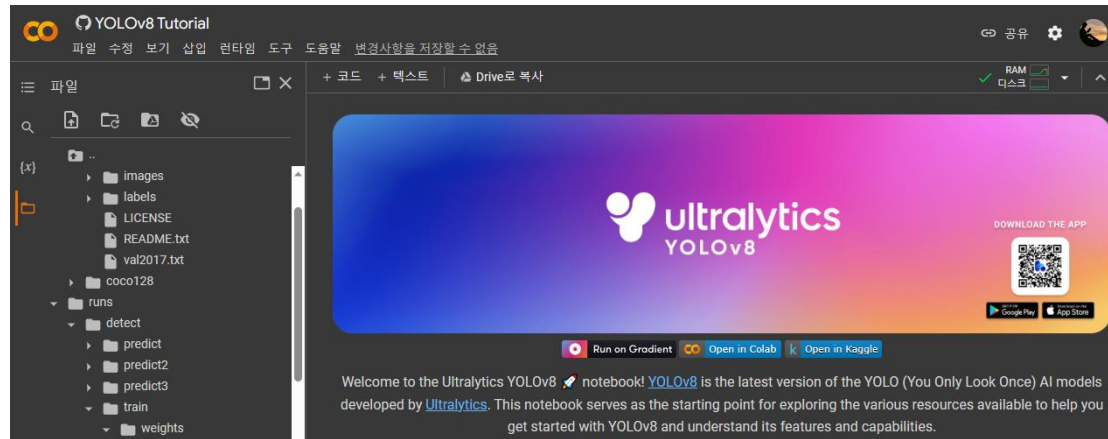
---

## Ultralytics HUB

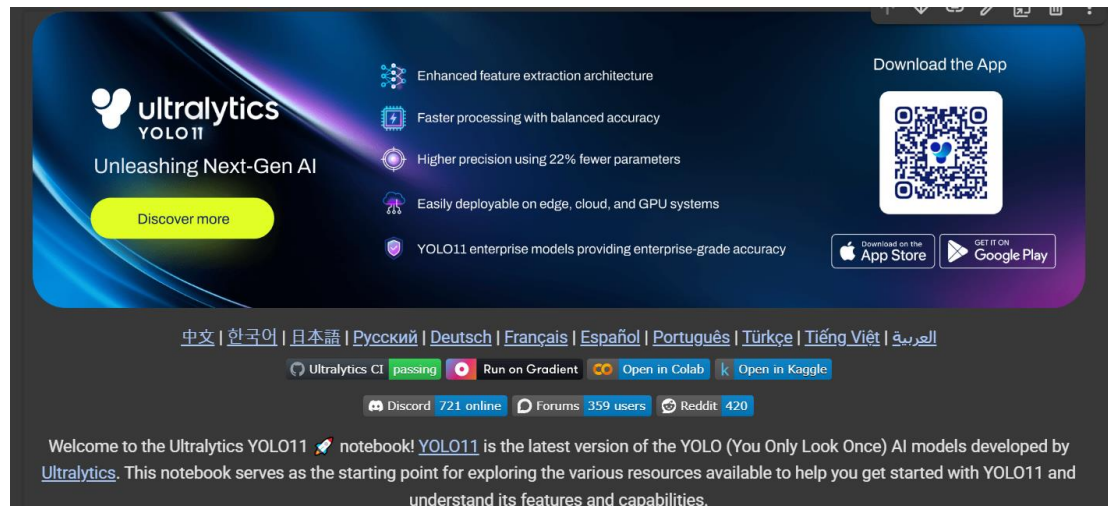
Experience seamless AI with [Ultralytics HUB](#) ★, the all-in-one solution for data visualization, YOLOv5 and YOLOv8 🚀 model training and deployment, without any coding. Transform images into actionable insights and bring your AI visions to life with ease using our cutting-edge platform and user-friendly [Ultralytics App](#). Start your journey for Free now!



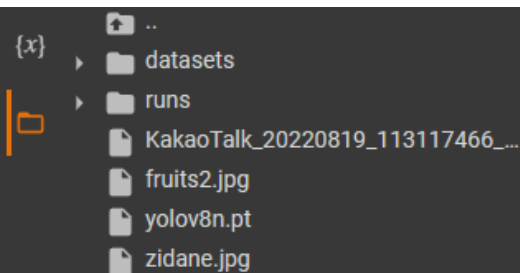
# III. Transfer Learning(YOLO8 >11 )



YOLO의 튜토리얼을 통한 객체 탐지 체험



# III. Transfer Learning(YOLO 8)



## 5. Python Usage

YOLOv8 was reimaged using Python-first principles for the most seamless Python YOLO experience yet. YOLOv8 models can be loaded from a trained checkpoint or created from scratch. Then methods are used to train, val, predict, and export the model. See detailed Python usage examples in the [YOLOv8 Python Docs](#).

```
1 from ultralytics import YOLO
2
3 # Load a model
4 model = YOLO('yolov8n.yaml') # build a new model from scratch
5 model = YOLO('yolov8n.pt') # load a pretrained model (recommended for training)
6
7 # Use the model
8 results = model.train(data='coco128.yaml', epochs=3) # train the model
9 results = model.val() # evaluate model performance on the validation set
10 results = model('https://ultralytics.com/images/bus.jpg') # predict on an image
11 success = model.export(format='onnx') # export the model to ONNX format
```

**YOLO 모델의 pretrained model 지정과 학습방법**

# III.Transfer Learning(YOLO 8 )

## Arguments YOLO 모델의 다양한 하이퍼 파라미터 사용법

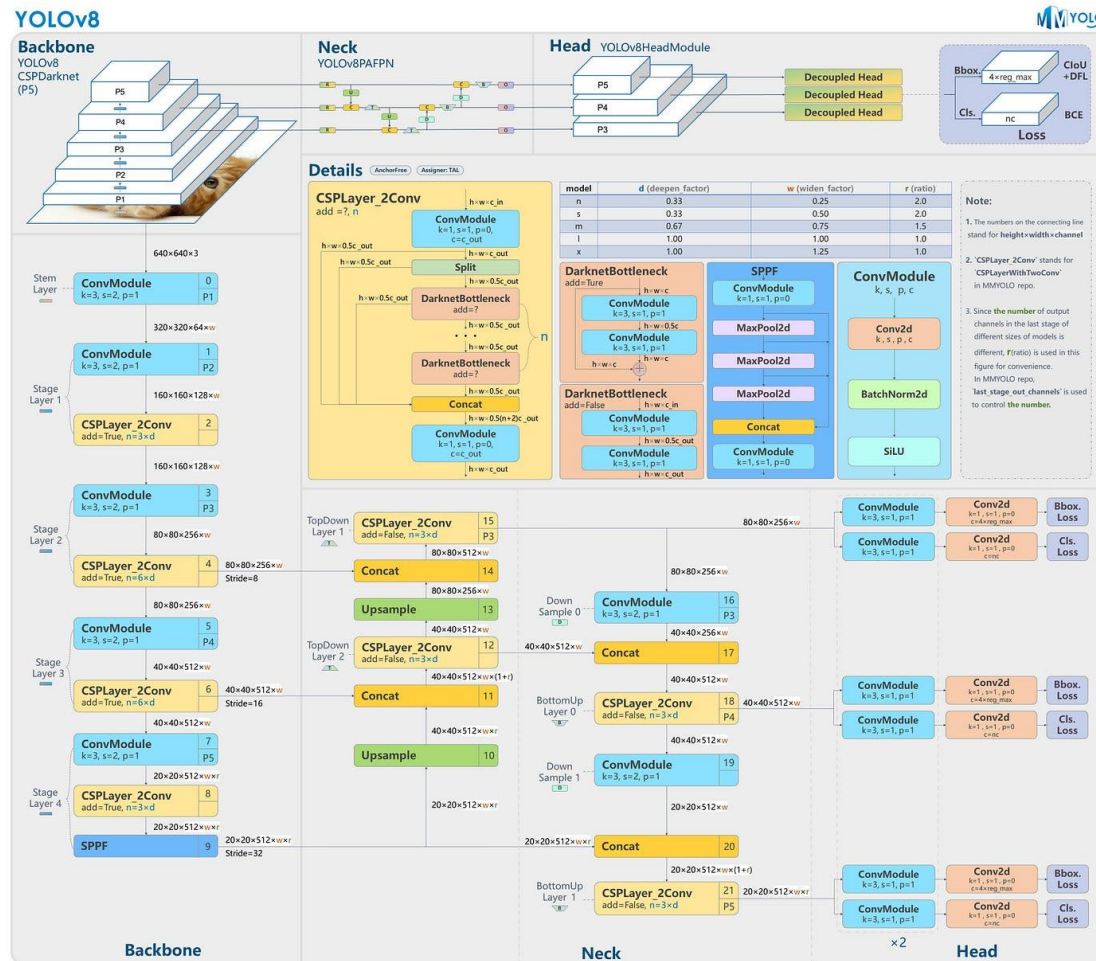
Training settings for YOLO models refer to the various hyperparameters and configurations used to train the model on a dataset. These settings can affect the model's performance, speed, and accuracy. Some common YOLO training settings include the batch size, learning rate, momentum, and weight decay. Other factors that may affect the training process include the choice of optimizer, the choice of loss function, and the size and composition of the training dataset. It is important to carefully tune and experiment with these settings to achieve the best possible performance for a given task.

| Key         | Value | Description                                                                 |
|-------------|-------|-----------------------------------------------------------------------------|
| model       | None  | path to model file, i.e. yolov8n.pt, yolov8n.yaml                           |
| data        | None  | path to data file, i.e. coco128.yaml                                        |
| epochs      | 100   | number of epochs to train for                                               |
| patience    | 50    | epochs to wait for no observable improvement for early stopping of training |
| batch       | 16    | number of images per batch (-1 for AutoBatch)                               |
| imgsz       | 640   | size of input images as integer or w,h                                      |
| save        | True  | save train checkpoints and predict results                                  |
| save_period | -1    | Save checkpoint every x epochs (disabled if < 1)                            |
| cache       | False | True/ram, disk or False. Use cache for data loading                         |

# III. Transfer Learning (YOLO 8)

|               |       |                                                                        |
|---------------|-------|------------------------------------------------------------------------|
| device        | None  | device to run on, i.e. cuda device=0 or device=0,1,2,3 or device=cpu   |
| workers       | 8     | number of worker threads for data loading (per RANK if DDP)            |
| project       | None  | project name                                                           |
| name          | None  | experiment name                                                        |
| exist_ok      | False | whether to overwrite existing experiment                               |
| pretrained    | False | whether to use a pretrained model                                      |
| optimizer     | 'SGD' | optimizer to use, choices=['SGD', 'Adam', 'AdamW', 'RMSProp']          |
| verbose       | False | whether to print verbose output                                        |
| seed          | 0     | random seed for reproducibility                                        |
| deterministic | True  | whether to enable deterministic mode                                   |
| single_cls    | False | train multi-class data as single-class                                 |
| rect          | False | rectangular training with each batch collated for minimum padding      |
| cos_lr        | False | use cosine learning rate scheduler                                     |
| close_mosaic  | 0     | (int) disable mosaic augmentation for final epochs                     |
| resume        | False | resume training from last checkpoint                                   |
| amp           | True  | Automatic Mixed Precision (AMP) training, choices=[True, False]        |
| fraction      | 1.0   | dataset fraction to train on (default is 1.0, all images in train set) |

# III. Transfer Learning (YOLO 8)



# III. Transfer Learning(YOLO 8)

---

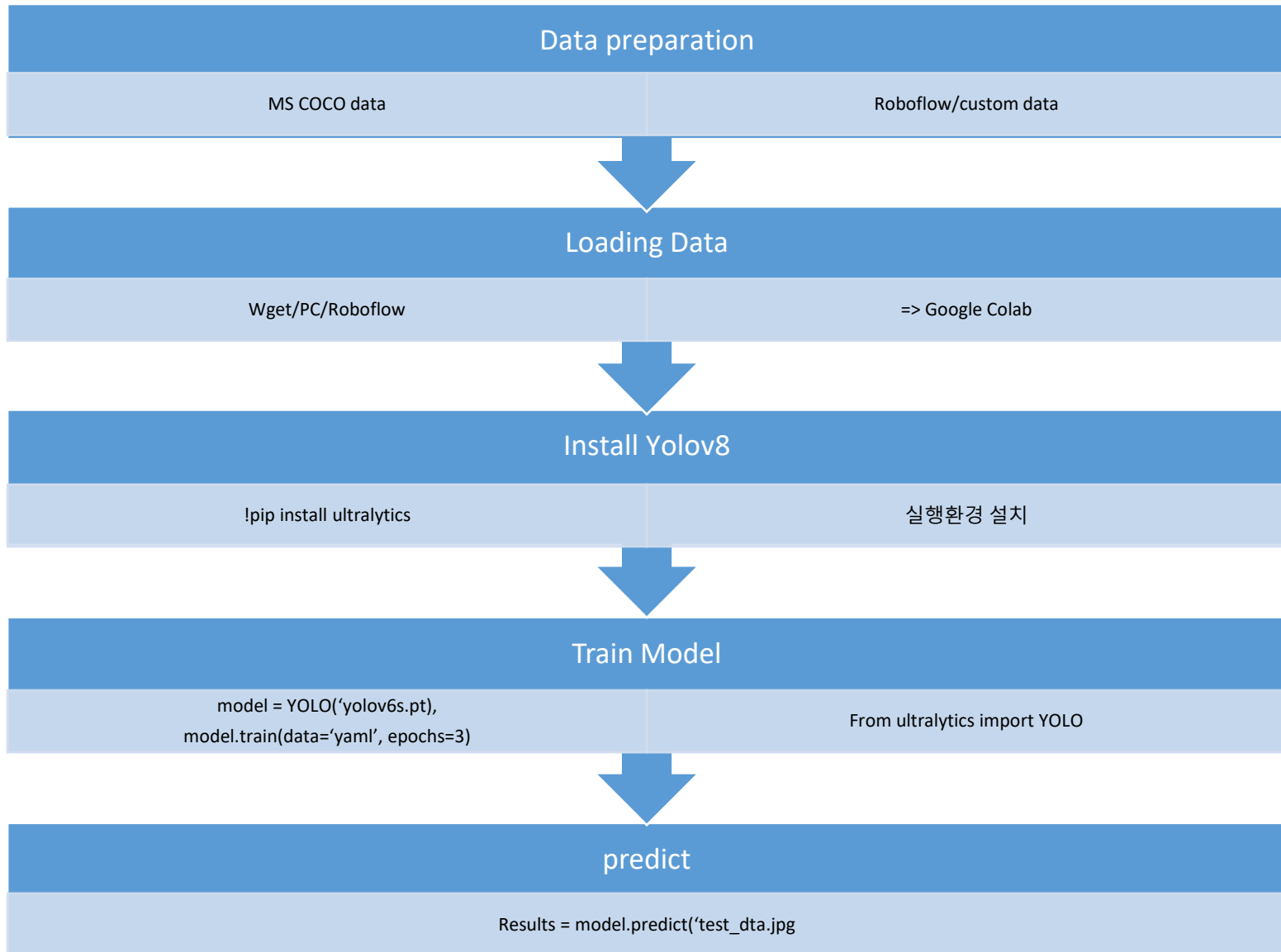
**Backbone:** responsible for **extracting high-level features** from the input data. It usually consists of **a series of convolutional layers and pooling layers**. The backbone's purpose is to capture relevant patterns and spatial information in the input data. => **CNN 특징 추출**

**Neck:** **the intermediate or transition layers**. It typically consists of additional convolutional layers or other operations that further process the features extracted by the backbone. The purpose of the neck is **to refine and transform the features into a format that is suitable for the specific task** the model is designed for. => **특징의 적절한 변환**

**Head:** responsible for **producing the desired output or predictions**. It usually includes one or more fully connected layers or convolutional layers followed by a classifier or regression layer, depending on the task. => **예측결과의 분류 및 생성**

# III. Transfer Learning(YOLO 8)

---



# **III.Transfer Learning(YOLO 8)**

---

**Code(YOLO 8)**

# III.Transfer Learning(YOLO 8)

---

```
!pip install ultralytics
```

```
Collecting ultralytics
  Downloading ultralytics-8.3.146-py3-none-any.whl.metadata (37 kB)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (4.11.0.86)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (11.2.1)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (6.0.2)
```

# III. Transfer Learning(YOLO 8)

```
from ultralytics import YOLO
```

```
# ❶ 저장된 모델 불러오기 (학습된 모델이나 기본 yolov8n.pt 사용)
```

```
model = YOLO('yolov8n.pt') # or 'runs/detect/train3/weights/best.pt'
```

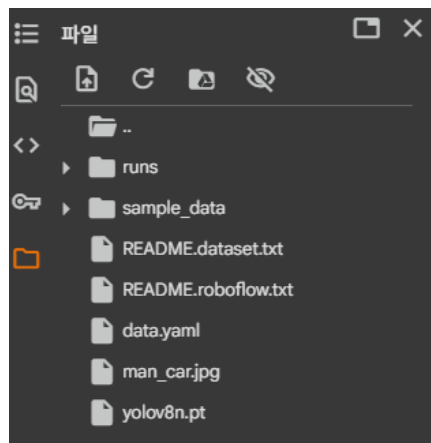
```
# ❷ 예측할 미디어 경로 설정
```

```
image_path = '/content/man_car.jpg' # 이미지 파일
```

```
# ❸ 이미지 예측
```

```
results_image = model.predict(source=image_path, save=True, show=True)
```

```
print("이미지 예측 결과:", results_image)
```



# III. Transfer Learning(YOLO 8)

Q 명령어 + 코드 + 텍스트 ▶ 모두 실행

파일

- runs
- detect
- predict
- man\_car.jpg
- sample\_data
- README.dataset.txt
- README.roboflow.txt
- data.yaml
- man\_car.jpg
- yolov8n.pt

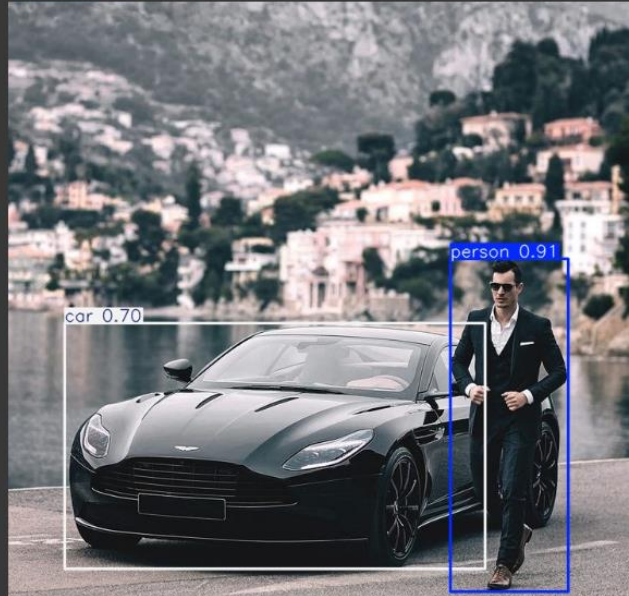
Downloading ultralytics-8.3.218-py3-none-any.whl (1.1 MB)  
----- 1.1/1.1 MB 16.3 MB/s eta 0:00:00  
Downloading ultralytics-thop-2.0.17-py3-none-any.whl (28 kB)  
Installing collected packages: ultralytics-thop, ultralytics  
Successfully installed ultralytics-8.3.218 ultralytics-thop-2.0.17

```
[4] 6초
1 from ultralytics import YOLO
2
3 # 1 저장된 모델 불러오기 (학습된 모델이나 기본 yolov8n.pt 사용)
4 model = YOLO('yolov8n.pt') # or 'runs/detect/train3/weights/best.pt'
5
6 # 2 예측할 미디어 경로 설정
7 image_path = '/content/man_car.jpg' # 이미지 파일
8
9 # 3 이미지 예측
10 results_image = model.predict(source=image_path, save=True, show=True)
11 print("이미지 예측 결과:", results_image)
12
```

Creating new Ultralytics Settings v0.0.6 file ✓  
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'  
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs\_dir=path/to/dir'. For help see <https://docs.ultralytics.com>  
Downloading <https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt> to 'yolov8n.pt': 100% -----  
WARNING ⚠ Environment does not support cv2.imshow() or PIL Image.show()

Image 1/1 /content/man\_car.jpg: 640x640 1 person, 1 car, 419.3ms  
Speed: 19.8ms preprocess, 419.3ms inference, 32.7ms postprocess per image at shape (1, 3, 640, 640)  
Results saved to **/content/runs/detect/predict**  
이미지 예측 결과: [ultralytics.engine.results.Results object with attributes:  
  
boxes: ultralytics.engine.results.Boxes object  
keypoints: None  
masks: None  
names: {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'tr

man\_car.jpg X



# III. Transfer Learning(YOLO 8)

---

```
print(type(model.names), len(model.names))  
print(model.names)
```

**최초 YOLO 모델은 coco data set를 학습한 바와 같이 80개의 객체를 인식할 수 있다**

```
<class 'dict'> 80  
{0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant',
```

# III. Transfer Learning(YOLO 8)

roboflow

Universe Public Datasets Model Zoo Blog Docs

Explore these datasets, models, and more on Roboflow Universe. →

66+ MILLION IMAGES 90,000+ DATASETS 7,000+ PRE-TRAINED MODELS

Dataset Summary

Dataset Health Check

DOWNLOADS

raw-1024

638

Try Pre-Trained Model

## Aquarium Dataset

Shared By  
Roboflow  
November 2020

License  
CC BY 4.0  
More Info

Annotations  
creatures  
Object Detection

Downloads

raw-1024

638 Images



### Dataset Details

This dataset consists of 638 images collected by Roboflow from two aquariums in the United States: The Henry Doorly Zoo in Omaha (October 16, 2020) and the National Aquarium in Baltimore (November 14, 2020). The images were labeled for object detection by the Roboflow team (with some help from SageMaker Ground Truth). Images and annotations are released under a Creative Commons By-Attribution license. You are free to use them for any purposes personal, commercial, or academic provided you give acknowledgement of their source.

# III. Transfer Learning(YOLO 8)

Public Datasets : Roboflow에서 선별하여 호스팅하는 공개 데이터셋.

roboflow Universe Public Datasets Model Zoo Blog Docs

Explore these datasets, models, and more on Roboflow Universe. →  
350+ MILLION IMAGES 500,000+ DATASETS 100,000+ PRE-TRAINED MODELS

**Aquarium Dataset >> raw-1024**

Dataset Summary  
Dataset Analytics  
DOWNLOADS  
raw-1024 638

Export Created  
5 years ago  
November 19, 2020

Available Download Formats

YOLOv8

- YOLO v3 Keras
- YOLO v4 PyTorch
- Scaled-YOLOv4
- YOLOv5 Oriented Bounding Boxes
- meituan/YOLOv6
- YOLO v5 PyTorch
- YOLO v7 PyTorch
- YOLOv8**
- YOLOv8 Oriented Bounding Boxes
- YOLOv9
- YOLOv11
- YOLOv12

**Download**

Image and Annotation Format  
YOLOv8  
TXT annotations and YAML config used with YOLOv8.

Download Options  
☐ Download zip to computer  
Downloads all images, annotations, and classes.  
☒ Show download code  
Custom train this dataset using the provided code snippet in a notebook.

Cancel Continue

**Download**

Jupyter Terminal Raw URL

Use this code to download and unzip your dataset via the command line on any \*nix machine:

```
curl -L "https://public.roboflow.com/ds/c75e1c4f4?key=5BCTbB5LTm" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

Done Choose a Model

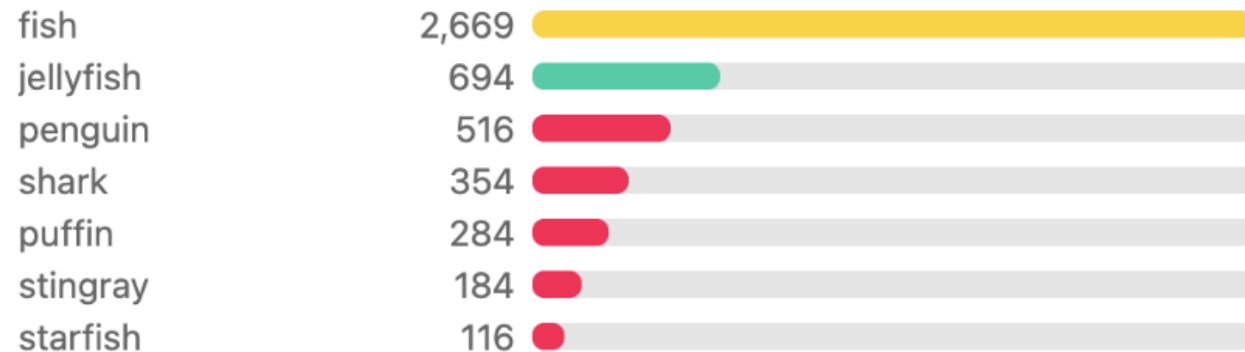
# III. Transfer Learning(YOLO 8)

```
curl -L "https://public.roboflow.com/ds/c7SehbkfK4?key=5BCTbB5LTm" > roboflow.zip;  
unzip roboflow.zip; rm roboflow.zip
```

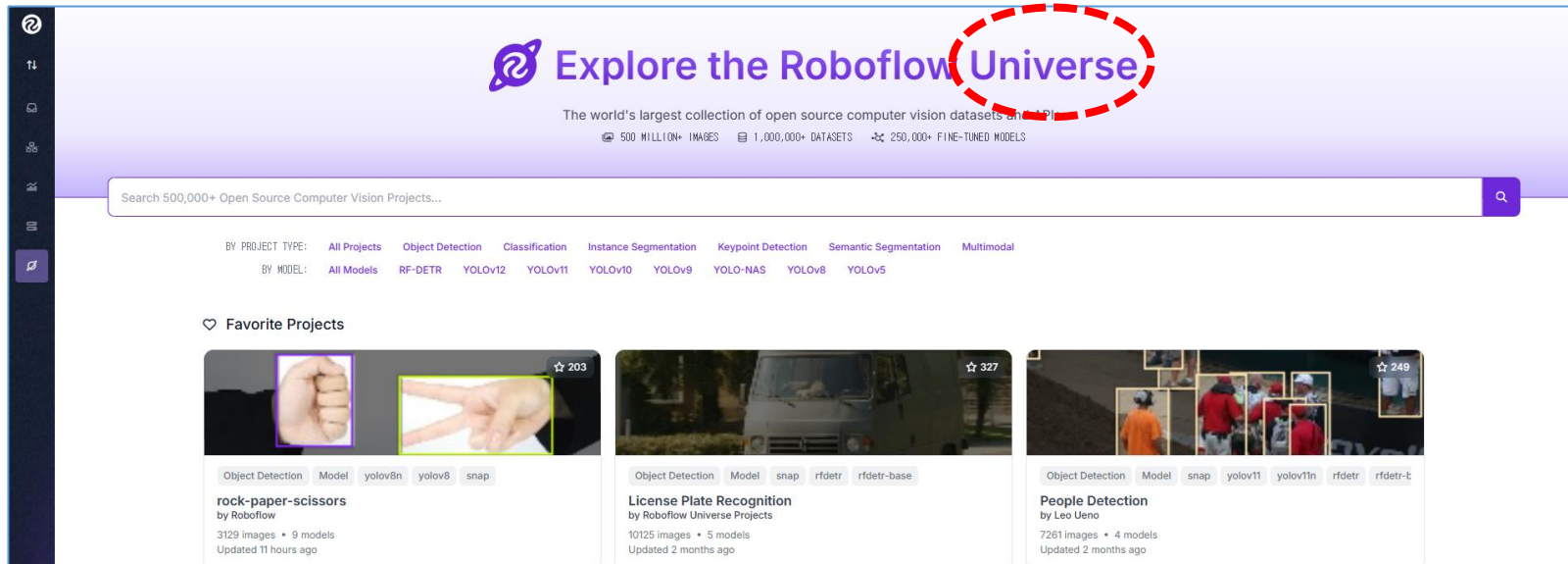
## Class Breakdown

The following classes are labeled: fish, jellyfish, penguins, sharks, puffins, stingrays, and starfish. Most images contain multiple bounding boxes.

## Class Balance



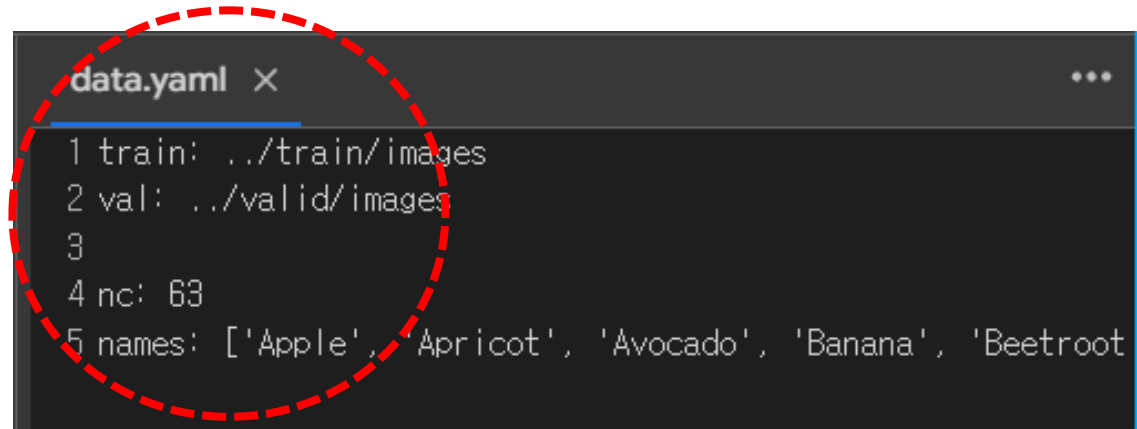
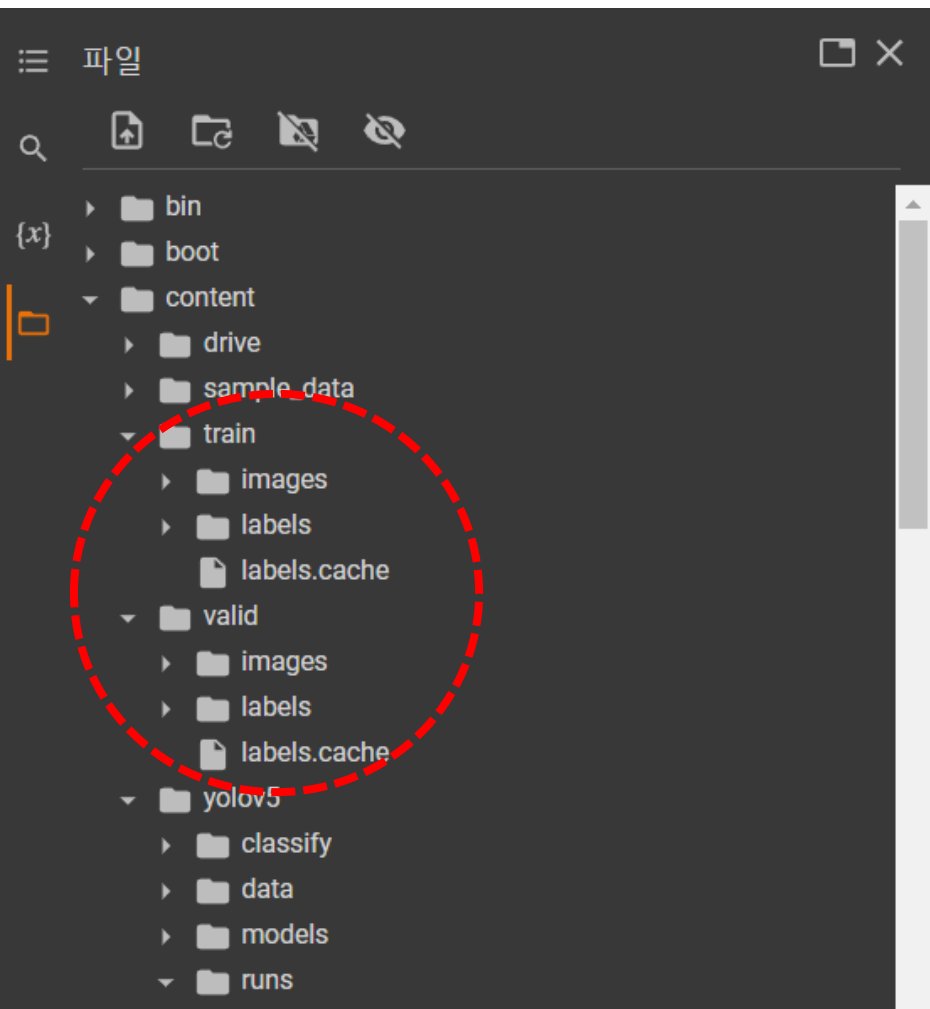
# III. Transfer Learning(YOLO 8)public data



**Roboflow의 Universe사이트에서도 데이터 코드의 선택 및 설치 가능**

**Universe Datasets은 일반 유저가 올리는 공간으로 오류 가능성이 있다 !!**

# III. Transfer Learning(YOLO 8)



데이터 URL을 설치하면(/content/에 설치)

train, valid 폴더와 data.yaml 파일이 생성된다

**Yaml 파일에는** 데이터에 대한 정보가 포함된다  
(위치, 클래스 갯수, 클래스 이름 등)

# III. Transfer Learning(YOLO 8)

---

학습데이터셋의 YAML 파일로 데이터셋에 준비된 객체(어류 7종)를 확인 가능 !!

```
!cat data.yaml
```

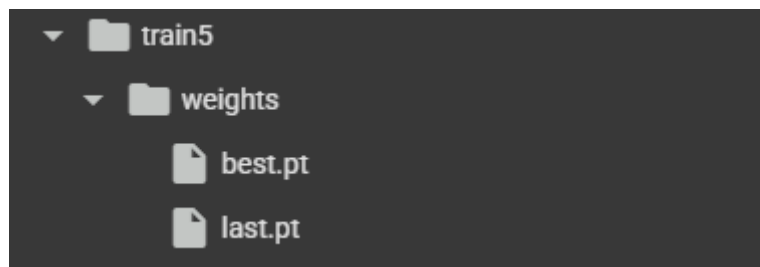
```
➡ train: ../train/images  
   val: ../valid/images  
   test: ../test/images  
  
nc: 7  
names: ['fish', 'jellyfish', 'penguin', 'puffin', 'shark', 'starfish', 'stingray']  
  
roboflow:  
  workspace: data-science-day-jeztq  
  project: aquarium-xgl5h  
  version: 1  
  license: CC BY 4.0  
  url: https://universe.roboflow.com/data-science-day-jeztq/aquarium-xgl5h/dataset/1
```

# III.Transfer Learning(YOLO 8)

```
model.train(data='/content/data.yaml', epochs=10, batch=32, imgsz=640, lr0=0.001, augment=True, patience=10, save=True)
```

**모델의 훈련을 위해 파라미터를 선택(학습데이터셋의 YMAL 파일 위치 입력에 주의 !!)**

```
fitness: np.float64(0.3881757112000368)
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.38387,  0.54003,  0.26967,  0.19332,  0.30051,  0.39508,  0.46326])
names: {0: 'fish', 1: 'jellyfish', 2: 'penguin', 3: 'puffin', 4: 'shark', 5: 'starfish', 6: 'stingray'}
plot: True
results_dict: {'metrics/precision(B)': np.float64(0.6724563040086318), 'metrics/recall(B)': np.float64(0.5633853324948074), 'metrics/mAP50(B)': np.float64(0.6086682946379455),
'metrics/mAP50-95(B)': np.float64(0.3636765352624914), 'fitness': np.float64(0.3881757112000368)}
save_dir: PosixPath('runs/detect/train5')
speed: {'preprocess': 0.2139230157515308, 'inference': 7.410926456683715, 'loss': 0.00018432282967131776, 'postprocess': 7.4196420157566925}
task: 'detect'
```



# III. Transfer Learning(YOLO 8)

---

```
print(type(model.names), len(model.names))  
print(model.names)
```

**학습 후의 YOLO 모델은 coco dataset과 달리 학습한 객체를 인식할 수 있다 !!**

```
<class 'dict'> 7  
{0: 'fish', 1: 'jellyfish', 2: 'penguin', 3: 'puffin', 4: 'shark', 5: 'starfish', 6: 'stingray'}
```

# III. Transfer Learning(YOLO 8)

```
from ultralytics import YOLO
```

```
# ❶ best.pt 모델 로드
```

```
model = YOLO('runs/detect/train/weights/best.pt') # 경로를 본인의 best.pt로 수정!
```

```
# ❷ 예측할 이미지 경로
```

```
image_path = '/content/jellyfish.jpg' # 예측할 이미지 경로
```

```
# ❸ 예측 수행
```

```
results = model.predict(
```

```
    source=image_path, # 이미지 파일 경로 (또는 폴더 경로, 비디오 경로, 0: 웹캠)
```

```
    save=True,        # 예측 결과 이미지 저장
```

```
    show=True,        # 팝업 창에 결과 출력
```

```
    conf=0.5,         # 탐지 신뢰도 threshold
```

```
    device=0          # GPU: 0 / CPU: 'cpu'
```

```
)
```

```
# ❹ 결과 확인
```

```
for result in results:
```

```
    print(result boxes) # 감지된 객체의 경계 상자 정보
```

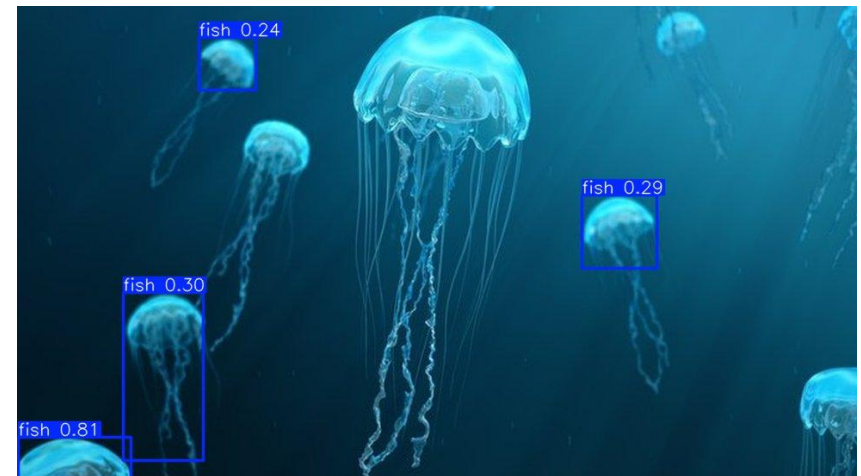
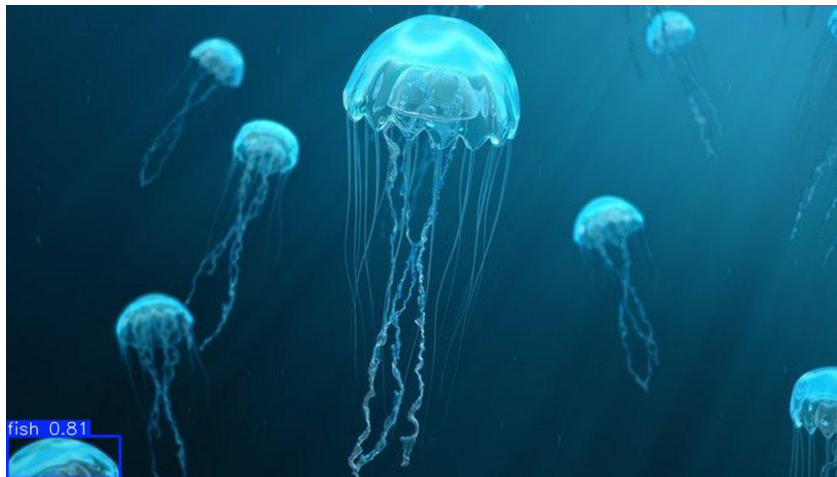
```
image 1/1 /content/jellyfish.jpg: 448x640 (no detections), 14.5ms
Speed: 6.2ms preprocess, 14.5ms inference, 1.2ms postprocess per image at shape (1, 3, 448, 640)
Results saved to runs/detect/predict4
ultralytics.engine.results.Boxes object with attributes:
```

```
cls: tensor([], device='cuda:0')
conf: tensor([], device='cuda:0')
data: tensor([], device='cuda:0', size=(0, 6))
id: None
is_track: False
orig_shape: (854, 1280)
shape: torch.Size([0, 6])
xywh: tensor([], device='cuda:0', size=(0, 4))
xywhn: tensor([], device='cuda:0', size=(0, 4))
xyxy: tensor([], device='cuda:0', size=(0, 4))
xyxyn: tensor([], device='cuda:0', size=(0, 4))
```

# III. Transfer Learning(YOLO 8)

---

conf=0.5 vs 0.2      # 탐지 신뢰도 threshold



환경구성 : 웹캠이 설치된 노트북에서 vscode 실행: ultralytics, opencv-python 설치

## III. Transfer Learning(YOLO 8) 실시간 객체 탐지

```
import cv2
from ultralytics import YOLO # YOLOv8 라이브러리

# 1. YOLO 모델 불러오기 (사전학습된 모델 사용)
model = YOLO('yolov8n.pt') # 경량화 모델로 빠르게 확인 가능

# 2. 웹캠 열기 (기본 카메라 index=0)
cap = cv2.VideoCapture(0)

# 3. 반복적으로 프레임 읽고 객체 감지
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # 4. YOLO 예측
    results = model(frame, show=False, stream=True) # stream=True: 실시간 감지

    # 5. 결과 프레임 출력
    for result in results:
        annotated_frame = result.plot() # 예측결과로 이미지 그리기
        cv2.imshow('YOLOv8 Real-time Detection', annotated_frame)

    # 6. 종료 조건 (q 키를 누르면 종료)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# 7. 자원 해제
cap.release()
cv2.destroyAllWindows()
```

# III. Transfer Learning (YOLO 8)

## YAML :

xml과 json 포맷과 같이 타 시스템 간에 데이터를 주고받을 때 약속된 포맷(규칙)이 정의되어있는 또 하나의 파일 형식

yaml은 주로 Docker Compose, Kubernetes, Flutter, Spring boot 프로젝트에서 **설정파일을 정의할때** 자주 애용된다.

| XML                                                                                                                                                                                                                                                                                                                                                                               | JSON                                                                                                                                                                                                                                           | YAML                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 &lt;Servers&gt; 2   &lt;Server&gt; 3     &lt;name&gt;Server1&lt;/name&gt; 4     &lt;owner&gt;Prajwal&lt;/owner&gt; 5     &lt;status&gt;active&lt;/status&gt; 6   &lt;/Server&gt; 7   &lt;Server&gt; 8     &lt;name&gt;Server2&lt;/name&gt; 9     &lt;owner&gt;John&lt;/owner&gt; 10    &lt;status&gt;inactive&lt;/status&gt; 11  &lt;/Server&gt; 12 &lt;/Servers&gt;</pre> | <pre>1 { 2   "Servers": [ 3     { 4       "name": "Server1", 5       "owner": "Prajwal", 6       "status": "active" 7     }, 8     { 9       "name": "Server2", 10      "owner": "John", 11      "status": "inactive" 12    } 13  ] 14 }</pre> | <pre>1 Servers: 2   - name: Server1 3     owner: Prajwal 4     status: active 5   - name: Server2 6     owner: John 7     status: inactive</pre> |

# III.Transfer Learning (YOLO 8)

---

## YAML :

객체 탐지(Object Detection)와 같은 작업에 사용되는 **설정 파일**.  
모델 아키텍처, 데이터 경로, 하이퍼파라미터, 학습 및 평가 설정 등 설정 값을 포함

- **데이터 경로:** 학습 및 검증 데이터셋의 경로와 형식을 지정.
- **모델 아키텍처:** 사용할 모델의 구조와 레이어를 정의.
- **하이퍼파라미터:** 학습률, 배치 크기, 가중치 감소 등 관련된 하이퍼파라미터를 설정.
- **데이터 전처리:** 이미지 크기 조정, 데이터 증강 등의 전처리 단계를 정의.
- **학습 설정:** 학습 반복 횟수(epoch), 옵티마이저 종류, 학습률 스케줄링 등 지정.
- **검증 설정:** 평가 지표, 검증 주기 등 검증에 관련된 설정을 지정.
- **클래스 정보:** 탐지할 객체 클래스의 목록을 정의.

# III. Transfer Learning (YOLO 8)

---

## 객체 탐지에서 YAML 파일을 사용하는 이유

- 1. 구성의 통합과 일관성:** YAML 파일을 사용하면 객체 탐지 시스템의 구성 요소를 하나의 파일에 통합. 모델 아키텍처, 데이터 경로, 하이퍼파라미터, 학습 및 평가 설정 등과 같은 여러 가지 요소를 하나의 장소에서 관리.
- 2. 유연성과 재사용성:** YAML 파일은 수정 가능한 설정 값을 포함하므로 사용자는 해당 값을 변경하여 다양한 실험을 수행. 모델의 하이퍼파라미터, 데이터 경로, 전처리 단계 등.
- 3. 가독성과 관리 용이성:** YAML 파일은 사람이 읽고 이해하기 쉬운 형식. 들여쓰기를 사용하여 계층적인 구조를 표현하며, 키-값 쌍의 형태로 데이터를 표현.
- 4. 설정의 분리와 추상화:** YAML 파일은 설정 값을 분리하고 추상화할 수 있는 기능을 제공. 예를 들어, 데이터 경로, 모델 아키텍처, 학습 설정 등을 YAML 파일로 정의하면 해당 값들을 추상화하여 여러 실험 또는 프로젝트에서 재사용할 수 있다.

# III. Transfer Learning(YOLO 8)

```
1 model.train(data = '/content/data.yaml', epochs=10, batch=32, imgsz=416)
```

Ultralytics YOLOv8.0.110 Python-3.10.11 torch-2.0.1+cu118 CUDA:0 (Tesla T4, 15102MiB)  
**yolo/engine/trainer:** task=detect, mode=train, model=yolov8n.pt, data=/content/data.yaml, epochs=10, patience=50, batch=32, imgsz=416,  
Downloading <https://ultralytics.com/assets/Arial.ttf> to /root/.config/Ultralytics/Arial.ttf...  
100%|██████████| 755k/755k [00:00<00:00, 22.3MB/s]  
Overriding model.yaml nc=80 with nc=7

|    | from | n | params | module                               | arguments            |
|----|------|---|--------|--------------------------------------|----------------------|
| 0  | -1   | 1 | 464    | ultralytics.nn.modules.conv.Conv     | [3, 16, 3, 2]        |
| 1  | -1   | 1 | 4672   | ultralytics.nn.modules.conv.Conv     | [16, 32, 3, 2]       |
| 2  | -1   | 1 | 7360   | ultralytics.nn.modules.block.C2f     | [32, 32, 1, True]    |
| 3  | -1   | 1 | 18560  | ultralytics.nn.modules.conv.Conv     | [32, 64, 3, 2]       |
| 4  | -1   | 2 | 49664  | ultralytics.nn.modules.block.C2f     | [64, 64, 2, True]    |
| 5  | -1   | 1 | 73984  | ultralytics.nn.modules.conv.Conv     | [64, 128, 3, 2]      |
| 6  | -1   | 2 | 197632 | ultralytics.nn.modules.block.C2f     | [128, 128, 2, True]  |
| 7  | -1   | 1 | 295424 | ultralytics.nn.modules.conv.Conv     | [128, 256, 3, 2]     |
| 8  | -1   | 1 | 460288 | ultralytics.nn.modules.block.C2f     | [256, 256, 1, True]  |
| 9  | -1   | 1 | 164608 | ultralytics.nn.modules.block.SPPF    | [256, 256, 5]        |
| 10 | -1   | 1 | 0      | torch.nn.modules.upsampling.Upsample | [None, 2, 'nearest'] |

YOLO8 : 지정된 aqua data의 yaml파일과 지정된 파라미터 설정으로 학습을 진행

# III. Transfer Learning(YOLO 8)

```
1 model.train(data = '/content/data.yaml' epochs=10, batch=32, imgsz=416)
```

Ultralytics YOLOv8.0.110 Python-3.10.11 torch-2.0.1+cu118 CUDA:0 (Tesla T4, 15102MiB)  
**yolo/engine/trainer=** task=detect, mode=train, model=yolov8n.pt, data=/content/data.yaml, epochs=10, patience=50, batch=32, imgsz=416,  
Downloading <https://ultralytics.com/assets/Arial.ttf> to /root/.config/Ultralytics/Arial.ttf...  
100%|██████████| 755k/755k [00:00<00:00, 22.3MB/s]  
Overriding model.yaml nc=80 with nc=7

```
Validating runs/detect/train/weights/best.pt...  
Ultralytics YOLOv8.0.110 Python-3.10.11 torch-2.0.1+cu118 CUDA:0 (Tesla T4, 15102MiB)  
Model summary (fused): 168 layers, 3007013 parameters, 0 gradients
```

| Class     | Images | Instances | Box(P) | R     | mAP50  | mAP50-95): |
|-----------|--------|-----------|--------|-------|--------|------------|
| all       | 127    | 909       | 0.474  | 0.419 | 0.409  | 0.218      |
| fish      | 127    | 459       | 0.674  | 0.437 | 0.539  | 0.25       |
| jellyfish | 127    | 155       | 0.676  | 0.832 | 0.8    | 0.417      |
| penguin   | 127    | 104       | 0.44   | 0.26  | 0.504  | 0.125      |
| puffin    | 127    | 74        | 0.256  | 0.137 | 0.0705 | 0.0365     |
| shark     | 127    | 57        | 0.316  | 0.298 | 0.256  | 0.15       |
| starfish  | 127    | 27        | 0.445  | 0.481 | 0.397  | 0.233      |
| stingray  | 127    | 33        | 0.507  | 0.515 | 0.498  | 0.313      |

Speed: 1.1ms preprocess, 1.1ms inference, 0.0ms loss, 3.4ms postprocess per image  
Results saved to **runs/detect/train**

YOLO8 : 지정된 aqua data의 yaml파일과 지정된 파라미터 설정으로 학습을 진행

# III.Transfer Learning(YOLO 8)

10 epochs completed in 0.019 hours.

Optimizer stripped from runs/detect/train/weights/last.pt, 22.5MB

Optimizer stripped from runs/detect/train/weights/best.pt, 22.5MB

Validating runs/detect/train/weights/best.pt...

Ultralytics 8.3.107 🚀 Python 3.11.12 torch 2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 72 layers, 11,128,293 parameters, 0 gradients, 28.5 GFLOPs

| Class     | Images | Instances | Box(P | R     | mAP50 | mAP50-95): 100% |  |
|-----------|--------|-----------|-------|-------|-------|-----------------|--|
| all       | 127    | 909       | 0.686 | 0.623 | 0.672 | 0.396           |  |
| fish      | 63     | 459       | 0.741 | 0.667 | 0.72  | 0.383           |  |
| jellyfish | 9      | 155       | 0.789 | 0.897 | 0.915 | 0.501           |  |
| penguin   | 17     | 104       | 0.59  | 0.683 | 0.597 | 0.275           |  |
| puffin    | 15     | 74        | 0.618 | 0.285 | 0.429 | 0.188           |  |
| shark     | 28     | 57        | 0.644 | 0.614 | 0.663 | 0.398           |  |
| starfish  | 17     | 27        | 0.852 | 0.638 | 0.702 | 0.518           |  |
| stingray  | 23     | 33        | 0.57  | 0.576 | 0.68  | 0.512           |  |

Speed: 0.1ms preprocess, 2.2ms inference, 0.0ms loss, 3.7ms postprocess per image

Results saved to **runs/detect/train**

# III. Transfer Learning(YOLO 8)

---

## 1. 모델 요약

층 수: 72층.

파라미터 수: 모델에는 11,128,293개의 파라미터

GFLOPs: 28.5 GFLOPs (Giga Floating Point Operations).

모델의 계산 복잡도를 나타내며, GFLOPs 수가 클수록 더 많은 계산 자원이 필요

## 2. 검증 결과

모델은 127개의 이미지에서 909개의 객체를 검증

평가 지표로는 정밀도(Precision), 재현율(Recall), mAP (mean Average Precision)을 사용

**정밀도(Precision):** 0.686 모델이 탐지한 객체 중 실제로 정확한 객체인 비율

**재현율(Recall):** 0.623 실제 객체 중 모델이 탐지한 객체의 비율

**mAP50** (IoU 0.5 기준 mAP): 0.672

IoU(Intersection over Union)가 0.5 이상일 때의 평균 정밀도

**mAP50-95** (IoU 0.5부터 0.95까지 평균 mAP): 0.396

다양한 IoU 기준(0.5부터 0.95까지)에서의 평균 정밀도.

**이 값이 낮다는 것은 모델의 성능이 일정하지 않다는 의미.**

# III. Transfer Learning(YOLO 8)

---

## Fish (물고기) :

정밀도: 0.741 재현율: 0.667 mAP50: 0.72 mAP50-95: 0.383

물고기 탐지는 좋은 편으로, 높은 정밀도와 재현율

## Jellyfish (해파리)

정밀도: 0.789 재현율: 0.897 mAP50: 0.915 mAP50-95: 0.501

해파리는 매우 잘 탐지되고 있으며, 높은 정밀도와 재현율, 그리고 mAP50-95가 매우 우수하여 모델 성능이 뛰어남.

## Penguin (펭귄)

정밀도: 0.59 재현율: 0.683 mAP50: 0.597 mAP50-95: 0.275

펭귄 탐지는 상대적으로 약한 편

모델의 전반적인 성능은 해파리와 불가사리, 물고기 클래스에서는 상당히 좋은 성능을 보이고 있으며, 특히 해파리의 경우 매우 높은 정확도를 기록  
펭귄과 퍼핀 클래스에서는 성능이 부족한 것으로 나타났으며, 일부 클래스에서는 mAP50-95가 낮아 성능이 일관되지 않다는 특징

# III. Transfer Learning(YOLO 8)

---

## 모델을 개선하기 위한 접근법

- **데이터 부족 클래스**

개선펄 권과 퍼핀 클래스의 성능이 낮은 이유는 데이터가 부족하거나 모델이 학습하기 어려운 특성을 가질 가능성, 데이터셋을 확장하거나 데이터 증강 기법을 활용

- **정밀도와 재현율 균형 맞추기**

일부 클래스에서 정밀도가 높지만 재현율이 낮은 경우, 더 많은 객체를 탐지할 수 있도록 모델을 개선할 수 있습니다.

- **IoU 기준 최적화**

mAP50-95가 낮은 클래스에 대해서는 IoU 기준을 최적화하여 보다 정확한 경계 상자 예측을 할 수 있도록 하여 모델의 탐지 성능을 향상

- **모델 튜닝 및 하이퍼파라미터 조정**

학습률이나 배치 크기 조정을 통해 모델 성능을 개선

# III. Transfer Learning(YOLO 8)

---

```
model.train(data='/content/data.yaml', epochs=50, batch=64, imgsz=640, lr0=0.001, augment=True, patience=10)
```

- epochs=50: 에폭을 50으로 설정하여 더 긴 학습을 진행.
- batch=64: 배치 크기를 64로 늘려서 처리 속도를 향상.
- imgsz=640: 이미지를 640x640 크기로 변경하여 모델이 더 많은 정보를 학습하도록
- lr0=0.001: 학습률을 0.001로 설정하여 모델이 안정적으로 학습.
- augment=True: 데이터 증강을 활성화하여 모델이 다양한 변형된 데이터를 학습.
- patience=10: 성능 개선이 없으면 10 에폭 후에 학습을 종료.

# III. Transfer Learning(YOLO 8)

```
model.train(data='/content/data.yaml', epochs=50, batch=64, imgsz=640, lr0=0.001, augment=True, patience=10)
```

```
Validating runs/detect/train5/weights/best.pt...
```

```
Ultralytics 8.3.107 Python 3.11.12 torch 2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
```

```
Model summary (fused): 72 layers, 11,128,293 parameters, 0 gradients, 28.5 GFLOPs
```

| Class     | Images | Instances | Box(P | R     | mAP50 | mAP50-95): 100% |
|-----------|--------|-----------|-------|-------|-------|-----------------|
| all       | 127    | 909       | 0.827 | 0.712 | 0.785 | 0.504           |
| fish      | 63     | 459       | 0.842 | 0.741 | 0.809 | 0.486           |
| jellyfish | 9      | 155       | 0.892 | 0.923 | 0.962 | 0.584           |
| penguin   | 17     | 104       | 0.717 | 0.658 | 0.746 | 0.352           |
| puffin    | 15     | 74        | 0.711 | 0.5   | 0.597 | 0.32            |
| shark     | 28     | 57        | 0.789 | 0.667 | 0.722 | 0.482           |
| starfish  | 17     | 27        | 0.989 | 0.741 | 0.809 | 0.645           |
| stingray  | 23     | 33        | 0.849 | 0.758 | 0.851 | 0.658           |

```
Speed: 0.2ms preprocess, 12.5ms inference, 0.0ms loss, 1.0ms postprocess per image
```

```
Results saved to runs/detect/train5
```

```
ultralytics.utils.metrics.DetMetrics object with attributes:
```

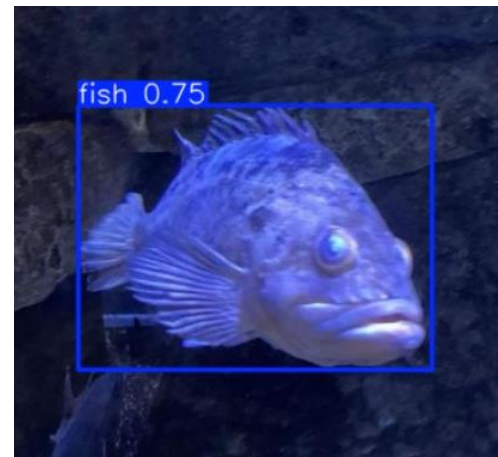
verbose=False, 불필요한 자료의 출력을 방지 !!

# III. Transfer Learning(YOLO 8)

```
[8] 1 results = model.predict(source = '/content/fish3.JPG', save = True)
```

```
image 1/1 /content/fish3.JPG: 416x384 22 jellyfishs, 58.0ms  
Speed: 2.1ms preprocess, 58.0ms inference, 2.2ms postprocess per image at shape (1, 3, 416, 416)  
Results saved to runs/detect/predict
```

YOLO8 : 테스트 데이터를 업로드하여 객체 탐지를 시도(22개의 객체를 탐지)



# III. Transfer Learning

---

YOLO의 객체 인식이 잘 안되는 경우에는 몇 가지 일반적인 상황

- 1.작은 객체:** 모델이 작은 객체의 세부 정보를 충분히 인식하기 어렵게 만들 수 있다.
- 2.멀리 떨어진 객체:** 이미지에 더 많은 노이즈와 함께 희미하게 나타날 수 있다.
- 3.비슷한 객체:** 같은 종류의 과일/동물들이 많이 있는 경우 구분하기 어렵게 될 수 있다.
- 4.부적절한 학습 데이터:** 모델이 실내 이미지로 학습되었지만 테스트 이미지가 야외에서 촬영된 경우, 모델의 성능이 저하될 수 있다.
- 5.모델 크기와 복잡도:** 작은 모델은 작은 객체를 잘 인식하지만, 대신 세부 정보를 잃을 수 있다. 반면 큰 모델은 더 많은 세부 정보를 인식할 수 있지만, 계산 비용이 증가

다양한 환경에서 촬영된 이미지들은 **조명 조건, 색감, 명암 대비 등의 차이**가 있을 수 있으며, 이는 모델의 성능에 영향을 미칠 수 있다.

# III. Transfer Learning

---

객체 인식의 성능을 향상시키기 위해서는

**다양한 조건**에서 학습된 데이터셋을 사용하고,  
**적절한 데이터 증강 기법**을 적용하여 모델의 **일반화 능력을 향상**시키는 것이 중요.

또한 특정 환경에서 모델의 성능을 검증하고 필요한 경우 추가적인 조치를 취하는 것도 중요.

(epoch/patience/model size/learning rate/confidence/training data vs prediction data etc)

# III. Transfer Learning(YOLO 8) summary

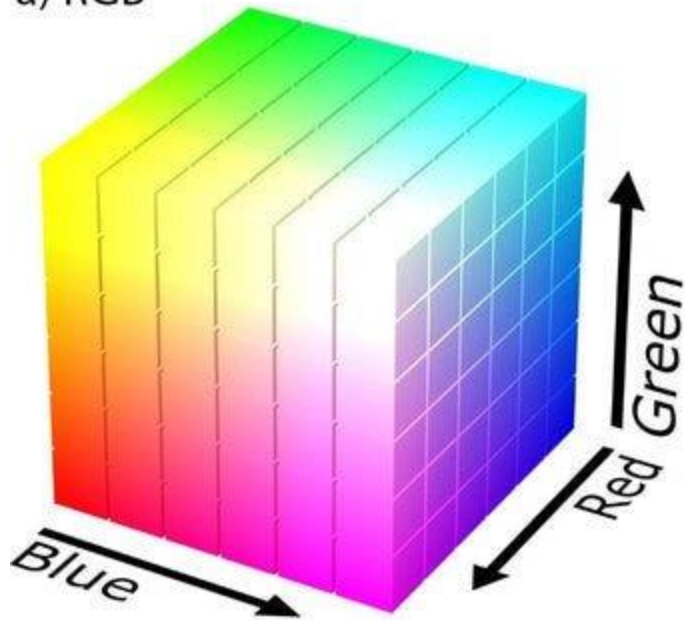
---

- 외부 데이터의 입력(사진/동영상)의 인식률이 떨어졌다.
- YOLOv8n.pt Yolov8s.pt의 정밀도와 정확도가 차이가 났으나 인식결과는 Yolov8s가 조금 더 개선되었다
- epoches가 낮으면 정밀도 정확도의 수치가 낮았다.
  - => 학습된 이미지의 화질/사이즈와 입력된 이미지의 동일성이 유지될 수록 예측결과가 개선되면 과적합이 일부 발생했다.

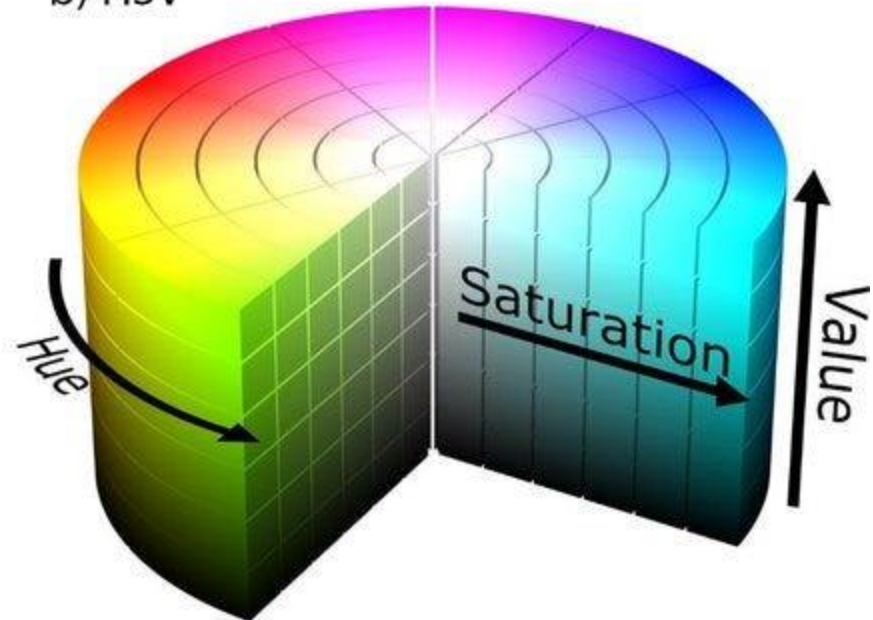
# III. Transfer Learning(YOLO 8)

---

a) RGB

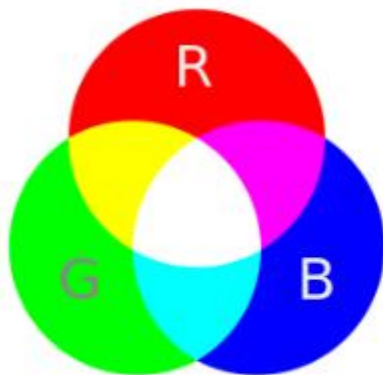


b) HSV

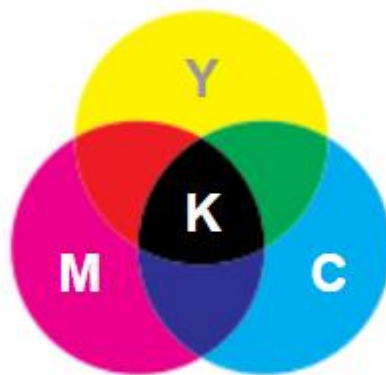


# III. Transfer Learning(YOLO 8)

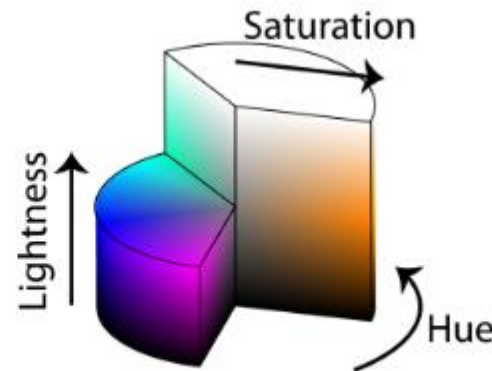
---



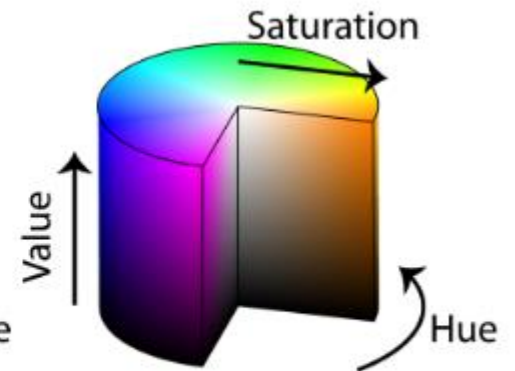
RGB



CMYK



HSL



HSV

# III.Transfer Learning(YOLO 8)

## 실습) 가중치 파일 선택 vs 이미지 사이즈 의 인식 정확도 비교

| 가중치 모델   | 224 pxl | 416 pxl | 640 pxl | mp4 | youtube |
|----------|---------|---------|---------|-----|---------|
| YOLOv8n  |         |         |         |     |         |
| YOLOv8s  |         |         |         |     |         |
| YOLOv8m  |         |         |         |     |         |
| YOLOv8l  |         |         |         |     |         |
| YOLOv8xl |         |         |         |     |         |

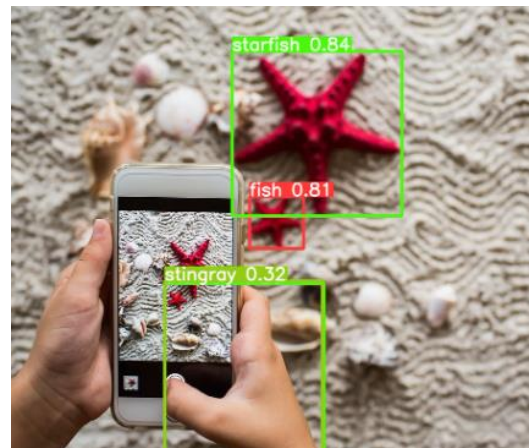
# III. Transfer Learning(YOLO 8)

---

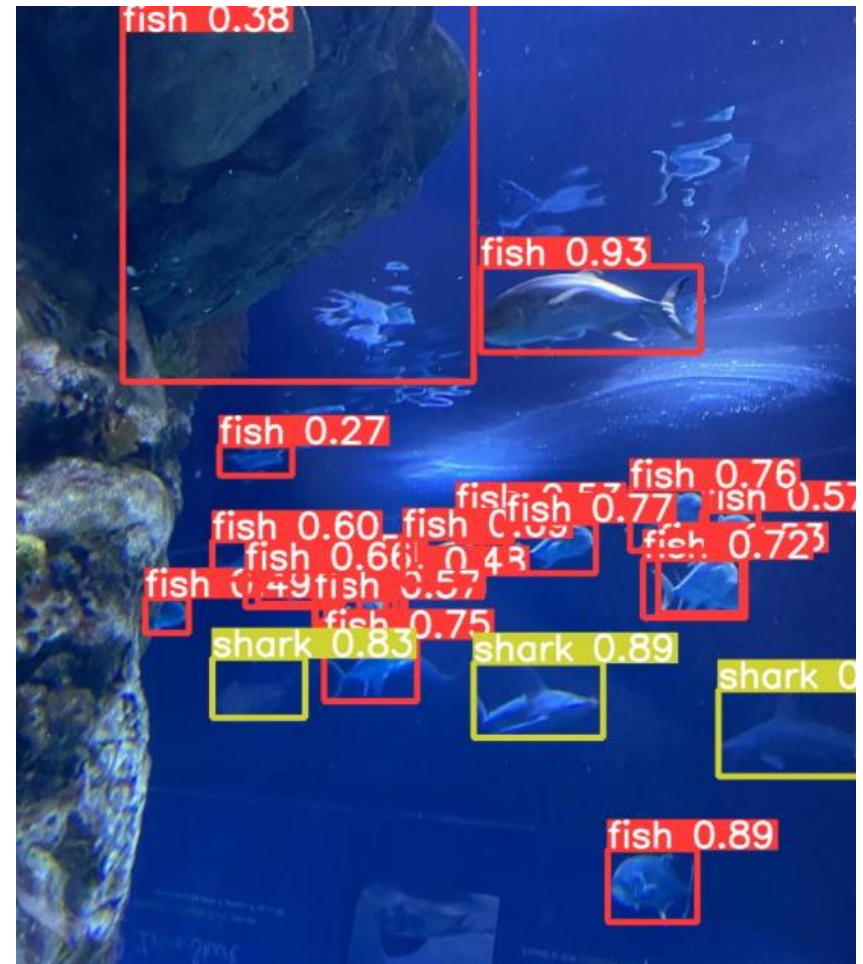
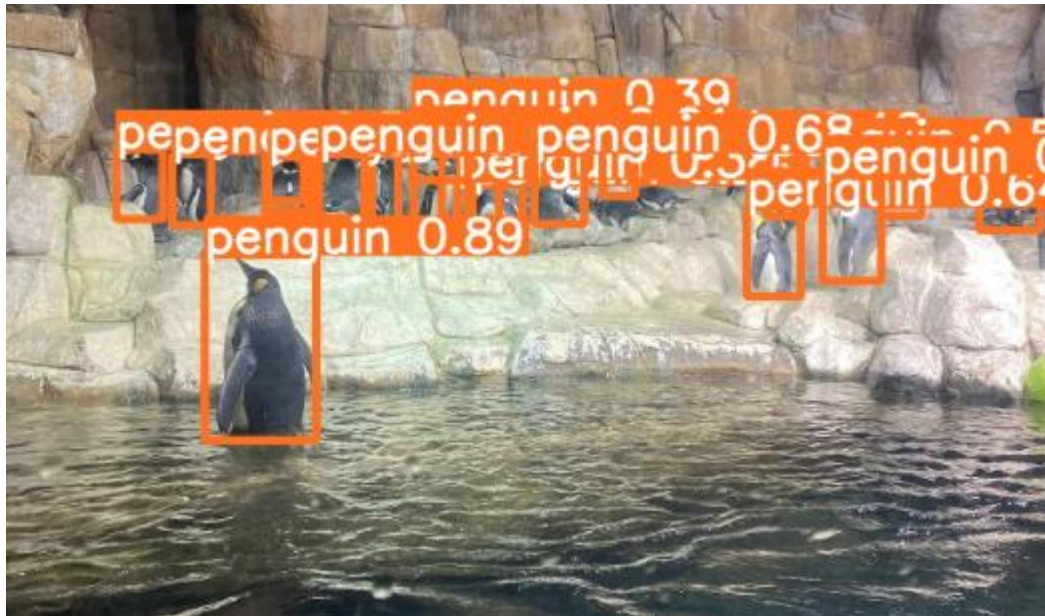
**실습) 1.각자 선택한 CUSTOM DATA로 학습 후 예측을 시도해 봅시다.**

**2. yolov8m.pt 등 다른 모델을 시도해 봅시다**

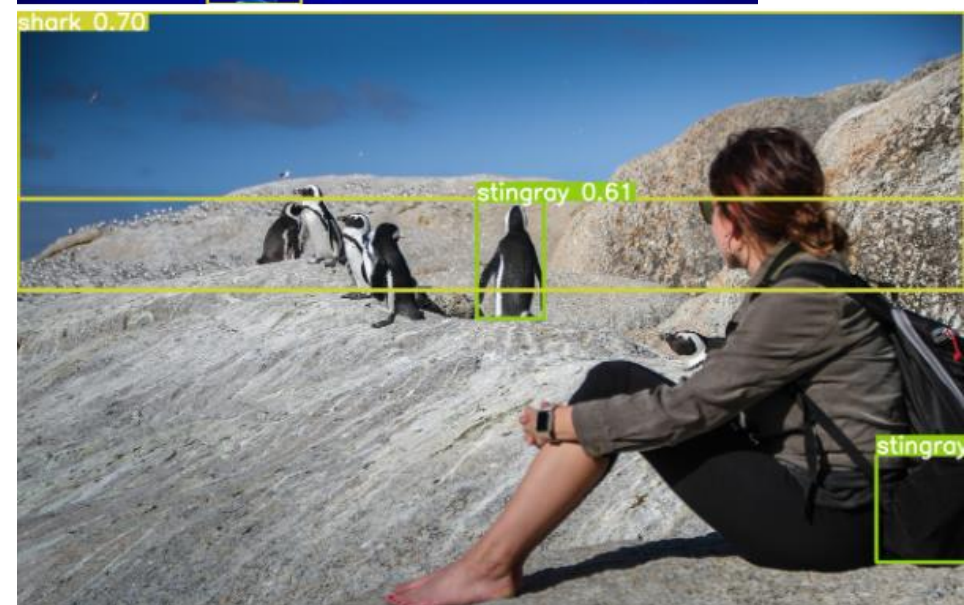
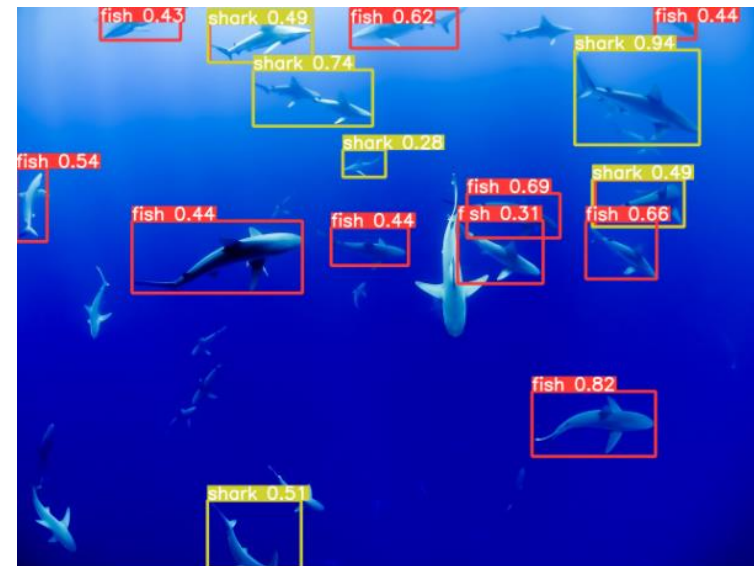
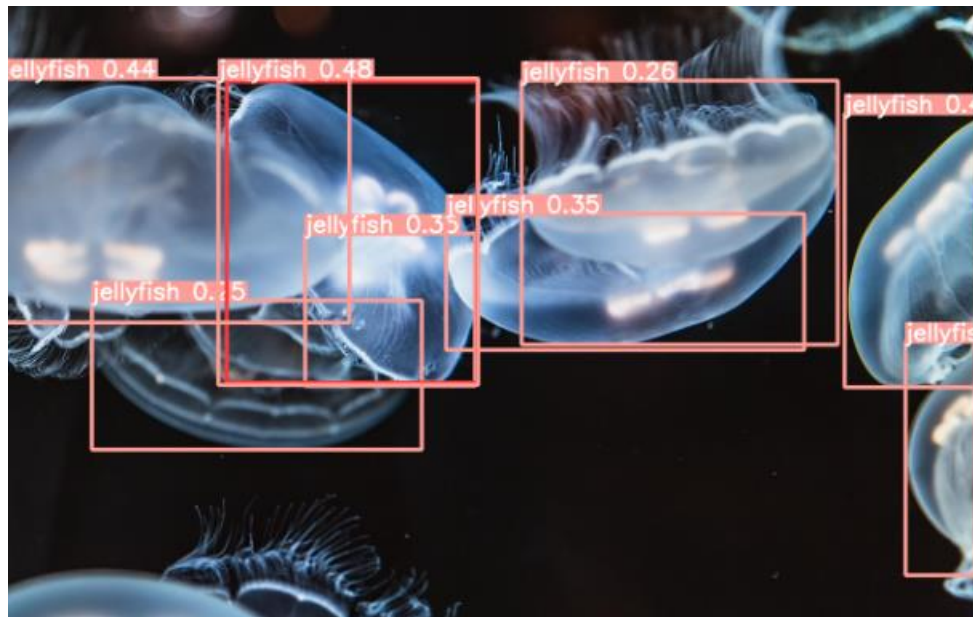
**3. 시도한 내용을 정리하여 비교분석 후 공유해 봅시다.**



# III. Transfer Learning(YOLO 8)

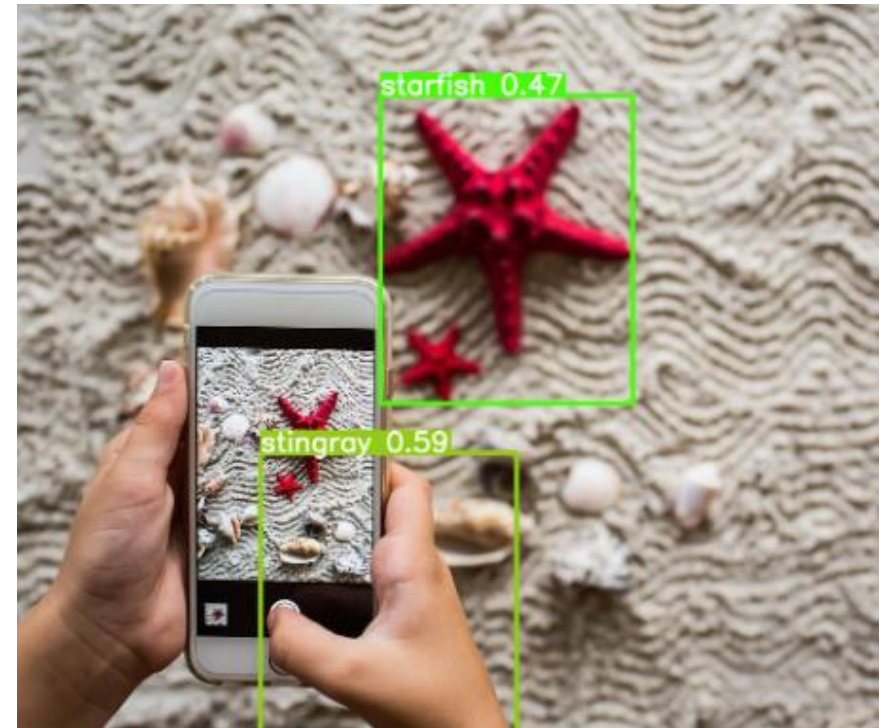


# III. Transfer Learning(YOLO 8)



# III. Transfer Learning(YOLO 8)

---



# **III.Transfer Learning(YOLO 11)**

---

**YOLO 11**

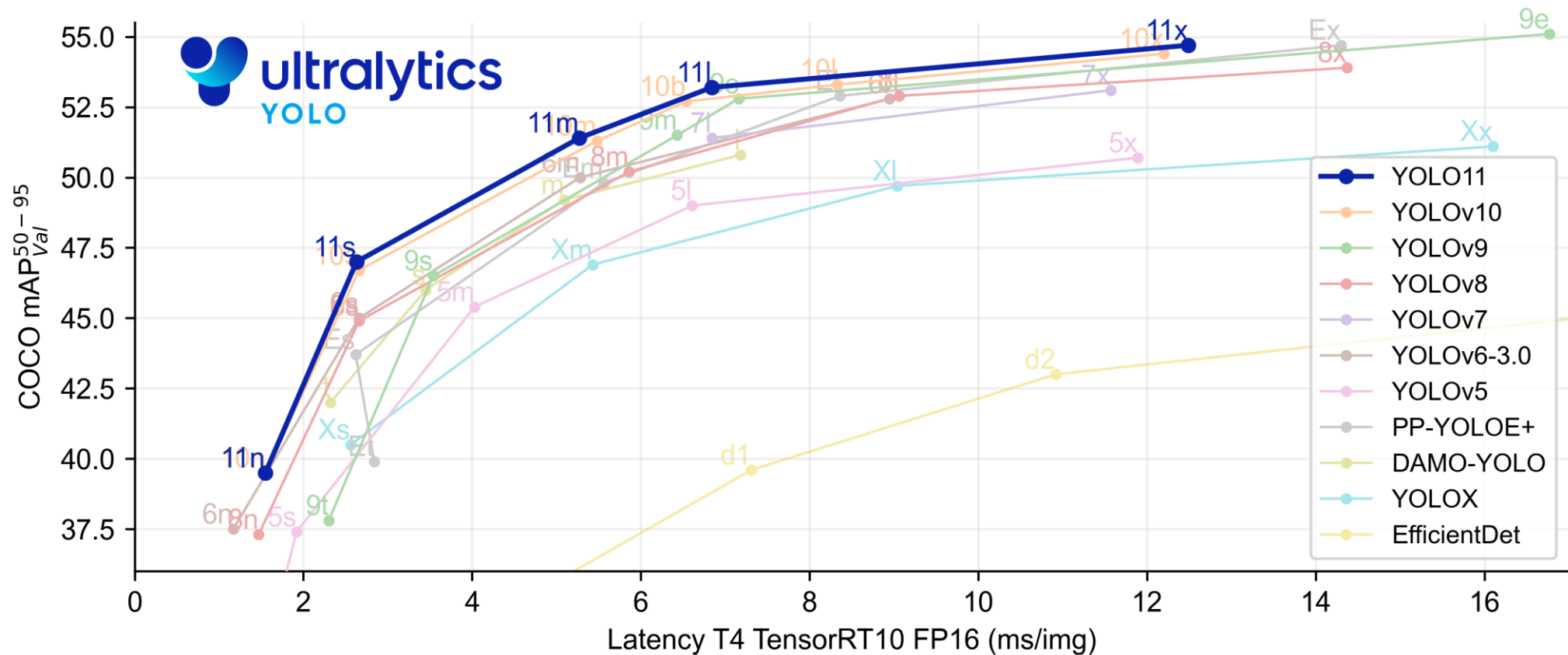
# III. Transfer Learning(YOLO 11)



## ▼ Detection (COCO)

Explore the [Detection Docs](#) for usage examples. These models are trained on the [COCO dataset](#), featuring 80 object classes.

# III. Transfer Learning(YOLO 11)



# III.Transfer Learning(YOLO 11)

| 모델              | 파일 이름                                                                                       | 작업           | 추론 | 유효성 검사 | 교육 | 내보내기 |
|-----------------|---------------------------------------------------------------------------------------------|--------------|----|--------|----|------|
| YOLO11          | yolo11n.pt<br>yolo11s.pt<br>yolo11m.pt<br>yolo11l.pt<br>yolo11x.pt                          | 탐지           | ✓  | ✓      | ✓  | ✓    |
| YOLO11-seg      | yolo11n-seg.pt<br>yolo11s-seg.pt<br>yolo11m-seg.pt<br>yolo11l-seg.pt<br>yolo11x-seg.pt      | 인스턴스 세<br>분화 | ✓  | ✓      | ✓  | ✓    |
| YOLO11-<br>pose | yolo11n-pose.pt<br>yolo11s-pose.pt<br>yolo11m-pose.pt<br>yolo11l-pose.pt<br>yolo11x-pose.pt | 포즈/키폰<br>트   | ✓  | ✓      | ✓  | ✓    |
| YOLO11-obb      | yolo11n-obb.pt<br>yolo11s-obb.pt<br>yolo11m-obb.pt<br>yolo11l-obb.pt<br>yolo11x-obb.pt      | 방향 탐지        | ✓  | ✓      | ✓  | ✓    |
| YOLO11-cls      | yolo11n-cls.pt<br>yolo11s-cls.pt<br>yolo11m-cls.pt<br>yolo11l-cls.pt<br>yolo11x-cls.pt      | 분류           | ✓  | ✓      | ✓  | ✓    |

# III.Transfer Learning(YOLO 11)

---

| Model                   | size<br>(pixels) | mAP <sup>val</sup><br>50-95 | Speed<br>CPU ONNX<br>(ms) | Speed<br>T4 TensorRT10<br>(ms) | params<br>(M) | FLOPs<br>(B) |
|-------------------------|------------------|-----------------------------|---------------------------|--------------------------------|---------------|--------------|
| <a href="#">YOLO11n</a> | 640              | 39.5                        | 56.1 ± 0.8                | 1.5 ± 0.0                      | 2.6           | 6.5          |
| <a href="#">YOLO11s</a> | 640              | 47.0                        | 90.0 ± 1.2                | 2.5 ± 0.0                      | 9.4           | 21.5         |
| <a href="#">YOLO11m</a> | 640              | 51.5                        | 183.2 ± 2.0               | 4.7 ± 0.1                      | 20.1          | 68.0         |
| <a href="#">YOLO11l</a> | 640              | 53.4                        | 238.6 ± 1.4               | 6.2 ± 0.1                      | 25.3          | 86.9         |
| <a href="#">YOLO11x</a> | 640              | 54.7                        | 462.8 ± 6.7               | 11.3 ± 0.2                     | 56.9          | 194.9        |

# III. Transfer Learning(YOLO 11)

 Product ▾ Solutions ▾ Resources ▾ Open Source ▾ Enterprise ▾ Pricing

Search or jump to... 

Sign in 

Sign up

ultralytics / ultralytics Public

 Sponsor

 Notifications

 Fork 7.6k

 Star 39.2k

&lt;&gt; Code Issues 727 Pull requests 251 Discussions Actions Projects 1 Wiki Security Insights

main ▾ 146 Branches 472 Tags

Go to file

<> Code ▾

 4 people

Add new example Ultralytics YOLO-Interactive-Tracking-UI (#19...

  d642347 · 45 minutes ago

 2,445 Commits

|                                                                                               |                                                               |                |
|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------|----------------|
|  .github     | Revert CI runners to macOS 14 to avoid OpenVINO bug on ...    | 11 hours ago   |
|  docker      | ultralytics 8.3.100 New paddlepaddle>=3.0.0 with *.pdipar...  | last week      |
|  docs        | Add standalone export functions for onnx and tensorrt form... | 10 hours ago   |
|  examples    | Add new example Ultralytics YOLO-Interactive-Tracking-UI (... | 45 minutes ago |
|  tests       | Fix Solutions warnings (#20127)                               | 8 hours ago    |
|  ultralytics | Add squeeze() in category_freq before indexing (#20131)       | 1 hour ago     |

About

Ultralytics YOLO11 

 docs.ultralytics.com

python

di

tracking

machine-learning

computer-vision

deep-learning

hub

pytorch

yolo

image-classification

object-detection

pose-estimation

instance-segmentation

ultralytics

rotated-object-detection

yolov8

segment-anything

yolo-world

yolov10

yolo11

# III. Transfer Learning(YOLO 11)

**roboflow**

UniversePublic DatasetsModel ZooBlogDocs

Deploy a Model

Explore these datasets, models, and more on Roboflow Universe. →

350+ MILLION IMAGES500,000+ DATASETS100,000+ PRE-TRAINED MODELS

**DATASET TYPE**

All Datasets61

Object Detection37

Classification11

**Shellfish-OpenImages Dataset**

☐ Object Detection (Bounding Box)

581 images | 3 exports | Last updated 3 years ago

**Oxford Pets Dataset**

☐ Object Detection (Bounding Box)

3680 images | 3 exports | Last updated 3 years ago



**Images**

581

0 missing annotations  
0 null examples

**Annotations**

1,215

2.1 per image (average)  
across 3 classes

**Average Image Size**

0.78 mp

from 0.52 mp  
to 5.04 mp

**Median Image Ratio**

1024x768

wide

**Class Balance**

|         |     |  |                   |
|---------|-----|--|-------------------|
| Shrimp  | 710 |  |                   |
| Lobster | 269 |  | under represented |
| Crab    | 236 |  | under represented |



# III.Transfer Learning(YOLO 11)

roboflow

Universe

Public Datasets

Model Zoo

Blog

Docs

Download

Explore these datasets, models, and more

350+ MILLION IMAGES

500,000+ DATASETS

Dataset Summary

Dataset Analytics

DOWNLOADS

raw581

416x416581

Shellfish-OpenImages Dataset >> raw

Export Created

5 years ago

July 30, 2020

Export Size

581 images

Available Download Formats

COCO JSON

COCO-MMDetection

YOLO Darknet TXT

YOLO v3 Keras TXT

MT-YOLOv6

YOLO v5 PyTorch

YOLOv9

YOLOv11

YOLOv12

YOLO v4 PyTorch

Scaled-YOLOv4

YOLO v7 PyTorch

YOLOv8

YOLOv8-ORB

YOLOv5-ORB

YOLOv8-ORB

Tensorflow Object Detection CSV

RetinaNet Keras CSV

Multiclass Classification

OpenAI CLIP Classification

Tensorflow TFRecord

TXT annotations and YAML config used with YOLOv12

# III. Transfer Learning(YOLO 11)

The screenshot shows the Roboflow website interface. At the top, the Roboflow logo is on the left, and navigation links for Universe, Public Datasets, Model Zoo, Blog, and Docs are on the right. A teal banner below the navigation bar says "Explore these datasets, models, and more on Roboflow Universe. →" with statistics: 350+ MILLION IMAGES, 500,000+ DATASETS, and 100,000+ PRE-TRAINED MODELS.

The main content area is titled "Shellfish-OpenImages Dataset >> raw". On the left, there's a sidebar with "Dataset Summary", "Dataset Analytics", and "DOWNLOADS". Under "DOWNLOADS", "raw" is selected with a count of 581, and "416x416" is also listed with a count of 581.

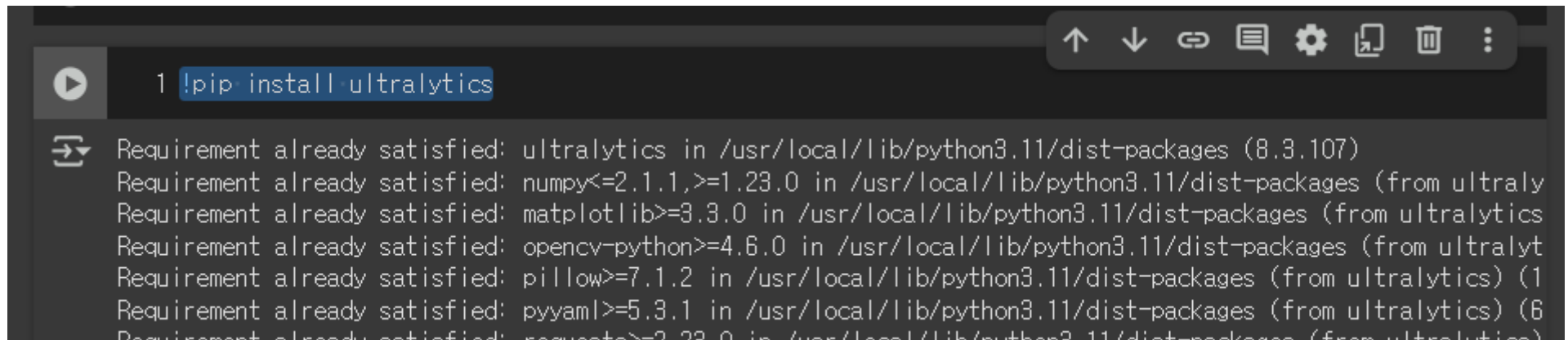
The main content area shows "Export Created 5 years ago July 30, 2020" and "Available Download Formats". A modal window titled "Download" is open in the center. It has three tabs: "Jupyter", "Terminal" (which is active), and "Raw URL". The "Terminal" tab contains the instruction: "Use this code to download and unzip your dataset via the command line on any \*nix machine:" followed by a code block with the command: `curl -L "https://public.roboflow.com/ds/PezCa3JM81?key=Xnu3xZPoHt" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip`. At the bottom of the modal, there are "Done" and "Choose a Model" buttons.

On the right side of the main content area, there's a section for "Annotations shellfish" with various format buttons like "COCO JSONL", "Pascal VOC XML", "YOLOv4", "YOLOv5-OB", "YOLOv8", "YOLOv8-OB", "Tensorflow Object Detection CSV", "RetinaNet Keras CSV", "Multiclass Classification", "OpenAI CLIP Classification", and "Tensorflow TFRecord".

# III. Transfer Learning(YOLO 11)

---

```
!pip install ultralytics
```



```
1 !pip install ultralytics
```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.11/dist-packages (8.3.107)  
Requirement already satisfied: numpy<=2.1.1,>=1.23.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.0.2)  
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (3.7.1)  
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (4.10.0.84)  
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (10.4.0)  
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (6.0.2)  
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.11/dist-packages (from ultralytics) (2.32.0)

# III. Transfer Learning(YOLO 11)

---

```
from ultralytics import YOLO
model = YOLO('yolo11n.pt')

print(type(model.names), len(model.names))
print(model.names)
```

```
[28] 1 from ultralytics import YOLO
      2 model = YOLO('yolo11n.pt')
      3
      4 print(type(model.names), len(model.names))
      5 print(model.names)
```

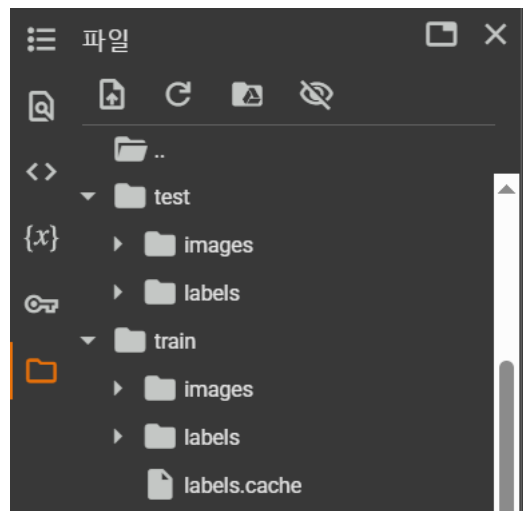
```
↩ <class 'dict'> 80
  {0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8:
```

# III. Transfer Learning(YOLO 11)

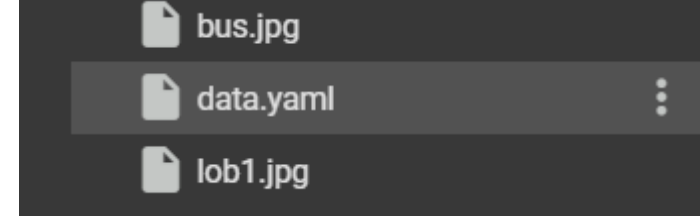
```
!curl -L "https://public.roboflow.com/ds/PezCa3JM81?key=Xnu3xZPoHt" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

```
1 !curl -L "https://public.roboflow.com/ds/PezCa3JM81?key=Xnu3xZPoHt" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

```
extracting: train/images/ccd6ed827632e845_jpg.rf.149de769969e70cb713e9b1881d604ba.jpg
extracting: train/images/cec1f95192614824_jpg.rf.49973cb5e84906575dfb5a3ebc8e4239.jpg
extracting: train/images/cf4899ff91bc7e52_jpg.rf.2215356223a31fd3cf96d8ec5c7e6b03.jpg
extracting: train/images/cfd8ba188d53a049_jpg.rf.7e2bcd53883b9b72efdecc46881895a0.jpg
extracting: train/images/d20b3661a1af49cd_jpg.rf.608081debf24590d376fe1afb2b057d5.jpg
extracting: train/images/d23846015c65244e_jpg.rf.1e1d53f25ef69e8244eb1d306db6f487.jpg
```



# III. Transfer Learning(YOLO 11)



```
!cat data.yaml
```

```
1 !cat data.yaml
```

```
train: ../train/images
val: ../valid/images
test: ../test/images
```

```
nc: 3
names: ['Crab', 'Lobster', 'Shrimp']
```

```
roboflow:
  workspace: jacob-solawetz
  project: shellfish-openimages
  version: 3
  license: CC BY 4.0
  url: https://universe.roboflow.com/jacob-solawetz/shellfish-openimages/dataset/3
```

# III. Transfer Learning(YOLO 11)

```
model.train(data='/content/data.yaml', epochs=3, batch=32, imgsz=416)
```

1 model.train(data='/content/data.yaml', epochs=3, batch=32, imgsz=416)

| Epoch | GPU_mem | box_loss | cls_loss  | dfl_loss | Instances | Size                                |
|-------|---------|----------|-----------|----------|-----------|-------------------------------------|
| 1/3   | OG      | 1.251    | 3.319     | 1.413    | 114       | 416: 100% ██████████  13/13 [02:... |
|       | Class   | Images   | Instances | Box(P    | R         | mAP50 mAP50-95): 100% ██████████    |
| Epoch | GPU_mem | box_loss | cls_loss  | dfl_loss | Instances | Size                                |
| 2/3   | OG      | 1.276    | 3.014     | 1.403    | 88        | 416: 100% ██████████  13/13 [02:... |
|       | Class   | Images   | Instances | Box(P    | R         | mAP50 mAP50-95): 100% ██████████    |
| Epoch | GPU_mem | box_loss | cls_loss  | dfl_loss | Instances | Size                                |
| 3/3   | OG      | 1.229    | 2.567     | 1.381    | 82        | 416: 100% ██████████  13/13 [02:... |
|       | Class   | Images   | Instances | Box(P    | R         | mAP50 mAP50-95): 100% ██████████    |

3 epochs completed in 0.134 hours.

Optimizer stripped from runs/detect/train3/weights/last.pt, 5.4MB

Optimizer stripped from runs/detect/train3/weights/best.pt, 5.4MB

Validating runs/detect/train3/weights/best.pt...

Ultralytics 8.3.107 Python-3.11.12 torch-2.6.0+cu124 CPU (Intel Xeon 2.20GHz)

YOLO11n summary (fused): 100 layers, 2,582,737 parameters, 0 gradients, 6.3 GFLOPs

파일

..

datasets

runs

classify

detect

predict

predict2

train

train2

train3

weights

best.pt

last.pt

F1\_curve.png

PR\_curve.png

P\_curve.png

R\_curve.png

# III. Transfer Learning(YOLO 11)

---

```
1 results = model.predict(source='/content/lob1.jpg')
```

```
image 1/1 /content/lob1.jpg: 288x416 (no detections), 105.8ms  
Speed: 3.8ms preprocess, 105.8ms inference, 1.0ms postprocess per image at shape (1, 3, 288, 416)
```

# III. Transfer Learning(YOLO 11)

---

**실습) YOLO 11을 사용해 보고 버전별/모델별 성능평가를 공유해 봅시다**

1. **YOLOv3**: The third iteration of the YOLO model family, originally by Joseph Redmon, known for its efficient real-time object detection capabilities.
2. **YOLOv4**: A darknet-native update to YOLOv3, released by Alexey Bochkovskiy in 2020.
3. **YOLOv5**: An improved version of the YOLO architecture by Ultralytics, offering better performance and speed trade-offs compared to previous versions.
4. **YOLOv6**: Released by [Meituan](#) in 2022, and in use in many of the company's autonomous delivery robots.
5. **YOLOv7**: Updated YOLO models released in 2022 by the authors of YOLOv4.
6. **YOLOv8**: A versatile model featuring enhanced capabilities such as [instance segmentation](#), pose/keypoints estimation, and classification.
7. **YOLOv9**: An experimental model trained on the Ultralytics [YOLOv5](#) codebase implementing Programmable Gradient Information (PGI).
8. **YOLOv10**: By Tsinghua University, featuring NMS-free training and efficiency-accuracy driven architecture, delivering state-of-the-art performance and latency.
9. **YOLO11** 🚀 **NEW**: Ultralytics' latest YOLO models delivering state-of-the-art (SOTA) performance across multiple tasks including detection, segmentation, pose estimation, tracking, and classification.

# III.Transfer Learning(YOLO 12)

## YOLO12: Attention-Centric Object Detection

### 개요

2025년 초에 출시된 YOLO12는 이전 YOLO 모델에서 사용된 전통적인 CNN 기반 접근 방식에서 벗어나면서도 많은 애플리케이션에 필수적인 실시간 추론 속도를 유지하는 어텐션 중심 아키텍처를 도입합니다. 이 모델은 어텐션 메커니즘 및 전체 네트워크 아키텍처의 새로운 방법론적 혁신을 통해 높은 객체 detect 정확도를 달성하면서 실시간 성능을 유지합니다. 이러한 장점에도 불구하고 YOLO12는 무거운 어텐션 블록으로 인해 학습 불안정성, 높은 메모리 소비, 느린 CPU 처리량을 보일 수 있는 커뮤니티 주도 릴리스이므로, Ultralytics는 대부분의 프로덕션 워크로드에 [YOLO11](#) 또는 [YOLO26](#)을 권장합니다.

#### 커뮤니티 모델

YOLO12는 주로 벤치마킹 및 연구 목적으로 유지됩니다. 안정적인 학습, 예측 가능한 메모리 사용량 및 최적화된 CPU 추론이 필요한 경우 배포를 위해 [YOLO11](#) 또는 [YOLO26](#)을 선택하십시오.

# III.Transfer Learning(YOLO 11)

---

## Custom Data Set 직접 만들기

# **III.Transfer Learning(YOLO 11)**

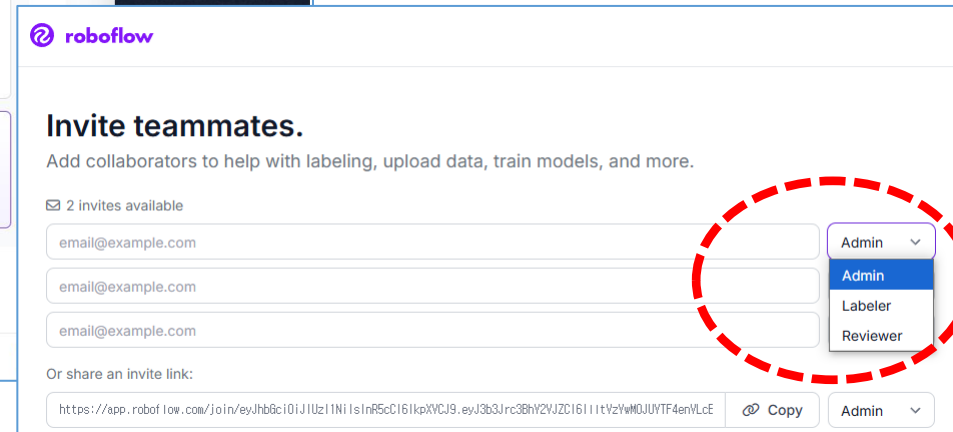
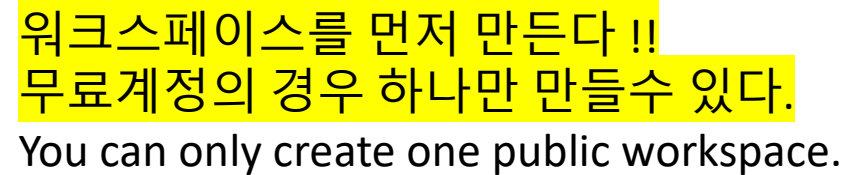
---

**Roboflow with Custom data set**

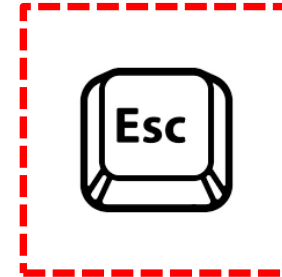
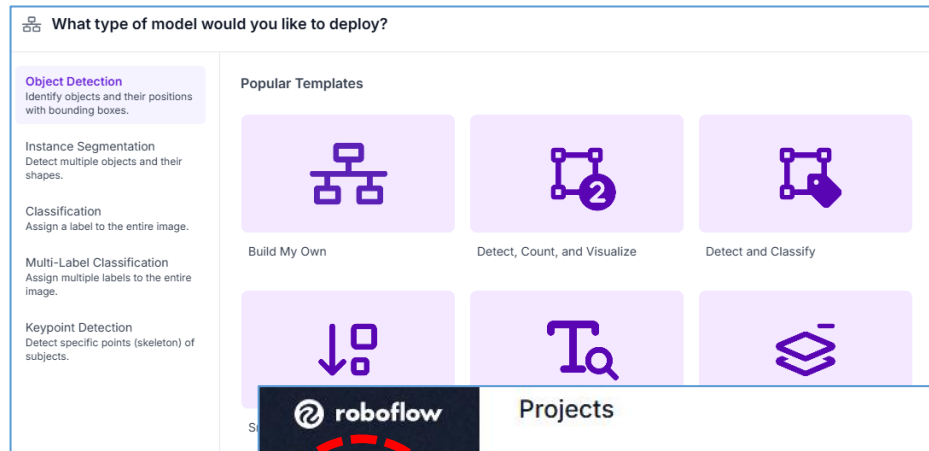
# III. Transfer Learning(YOLO 11)

---

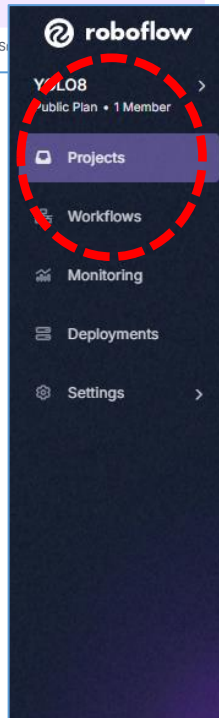
|                   |                    |                  |
|-------------------|--------------------|------------------|
| 0: person         | 30: skis           | 60: dining table |
| 1: bicycle        | 31: snowboard      | 61: toilet       |
| 2: car            | 32: sports ball    | 62: tv           |
| 3: motorcycle     | 33: kite           | 63: laptop       |
| 4: airplane       | 34: baseball bat   | 64: mouse        |
| 5: bus            | 35: baseball glove | 65: remote       |
| 6: train          | 36: skateboard     | 66: keyboard     |
| 7: truck          | 37: surfboard      | 67: cell phone   |
| 8: boat           | 38: tennis racket  | 68: microwave    |
| 9: traffic light  | 39: bottle         | 69: oven         |
| 10: fire hydrant  | 40: wine glass     | 70: toaster      |
| 11: stop sign     | 41: cup            | 71: sink         |
| 12: parking meter | 42: fork           | 72: refrigerator |
| 13: bench         | 43: knife          | 73: book         |
| 14: bird          | 44: spoon          | 74: clock        |
| 15: cat           | 45: bowl           | 75: vase         |
| 16: dog           | 46: banana         | 76: scissors     |
| 17: horse         | 47: apple          | 77: teddy bear   |
| 18: sheep         | 48: sandwich       | 78: hair drier   |
| 19: cow           | 49: orange         | 79: toothbrush   |
| 20: elephant      | 50: broccoli       |                  |
| 21: bear          | 51: carrot         |                  |
| 22: zebra         | 52: hot dog        |                  |
| 23: giraffe       | 53: pizza          |                  |
| 24: backpack      | 54: donut          |                  |
| 25: umbrella      | 55: cake           |                  |
| 26: handbag       | 56: chair          |                  |
| 27: tie           | 57: couch          |                  |
| 28: suitcase      | 58: potted plant   |                  |
| 29: frisbee       | 59: bed            |                  |



# III. Transfer Learning(YOLO 11)



프로젝트를 만든다 !!



Projects



There are no projects in this workspace.

Create a project and upload images to start labeling, training, and deploying your computer vision model.

+ New Project

View a Tutorial

# III. Transfer Learning(YOLO 11)

roboflow

Let's create your project.

ROBO2025 > Public MNIST\_H2

Project Name: MNIST\_H2

Annotation Group: numbers

Visibility: ☐ Private ☒ Public

Licenses: Public Domain

Tool: Traditional Rapid

Project Type: Identify objects and their positions with bounding boxes.

Let's create your project.

ROBO2025 > Public MNISTest

Project Name: MNISTest

Annotation Group: numbers

License: Public Domain

Project Type: **Object Detection** Bounding Boxes Counts Tracking

Identify objects and their positions with bounding boxes.

Classification: Image Labels Filtering Content Moderation

Assign labels to the entire image. ☐ Single-Label ☐ Multi-Label

Instance Segmentation: Polygons Measuring Odd Shapes

Detect multiple objects and their actual shape.

Keypoint Detection: Skeleton Structure Pose Estimation

Identify keypoints ("skeletons") on subjects.

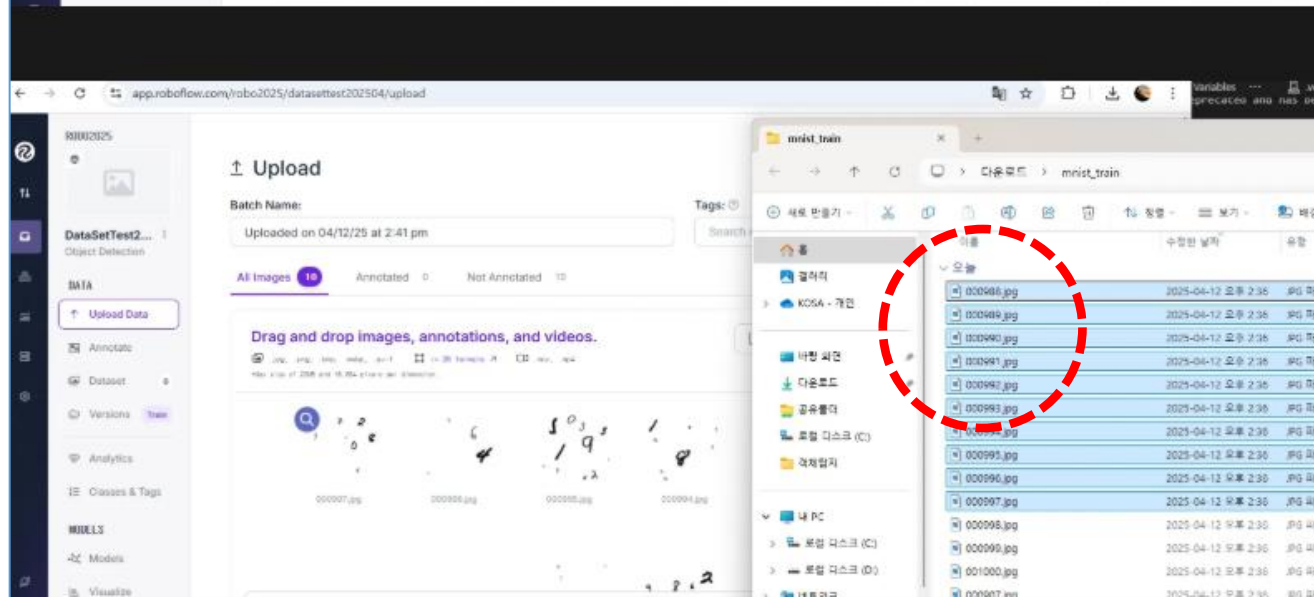
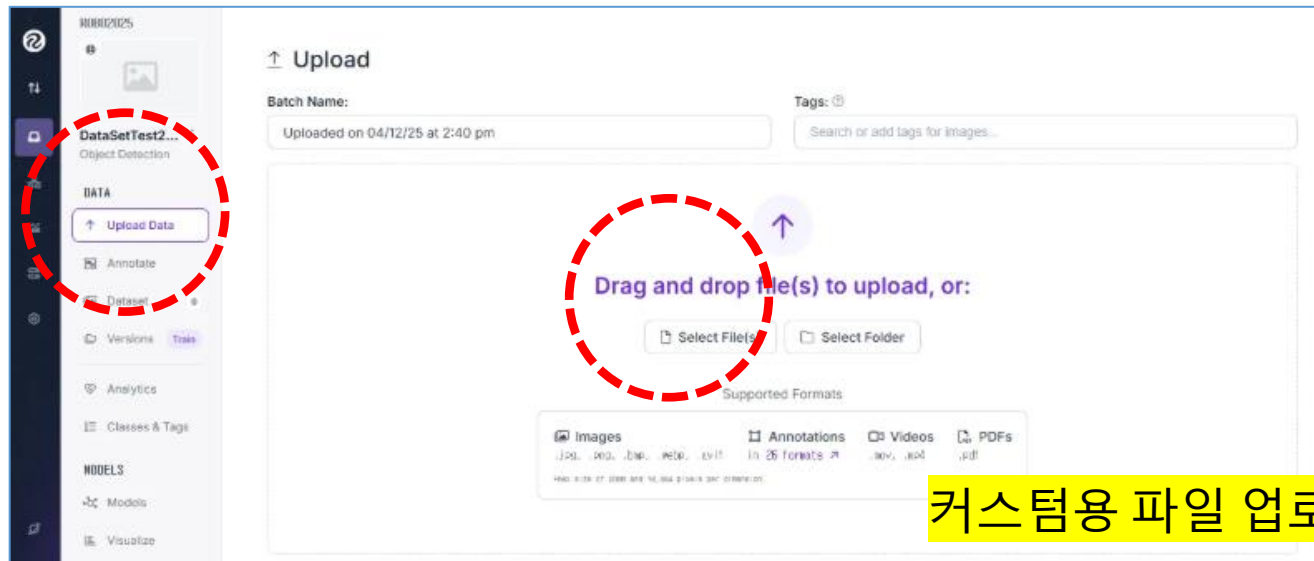
Multimodal: Prompts Visual Question Answering Captions

Describe images using text pairs.

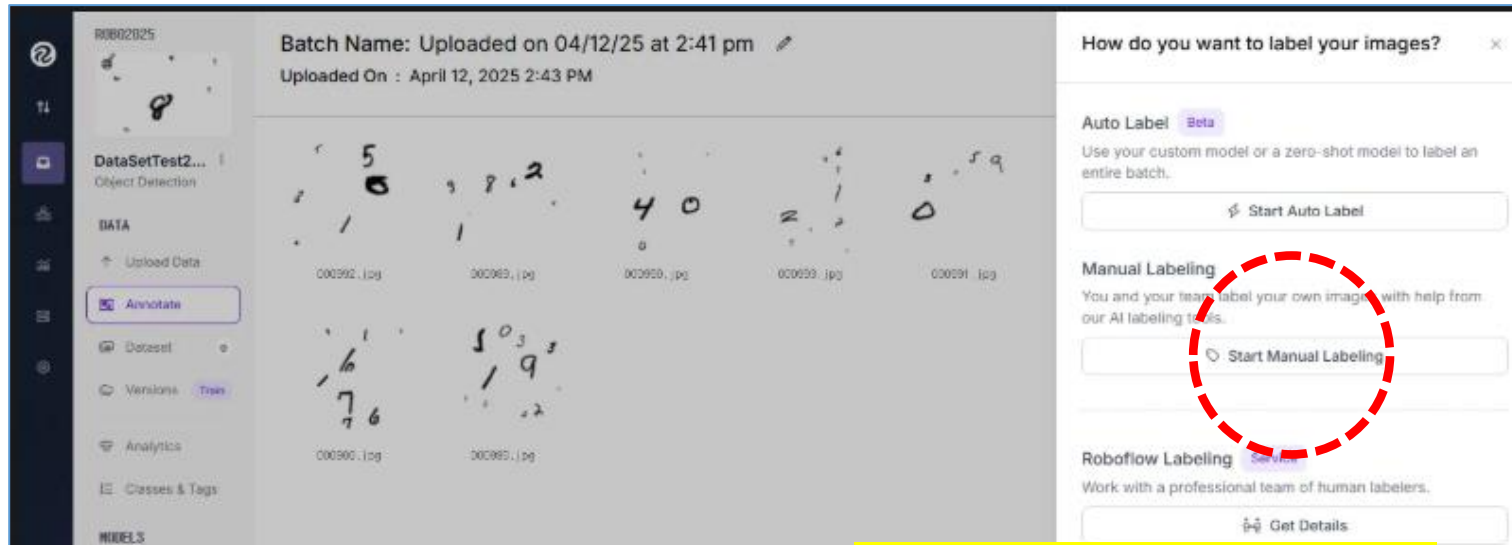
Cancel Create Public Project

퍼블릭으로 생성!! > 객체 탐지

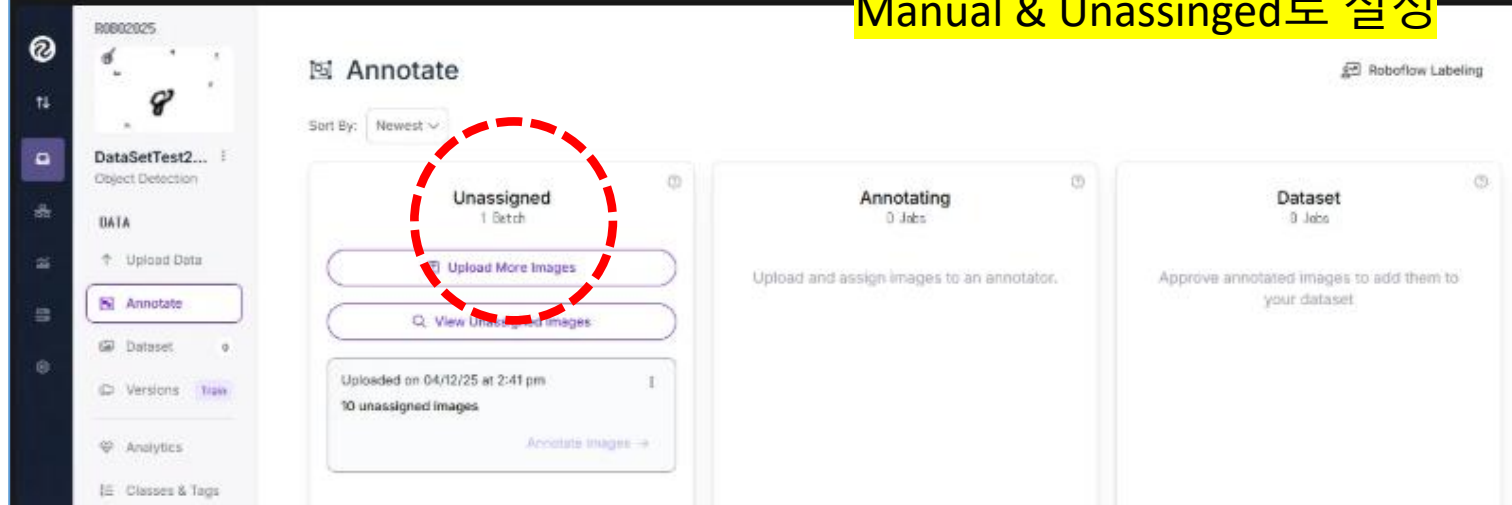
# III. Transfer Learning(YOLO 11)



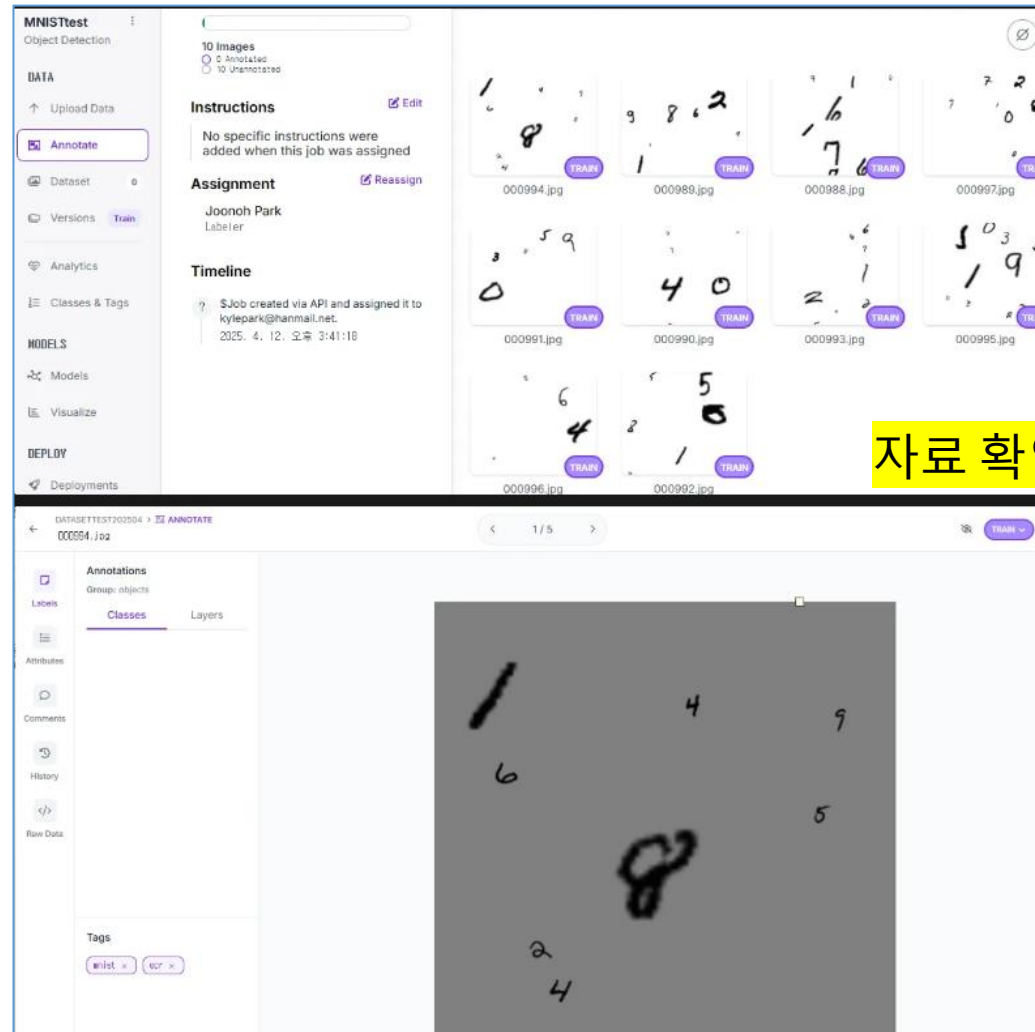
# III. Transfer Learning(YOLO 11)



Manual & Unassigned로 설정



# III. Transfer Learning(YOLO 11)



# III. Transfer Learning(YOLO 11)

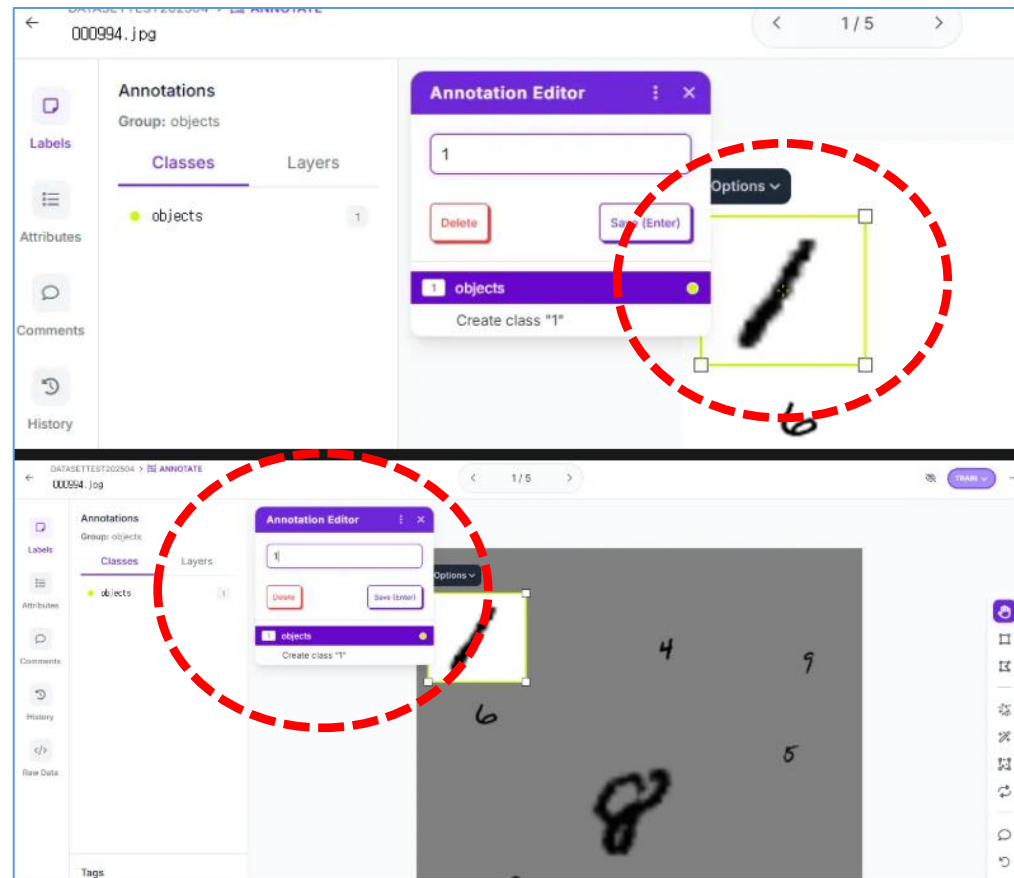
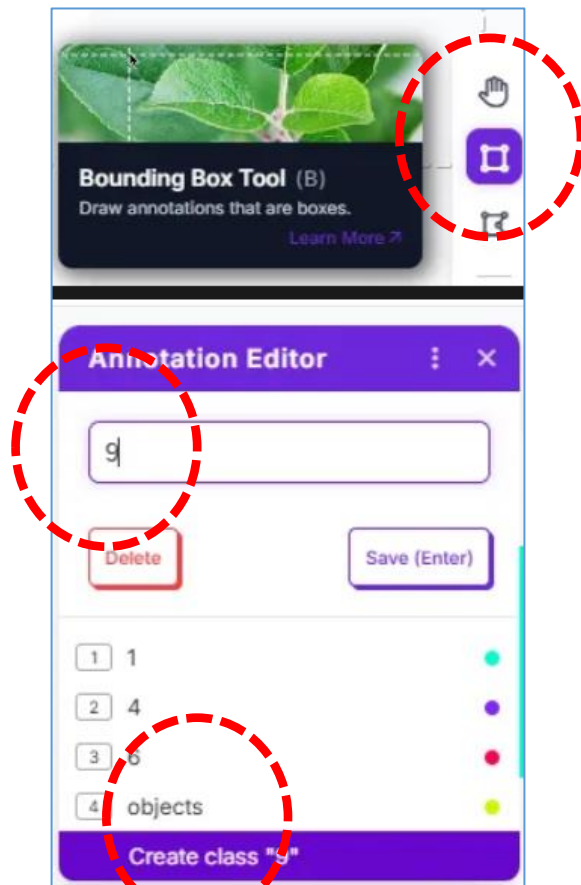
---

Here are some tips to help you build a good model:

-  Label every relevant object in every image, even if it's partially visible
-  Create tight bounding boxes, but don't cut off part of the object.
-  Your training data should be representative of what your model will see when deployed
-  Train your first model after ~70 images are labeled, so you can use Roboflow's ML-powered labeling features.

# III. Transfer Learning(YOLO 11)

사각형 선택 후 클래스(라벨)을  
선택하고 클래스별로 입력!!



# III. Transfer Learning(YOLO 11)

5 / 12

**Add Images To Dataset**

Labeled Images to Add: 12  
Remaining Batch Images: 0

Your remaining images will be split into a new batch.

Method: Use Existing Values

Distribution: Images with an existing split (12)  
Train: 8 images  
Valid: 2 images  
Test: 2 images

Submit for Review

Add 12 Images

**Add Images To Dataset**

Labeled Images to Add: 12  
Remaining Batch Images: 0

Your remaining images will be split into a new batch.

Method: Use Existing Values

What's Train, Valid, Test?

Split Images Between Train/Valid/Test

Add All Images to Training Set

Add All Images to Validation Set

Add All Images to Testing Set

# III. Transfer Learning(YOLO 11)

클래스별 오류 확인 후 추가 또는 train/test 분리!!

The screenshot shows the Roboflow interface. The 'Classes & Tags' section is highlighted with a red dashed circle. Below it, a table lists classes and their counts:

| COLOR | CLASS NAME | COUNT |
|-------|------------|-------|
|       | 0          | 3     |
|       | 1          | 5     |
|       | 2          | 4     |
|       | 3          | 3     |
|       | 4          | 4     |
|       | 5          | 2     |
|       | 6          | 3     |
|       | 7          | 3     |
|       | 8          | 4     |

Below the table, a button 'Add 5 images to Dataset' is highlighted with a red dashed circle. At the bottom, a grid of image thumbnails is shown, with four labeled 'TRAIN' and one labeled 'VALID', all highlighted with a red dashed circle.

The screenshot shows the 'Method' section of the Roboflow interface. The 'Split Images Between Train/Valid/Test' option is highlighted with a red dashed circle. Below it, a bar chart shows the image distribution:

Train 70% Valid 20% Test 10%

The 'Valid 20%' and 'Test 10%' labels are highlighted with a red dashed circle. Below the bar chart, the 'Image Distribution' section shows:

Train: 4 images  
Valid: 1 images  
Test: 0 images

# III. Transfer Learning(YOLO 11)

Versions : 데이터 전처리와 증강 선택(2개 무료)  
우상단의 create New Version을 선택하면 증강처리를 할수 있는 메뉴가 생성된다  
필요없는 경우 Download Dataset !!

← ROB02025

View on Universe

MNIST\_H2  
Object Detection

DATA

↑ Upload Data

Annotate

Dataset 2

**Versions** Train

Analytics

Classes & Tags

MODELS

Models

Visualize

Versions

Roboflow Instant 1 [Eval]

v1 2 Joonoh Park

v1 Roboflow Instant 1 [Eval]

Generated on Oct 21, 2025 by Joonoh Park

+ Create New Version

Download Dataset Edit

**This version doesn't have a model.**

Train an optimized, state of the art model with Roboflow or upload a custom trained model to use features like Label Assist and Model Evaluation and deployment options like our auto-scaling API and edge device support.

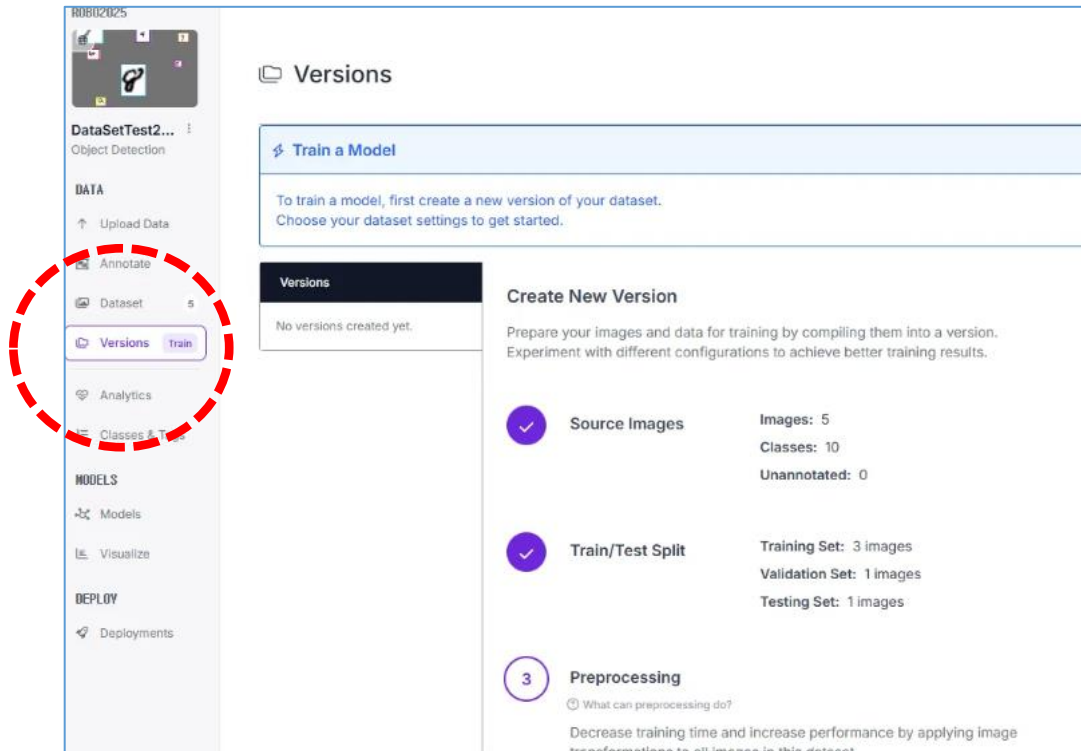
Custom Train

Uses Roboflow Credits View usage

How to Upload Custom Weights

# III. Transfer Learning(YOLO 11)

Versions : 데이터 전처리와 증강 선택(2개 무료)



**Versions**

[Train a Model](#)

To train a model, first create a new version of your dataset.  
Choose your dataset settings to get started.

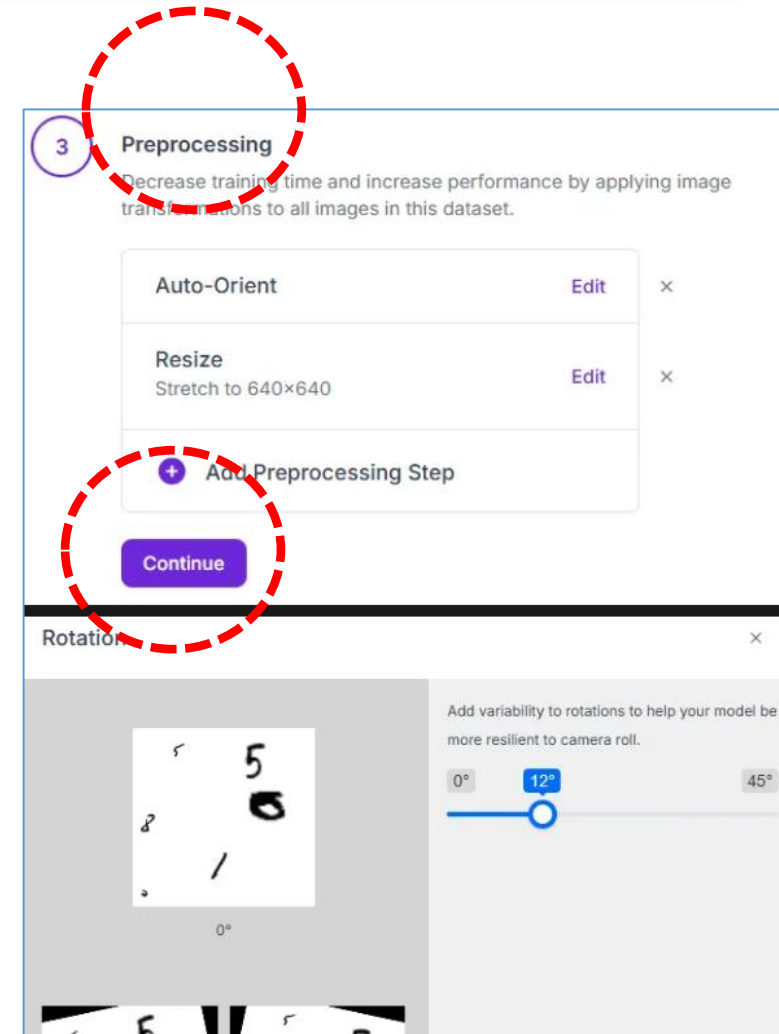
**Versions**

No versions created yet.

**Create New Version**

Prepare your images and data for training by compiling them into a version.  
Experiment with different configurations to achieve better training results.

- ✓ **Source Images**  
Images: 5  
Classes: 10  
Unannotated: 0
- ✓ **Train/Test Split**  
Training Set: 3 images  
Validation Set: 1 images  
Testing Set: 1 images
- 3 **Preprocessing**  
What can preprocessing do?  
Decrease training time and increase performance by applying image transformations to all images in this dataset.



3 **Preprocessing**

Decrease training time and increase performance by applying image transformations to all images in this dataset.

- Auto-Orient [Edit](#) ×
- Resize  
Stretch to 640×640 [Edit](#) ×

[+ Add Preprocessing Step](#)

[Continue](#)

**Rotation** ×

Add variability to rotations to help your model be more resilient to camera roll.

0° 12° 45°

0°

# III. Transfer Learning(YOLO 11)

Version : 전처리와 증강작업 완료 후 코드 출력 > 다운로드 !!  
증강처리가 완료되면 오른쪽과 같이 새로운 버전을 다운로드 가능

### Create New Version

Prepare your images and data for training by compiling them into a version.  
Experiment with different configurations to achieve better training results.

✓

Source Images

Images: 3  
Classes: 5  
Unannotated: 0

✓

Train/Test Split

Training Set: 2 images  
Validation Set: images  
Testing Set: 1 images

✓

Preprocessing

Auto-Orient: Applied  
Resize: Stretch to 640×640

✓

Augmentation

Rotation: Between -15° and +15°

5

Create

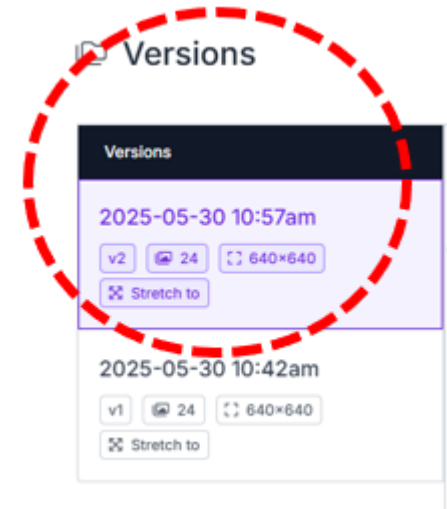
Review your selections and select a version size to create a moment-in-time snapshot of your dataset with the applied transformations.

Larger versions take longer to train but often result in better model performance. [See how this is calculated](#)

Maximum Version Size

7 images (3x) ▾

Create



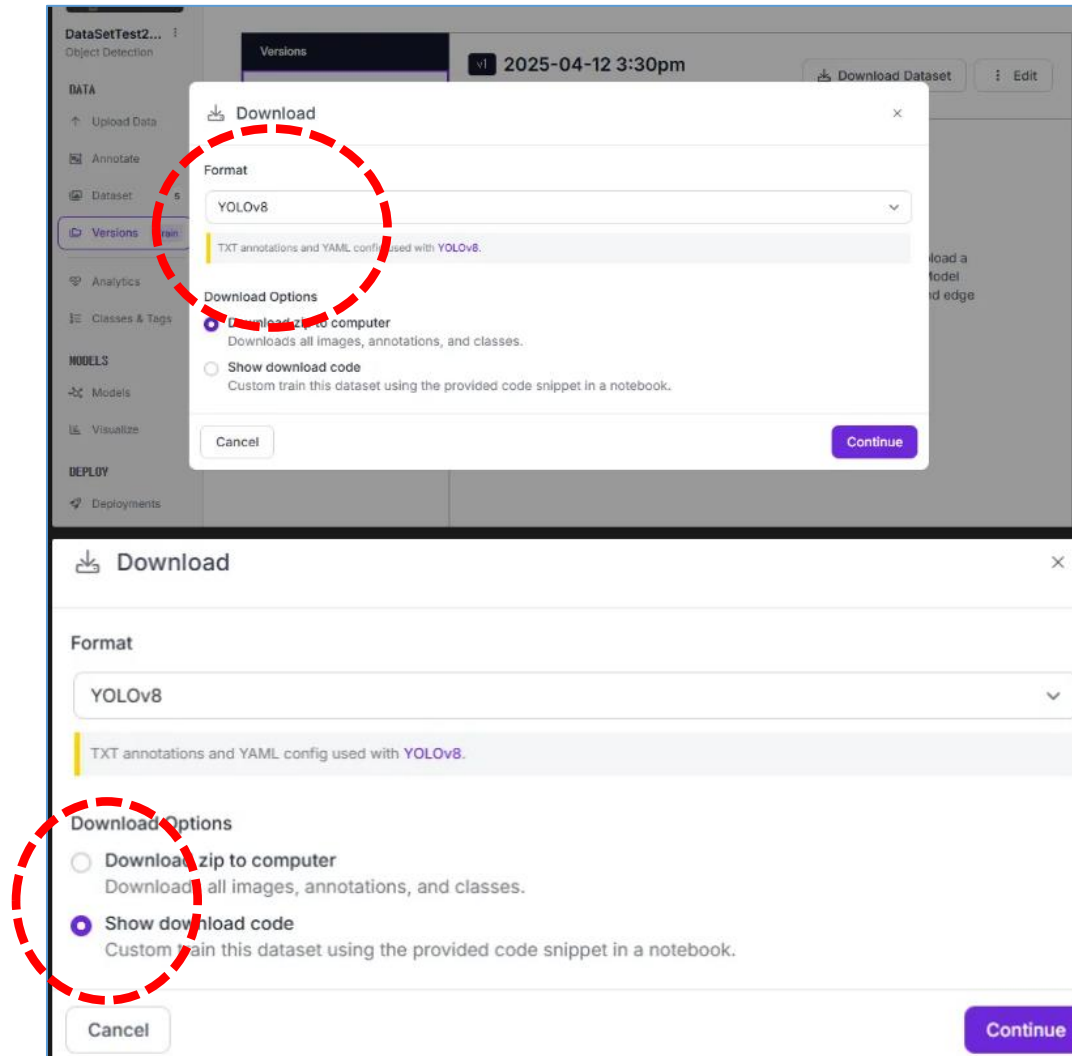
# III. Transfer Learning(YOLO 11)

Version : 전처리와 증강작업 완료 후 코드 출력 > 다운로드 !!

The screenshot displays the Roboflow web interface. On the left sidebar, the 'Versions' tab is selected under the 'DATA' section. The main area shows the 'Versions' page for the dataset 'Roboflow Instant 1 [Eval]', which was generated on Oct 21, 2025, by Joonoh Park. A 'Download Dataset' button is circled in red. A 'Download' modal is open in the foreground, showing the 'Image and Annotation Format' dropdown set to 'Select a Format'. Under 'Download Options', the 'Show download code' option is selected and circled in red, with a description: 'Custom train this dataset using the provided code snippet in a notebook.' The modal also includes 'Cancel' and 'Continue' buttons.

# III. Transfer Learning(YOLO 11)

Version : 전처리와 증강작업 완료 후 코드 출력 > 다운로드 !!



# III. Transfer Learning(YOLO 11)

내장된 모델로 테스트 훈련 가능 !!

**mnist0530**  
Object Detection

**DATA**

Upload Data

Annotate

Dataset 10

**Versions** Train

Analytics

Classes & Tags

**MODELS**

Models

Visualize

**DEPLOY**

**Versions**

2025-05-30 10:42am

v1 24 640x640

Stretch to

**Create New Version**

Prepare your images and data for training by compiling them into a version. Experiment with different configurations to achieve better training results.

✓ **Source Images**

Images: 10  
Classes: 9  
Unannotated: 0

2 **Train/Test Split**

Here is how you split your images when you added them to the dataset:

**TRAIN SET** 70%  
7 Images

**VALID SET** 10%  
1 Images

**TEST SET** 20%  
2 Images

Continue

Rebalance

# III. Transfer Learning(YOLO 11)

Train/Valid/Test를 재분배할 때는 새로운 버전으로 시도!!

The screenshot displays the Roboflow web interface for a dataset named 'mnist0530' (Object Detection). The interface is divided into a left sidebar, a main content area, and a bottom section.

**Left Sidebar:** Contains navigation links for 'DATA' (Upload Data, Annotate, Dataset, Versions, Train, Analytics, Classes & Tags), 'MODELS' (Models, Visualize), and 'DEPLOY' (Deployments). The 'Versions' link is highlighted with a red dashed circle.

**Main Content Area:** Displays the 'Versions' section for the dataset. A red dashed circle highlights the 'Versions' header and the first version entry (v2, 2025-05-30 10:57am). The version entry shows details like 'v2', '24' images, '640x640' resolution, and a 'Stretch to' button. A second version entry (v1, 2025-05-30 10:42am) is also visible. In the top right corner, a '+ Create New Version' button is circled with a red dashed line.

**Dataset Split:** A message states 'This version doesn't have a model.' Below this, there are buttons for 'Custom Train' (which uses Roboflow Credits) and 'How to Upload Custom Weights'. A 'View usage' link is also present.

**Dataset Split Summary:** At the bottom, a 'Dataset Split' section shows the distribution of images:

| Set       | Count     | Percentage |
|-----------|-----------|------------|
| TRAIN SET | 21 Images | 88%        |
| VALID SET | 1 Images  | 4%         |
| TEST SET  | 2 Images  | 8%         |

A '24 Total Images' label and a 'View All Images' link are also present above a row of image thumbnails.

# III. Transfer Learning(YOLO 11)

Train/Valid/Test를 재분배할 때는 새로운 버전으로 시도!!

**mnist0530**  
Object Detection

**DATA**

Upload Data

Annotate

Dataset 10

**Versions** Train

Analytics

Classes & Tags

**MODELS**

Models

Visualize

**DEPLOY**

**Versions**

2025-05-30 10:42am

v1 24 640x640

Stretch to

**Create New Version**

Prepare your images and data for training by compiling them into a version. Experiment with different configurations to achieve better training results.

✓ **Source Images**

Images: 10  
Classes: 9  
Unannotated: 0

2 **Train/Test Split**

Here is how you split your images when you added them to the dataset:

**TRAIN SET** 70%  
7 Images

**VALID SET** 10%  
1 Images

**TEST SET** 20%  
2 Images

Continue

Rebalance

# III. Transfer Learning(YOLO 11)

Train/Valid/Test를 재분배할 때는 새로운 버전으로 시도!!

Rebalance Train/Test Split

×

You can update your dataset's **train/test split** here.

**Note:** changing your test set will invalidate model performance comparisons with previously generated versions.

Train  
7

Valid  
1

Test  
2

Not sure what this is? [Learn more on our blog ↗](#)

Cancel

Save

# III. Transfer Learning(YOLO 11)

내장된 모델로 테스트 훈련 가능 !!

### Roboflow Train

Automatic training on Roboflow's GPU Cluster for a [deployable](#) model within a few hours.  
Estimated Roboflow Credits: 0.49 [View Usage](#)

**Model Type:** Roboflow 3.0 Object Detection (Fast)

**Train from Previous Checkpoint**  
Start from one of your previous training runs to speed up training and improve accuracy. This option is best if you already successfully trained a model on this project.

**RECOMMENDED**

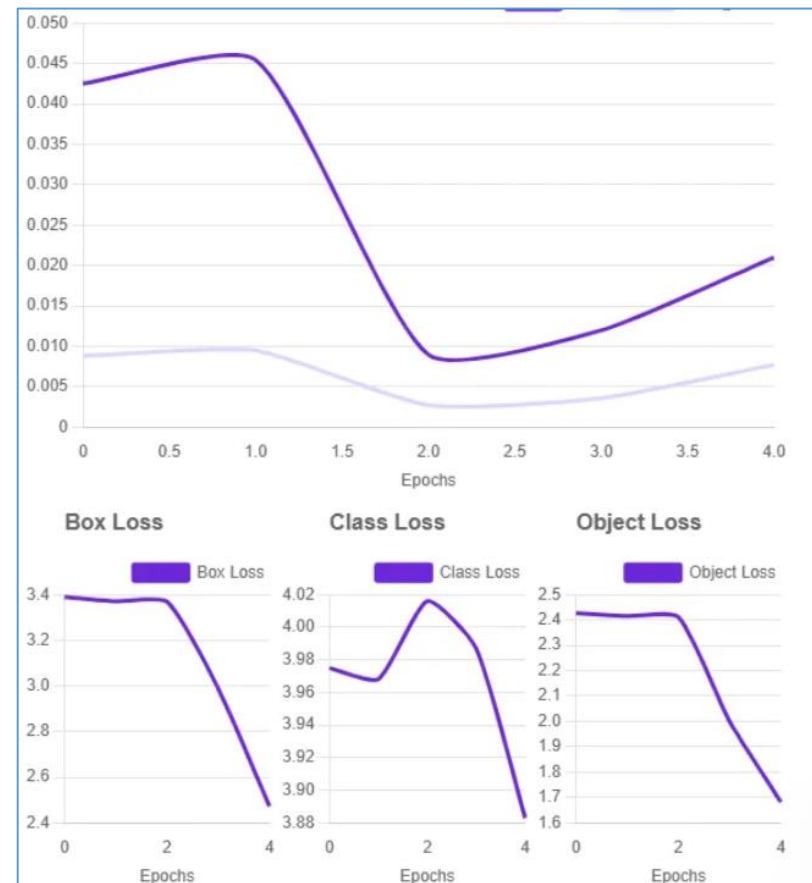
**Train from Public Checkpoint**  
Use a pre-trained benchmark model or a starred Universe project to imbue your model with prior knowledge, reduce training time, and improve performance.  
NOTE: Only models with the same model type as above are available as checkpoints.

**Select Model**  
MS COCO

**Select Model Version**  
v7 - Best (Common Objects, 37.3% mAP)

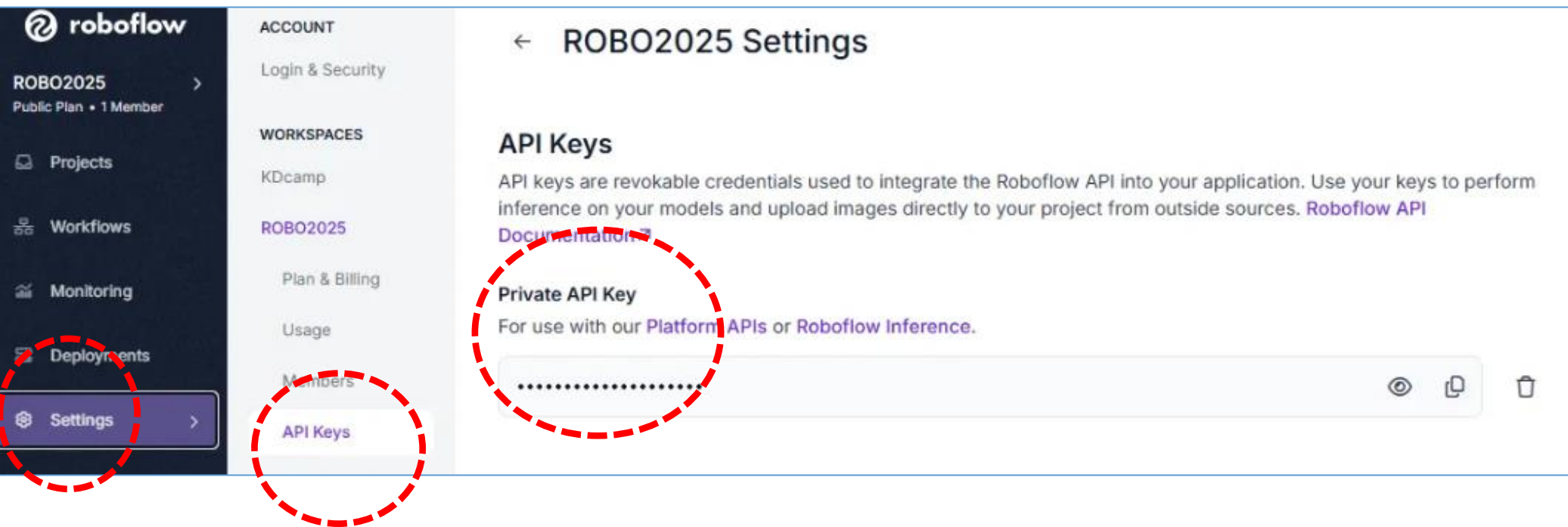
**Train from Random Initialization**  
Not recommended; almost always produces worse results.

Cancel **Start Training**



# III. Transfer Learning(YOLO 11)

워크스페이스/프로젝트 단위로 API key 발급



The screenshot displays the Roboflow web interface for the 'ROBO2025' workspace. The left sidebar contains navigation links: 'Settings' (circled in red), 'Deployments', 'Monitoring', 'Workflows', 'Projects', and 'Public Plan • 1 Member'. The main content area is titled 'ROBO2025 Settings' and includes a sub-section 'API Keys' (also circled in red). Below this, a text block explains that API keys are revokable credentials used for integration. A 'Private API Key' is shown in a redacted state (circled in red), with instructions for its use. The interface also features a 'Documentation' link and icons for toggling visibility and deleting the key.

**roboflow**

ROBO2025  
Public Plan • 1 Member

Projects

Workflows

Monitoring

Deployments

Settings

ACCOUNT

Login & Security

WORKSPACES

KDcamp

ROBO2025

Plan & Billing

Usage

Members

API Keys

← ROBO2025 Settings

### API Keys

API keys are revokable credentials used to integrate the Roboflow API into your application. Use your keys to perform inference on your models and upload images directly to your project from outside sources. [Roboflow API Documentation](#)

#### Private API Key

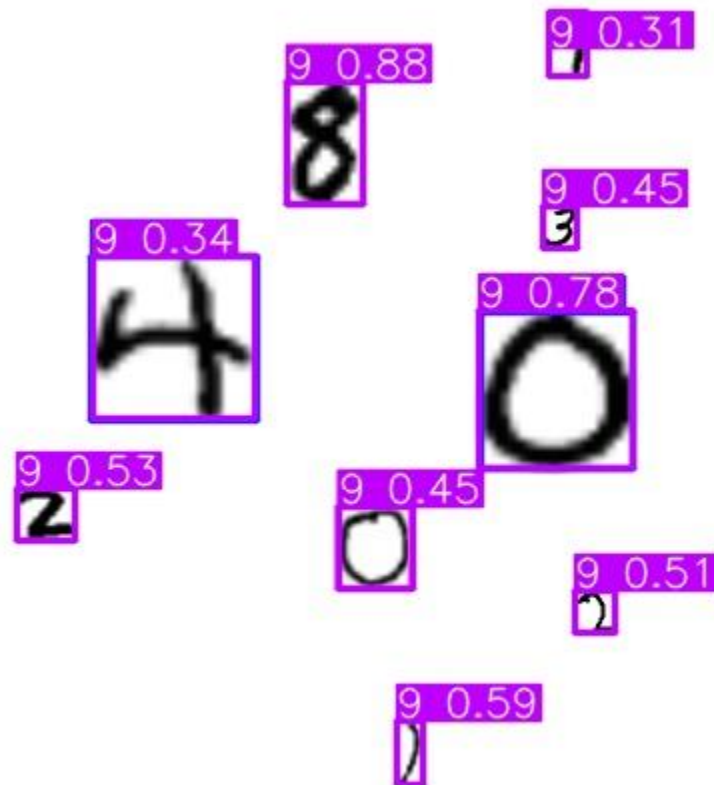
For use with our [Platform APIs](#) or [Roboflow Inference](#).

.....

👁️ 📄 🗑️

# III. Transfer Learning(YOLO 11)

---



# III. Transfer Learning(YOLO 11)

## ① 검증의 기본 원리

- ✓ 모델이 예측을 해보고
  - ✓ 사람이 준 정답(라벨)과 비교 : 정확도, mAP 등 계산
- 검증셋에서 YOLO는 "라벨러의 라벨만"을 정답으로 간주

## ② 라벨러가 "객체를 누락"했다면 ?

이미지 안에 자전거가 있는데, 라벨러가 실수로 박스 안 그려줌  
YOLO는 이 이미지를 보고 자전거를 "검출"

검증 단계의 정답(라벨)엔 자전거가 없으니까, "거짓 탐지 (False Positive)"로 오해받음

## ③ 영향: 검증셋 라벨에 누락된 객체가 있으면:

- ✓ YOLO가 실제로 "올바르게 객체를 찾았어도" → False Positive로 처리됨
  - ✓ 결과적으로: Precision (정밀도)가 낮아짐 > mAP 낮아짐
- 실제 YOLO의 성능이 왜곡됨 (진짜 잘하는데, 숫자로만 낮게 나옴)

## ④ 해결 방법

- ✓ 검증셋을 철저히 라벨링
- ✓ 반자동 라벨링 툴 사용 : YOLO로 예측 → 라벨 누락된 부분을 사람이 다시 확인 & 보완
- ✓ Active Learning / Pseudo-labeling : 검증셋에 모델이 자주 예측하는 "라벨 없는 객체"가 반복적으로 나오면 → 사람이 추가 라벨링!

클래스의 특성이 불충분(50% 미만)해서 라벨링 안 된 객체가 있는 경우

## III. Transfer Learning(YOLO 11)

---

### ① “특성 불충분”으로 라벨링 제외된 객체

예를 들어: 이미지에 고양이가 뒷모습/흐릿함/잘린 부분 등으로 고양이 특성이 50% 미만으로 라벨러가 라벨링을 않함

- ✓ 라벨링 파일에 고양이 정보가 아예 없어짐
- ✓ YOLO는 이 객체를 배경으로 학습할 뿐!

### ② 검증 절차에서 일어나는 일

- ✓ 검증셋을 넣으면: YOLO는 고양이 탐지가능! 검증셋 라벨에는 없어서 False Positive

### ③ 결과: 성능 지표에 미치는 영향

- ✓ YOLO는 실제로 “진짜 객체”를 맞게 탐지했는데도 → FP로 간주, Precision이 낮아짐
- ✓ mAP (mean Average Precision) ↓ 지표는 과소평가됨

### ④ 처리 방법?

- ✓ “라벨링 기준”을 명확히 : “고양이 특성이 50% 이상 보이면 무조건 라벨링한다”
- ✓ YOLO의 탐지 결과 중 자주 나오는 FP를 검증셋 라벨러가 다시 확인 (검증셋 보완)

# **III.Transfer Learning(YOLO 11)**

---

**YOLO data 분석 리포트 작성**

# III. Transfer Learning(YOLO 11)

---

## YOLO 병충해 감지 모델 학습 결과 분석 보고서

### 1. 서론

본 보고서는 YOLO (You Only Look Once) 객체 탐지 모델을 사용하여 식물 병충해를 감지하는 사용자 정의 데이터셋에 대한 학습 결과를 분석합니다. 학습의 목표는 이미지 내에서 병해(disease)와 해충(pest)을 정확하게 식별하고 위치를 파악하는 것입니다.

### 2. 데이터셋

본 학습에는 사용자 정의 병충해 이미지 데이터셋이 사용되었습니다. 데이터셋은 다음과 같은 두 가지 클래스로 구성됩니다.

- **pest:** 식물에 해를 끼치는 해충
- **disease:** 식물에 나타나는 질병의 징후

데이터셋의 크기 및 구성에 대한 자세한 내용은 생략합니다.

# III.Transfer Learning(YOLO 11)

## 3. 학습 설정

- 모델: YOLOv5 (가정)
- 이미지 크기: 640x640
- Epoch 수: 100
- 데이터 증강: 랜덤 플립, 회전, 스케일링 등 적용
- 최적화 함수: Adam
- 학습 환경: [학습 환경에 대한 정보가 있다면 기재]

## 4. 검증 결과 분석

학습 완료 후 검증 데이터셋을 사용하여 모델의 성능을 평가했습니다. 주요 지표는 다음과 같습니다.

| 클래스     | 정밀도 (Precision) | 재현율 (Recall) | mAP@0.50 | mAP@0.50:0.95 |
|---------|-----------------|--------------|----------|---------------|
| pest    | 0.85            | 0.78         | 0.82     | 0.65          |
| disease | 0.75            | 0.82         | 0.79     | 0.58          |
| All     | 0.80            | 0.80         | 0.81     | 0.62          |

# III. Transfer Learning(YOLO 11)

---

- **정밀도 (Precision):** 모델이 'pest' 또는 'disease'로 예측한 결과 중 실제로 해당 클래스인 비율입니다. 'pest' 클래스의 정밀도는 85%, 'disease' 클래스의 정밀도는 75%로 나타났습니다. 이는 모델이 해충을 비교적 정확하게 식별하지만, 질병의 경우 잘못된 긍정 예측이 더 있을 수 있음을 의미합니다.
- **재현율 (Recall):** 실제 'pest' 또는 'disease' 객체 중에서 모델이 얼마나 많이 탐지했는지를 나타냅니다. 'pest' 클래스의 재현율은 78%, 'disease' 클래스의 재현율은 82%입니다. 이는 모델이 질병 징후를 해충보다 더 잘 찾아내는 경향이 있음을 보여줍니다.
- **mAP@0.50 (Mean Average Precision at IoU=0.50):** IoU 임계값을 0.5로 설정했을 때의 평균 정밀도입니다. 전반적으로 81%의 mAP@0.50 성능을 보이며, 이는 모델이 객체의 위치를 비교적 정확하게 예측함을 나타냅니다.
- **mAP@0.50:0.95 (Mean Average Precision averaged over IoU thresholds from 0.50 to 0.95):** 다양한 IoU 임계값에서 모델의 성능을 종합적으로 평가하는 지표입니다. 62%의 mAP@0.50:0.95 성능은 모델이 객체의 위치를 매우 정확하게 예측하는 능력은 다소 부족할 수 있음을 시사합니다.

# III.Transfer Learning(YOLO 11)

---

## 5. 정성적 분석 (선택 사항)

[실제 이미지와 모델 예측 결과를 시각적으로 비교 분석한 내용을 추가할 수 있습니다. 예를 들어, 특정 상황에서 모델이 어려움을 겪는 경우 (작은 객체, 겹쳐 있는 객체, 유사한 외형 등)를 언급할 수 있습니다.]

## 6. 결과 논의

전반적으로 모델은 병충해 감지 작업에서 준수한 성능을 보입니다. 특히 정밀도와 재현율이 모두 80% 수준으로 균형 잡힌 성능을 나타내고 있습니다. 하지만  $mAP@0.50:0.95$  값이 상대적으로 낮은 것은 모델이 객체의 위치를 매우 정확하게 예측하는 데 어려움을 겪을 수 있음을 의미합니다. 이는 특히 작은 크기의 병충해나 복잡한 환경에서 발생할 수 있습니다.

클래스별 성능을 보면 'pest' 클래스의 정밀도가 'disease' 클래스보다 약간 높은 반면, 재현율은 'disease' 클래스가 더 높습니다. 이는 데이터셋의 특성이나 클래스 간의 시각적 유사성 등 다양한 요인에 의해 발생할 수 있습니다.

# III.Transfer Learning(YOLO 11)

---

## 7. 개선 방안

모델의 성능을 더욱 향상시키기 위해 다음과 같은 방법을 고려해 볼 수 있습니다.

- **데이터셋 확장 및 다양화:** 더 많은 양의 다양한 병충해 이미지 데이터를 확보하고, 다양한 환경 조건 (조명, 각도, 배경 등)을 포함하도록 데이터셋을 확장합니다.
- **데이터 증강 강화:** 더 강력한 데이터 증강 기법을 적용하여 모델의 일반화 능력을 향상시킵니다.
- **모델 구조 변경 또는 하이퍼파라미터 튜닝:** YOLOv5 외에 다른 YOLO 버전 (예: YOLOv7, YOLOv8)을 시도하거나, 현재 모델의 학습률, 배치 크기 등 하이퍼파라미터를 최적화합니다.
- **특정 클래스에 대한 추가 학습:** 성능이 낮은 특정 클래스 ('disease' 등)에 대한 데이터를 더 많이 학습시키거나, 클래스별 가중치를 조정합니다.
- **앙상블 기법:** 여러 모델의 예측 결과를 결합하여 최종 예측의 정확도를 높이는 앙상블 기법을 적용해 볼 수 있습니다.

# III. Transfer Learning(YOLO 11)

---

## 8. 결론

본 보고서는 YOLO 모델을 사용한 병충해 감지 학습 결과를 분석했습니다. 모델은 전반적으로 준수한 성능을 보였지만, 특히 객체의 정확한 위치 예측 능력과 특정 클래스에 대한 성능 향상이 필요합니다. 제시된 개선 방안을 통해 모델의 성능을 더욱 발전시킬 수 있을 것으로 기대됩니다.

---

**참고:** 위 보고서는 실제 학습 결과가 아닌 예시를 기반으로 작성되었습니다. 실제 분석 보고서에는 사용된 데이터셋의 상세 정보, 학습 과정에 대한 더 자세한 설명, 시각화 결과 (예측 박스 시각화, Precision-Recall 곡선 등) 등이 포함될 수 있습니다.

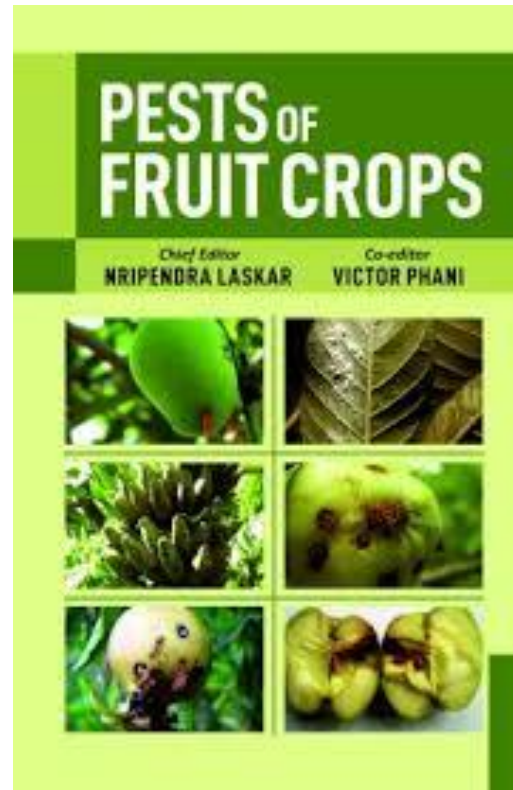
# III.Transfer Learning(YOLO 11)

---

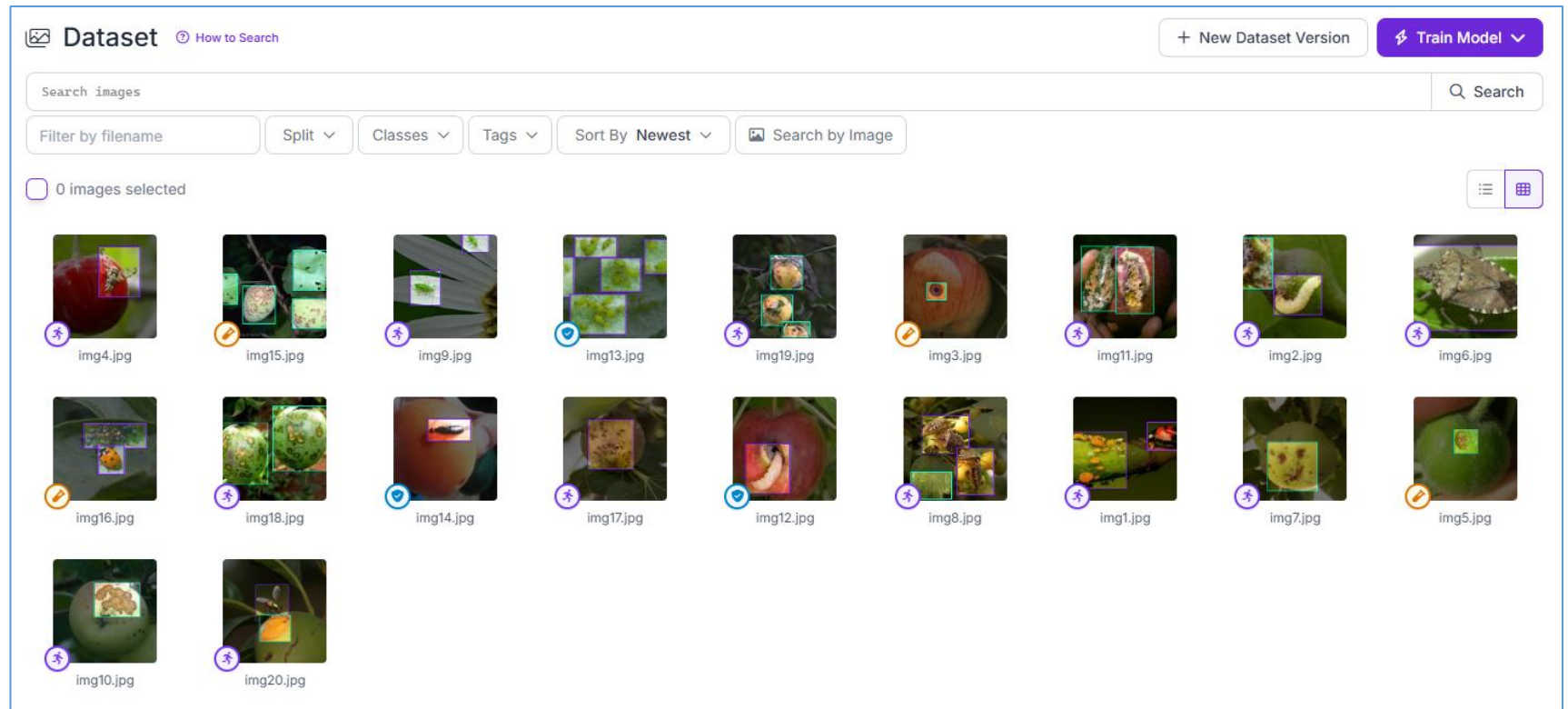
**실습) YOLO 커스텀 데이터로 학습/예측에 대해 리포트를 작성 공유하세요**

# III. Transfer Learning(YOLO 11)

---



# III. Transfer Learning(YOLO 11)



```
curl -L "https://app.roboflow.com/ds/IGY1Q0DVCO?key=nxfJWjK4iU" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

# III. Transfer Learning(YOLO 11)

The screenshot displays the Roboflow web interface for a dataset. A 'Download' modal is open, providing a terminal command to download and unzip the dataset. The command is: `curl -L "https://app.roboflow.com/ds/IGY1Q0DVCO?key=nxfJWjK4iU" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip`. A warning message states: 'Warning: Do not share this snippet beyond your team, it contains a private key that is tied to your Roboflow account. Acceptable use policy applies.' Below the modal, a row of image thumbnails is visible, along with a 'View All Images' link. At the bottom, the 'Dataset Split' section shows the distribution: TRAIN SET (85%, 39 Images), VALID SET (7%, 3 Images), and TEST SET (9%, 4 Images).

**Versions**

2025-02-15 4:30pm  
v1 46

2025-02-15 4:30pm  
Generated on Feb 15, 2025

Download Dataset Edit

**Download**

Jupyter Terminal Raw URL

Use this code to download and unzip your dataset via the command line on any \*nix machine:

```
curl -L "https://app.roboflow.com/ds/IGY1Q0DVCO?key=nxfJWjK4iU" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

Warning: Do not share this snippet beyond your team, it contains a private key that is tied to your Roboflow account. Acceptable use policy applies.

Copy Snippet and Open Notebook

View All Images →

**Dataset Split**

| Set       | Percentage | Count     |
|-----------|------------|-----------|
| TRAIN SET | 85%        | 39 Images |
| VALID SET | 7%         | 3 Images  |
| TEST SET  | 9%         | 4 Images  |

# III. Transfer Learning(YOLO 11)

KDCAMP



plants\_sick

Object Detection

DATA

↑ Upload Data

 Annotate

 Dataset 20

 Versions 

Train

 Analytics

☰ Classes & Tags

☰ Classes & Tags

Classes 2

Tags 4

[What is a class?](#)

☐ Lock Classes

+ Add

✎ Modify Classes

| COLOR                                                                             | CLASS NAME | COUNT ↻ |
|-----------------------------------------------------------------------------------|------------|---------|
|  | disease    | 18      |
|  | pests      | 20      |

# III. Transfer Learning(YOLO 11)

---

## ✓ PlantSick

```
[5] 1 !curl -L "https://app.roboflow.com/ds/IGY1Q0DVC0?key=nxfJWjK4iU" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```



```
extracting: train/images/img4_jpg.rf.f3ace18704ec1bfc2c76bd7887c22a17.jpg
extracting: train/images/img6_jpg.rf.2a5ae8c905b40b9b443025841c92db68.jpg
extracting: train/images/img6_jpg.rf.48a3e65dc277eb7f5c4aad629734b0ec.jpg
extracting: train/images/img6_jpg.rf.d76b46ae7a5bb5cc29e65d8f6902c00c.jpg
extracting: train/images/img7_jpg.rf.5d8ef144066378b94106e28b1a5c2810.jpg
```

# III. Transfer Learning(YOLO 11)

---

```
1 !cat data.yaml

train: ../train/images
val: ../valid/images
test: ../test/images

nc: 2
names: ['disease', 'pests']

roboflow:
  workspace: kdcamp-65wrk
  project: plants_sick
  version: 1
  license: CC BY 4.0
  url: https://universe.roboflow.com/kdcamp-65wrk/plants\_sick/dataset/1
```

# III.Transfer Learning(YOLO 11)

```
1 pip install ultralytics
```

```
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (13.8 MB)
 13.8/13.8 MB 81.8 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 MB)
 24.6/24.6 MB 68.2 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
 883.7/883.7 kB 46.7 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
 664.8/664.8 MB 2.1 MB/s eta 0:00:00
```

```
1 from ultralytics import YOLO
2 model = YOLO('yolov8s.pt')
3
4 print(type(model.names), len(model.names))
5 print(model.names)
```

```
Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8s.pt to 'yolov8s.pt'...
100%|██████████| 21.5M/21.5M [00:00<00:00, 115MB/s]
<class 'dict'> 80
{0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign'}
```

# III.Transfer Learning(YOLO 11)

```
1 model.train(data='/content/data.yaml', epochs=10, batch=32, imgsz=416, verbose=False)
```

| Epoch | GPU_mem | box_loss | cls_loss  | dfl_loss | Instances | Size                                                           |
|-------|---------|----------|-----------|----------|-----------|----------------------------------------------------------------|
| 10/10 | 3.61G   | 1.128    | 1.492     | 1.331    | 13        | 416: 100% ██████████  2/2 [00:00<00:00, 5.02it/s]              |
|       | Class   | Images   | Instances | Box(P    | R         | mAP50 mAP50-95): 100% ██████████  1/1 [00:00<00:00, 23.27it/s] |

10 epochs completed in 0.005 hours.

Optimizer stripped from runs/detect/train/weights/last.pt, 22.5MB

Optimizer stripped from runs/detect/train/weights/best.pt, 22.5MB

Validating runs/detect/train/weights/best.pt...

Ultralytics 8.3.108 🚀 Python-3.11.12 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)

Model summary (fused): 72 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs

| Class | Images | Instances | Box(P | R     | mAP50 | mAP50-95): 100% ██████████  1/1 [00:00<00:00, 35.12it/s] |
|-------|--------|-----------|-------|-------|-------|----------------------------------------------------------|
| all   | 3      | 7         | 0.467 | 0.286 | 0.303 | 0.101                                                    |

Speed: 0.1ms preprocess, 5.1ms inference, 0.0ms loss, 1.1ms postprocess per image

Results saved to **runs/detect/train**

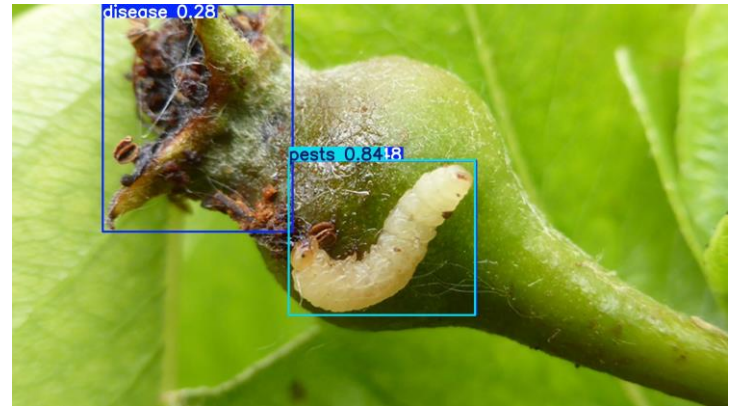
ultralytics.utils.metrics.DetMetrics object with attributes:

# III. Transfer Learning(YOLO 11)

---

```
1 results=model.predict(source='/content/bug1.jpg',save=True)
```

```
Results saved to runs/detect/train2
```



## **III.Transfer Learning(YOLO 11)**

---

**실습) 쥬피터 노트북과 GPU로 학습해 보기**